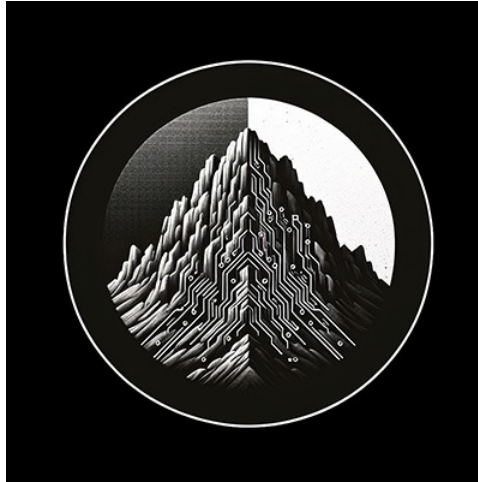




RTL Design Sherpa

DDR2 / LPDDR2 Family Controller Hardware Architecture Specification 0.2

June 13, 2026

Table of Contents

1 Ddr2 Lpddr2 Has Index.....	20
2 Document Information.....	20
2.1 DDR2 / LPDDR2 Family Controller Hardware Architecture Specification.....	20
2.2 Document Purpose.....	20
2.3 Revision History.....	21
2.4 Scope of This Document.....	21
2.5 Out of Scope.....	21
3 Purpose and Scope.....	22
3.1 Document Purpose.....	22
3.2 Scope.....	22
3.3 Design Philosophy.....	23
3.4 Audience.....	23
4 Document Conventions.....	24
4.1 Typographic Conventions.....	24
4.2 Numbering Conventions.....	24
4.3 Signal Direction Conventions.....	24
4.4 Module Diagram Conventions.....	25
4.5 Cross-Reference Conventions.....	25
5 Definitions and Acronyms.....	25
5.1 Acronyms.....	25
5.2 Defined Terms.....	28
6 Scope and Goals.....	29

6.1 High-Level Goals.....	29
6.2 In Scope.....	29
6.3 Out of Scope.....	30
6.4 Target Performance Envelope.....	31
6.5 Differentiating Features.....	31
6.5.1 Features Adopted from Standard Memory-Controller Patterns.....	33
7 Block Diagram.....	33
7.1 Top-Level Block Diagram.....	33
7.2 Data Flow Summary.....	34
8 Module Hierarchy.....	35
8.1 Hierarchy Tree.....	35
8.2 Module Functional Groupings.....	35
8.3 Memtype-Conditional Modules.....	36
8.4 Optional Modules.....	36
9 Top-Level Interfaces.....	37
9.1 Parameter-Dependent Widths.....	37
9.2 1. AXI4 Slave (Host-side).....	38
9.2.1 Write Address Channel (AW).....	38
9.2.2 Write Data Channel (W).....	38
9.2.3 Write Response Channel (B).....	38
9.2.4 Read Address Channel (AR).....	39
9.2.5 Read Data Channel (R).....	39
9.3 2. DFI v2.1 Master (PHY-side).....	40

9.3.1 DFI Command Sub-Interface (Per-Phase).....	40
9.3.2 DFI Write-Data Sub-Interface.....	40
9.3.3 DFI Read-Data Sub-Interface.....	41
9.3.4 DFI Status and Update Sub-Interface.....	41
9.4 3. APB CSR Slave (Configuration / Observation).....	41
9.5 4. Clocks, Resets, Interrupts, Debug.....	42
9.6 Hierarchical Top-Level View.....	42
10 FUB Breakdown (First-Pass SWAG).....	43
10.1 Stream FUB Convention.....	43
10.2 FUB Inventory.....	44
10.2.1 Group A — AXI Frontend.....	44
10.2.2 Group B — Transaction Buffering and Scheduling.....	44
10.2.3 Group C — Per-Bank State and Cross-Bank Timing.....	45
10.2.4 Group D — Refresh, Init, Power.....	46
10.2.5 Group E — Command Encoding and Output Gear.....	46
10.2.6 Group F — Data Paths.....	47
10.2.7 Group G — CSR.....	47
10.3 FUB Count Summary.....	47
10.4 Integration.....	48
11 AXI4 Frontend.....	48
11.1 axi4_slave.....	48
11.1.1 Purpose.....	48
11.1.2 Internal State.....	48

11.1.3 Behavior.....	49
11.1.4 Per-ID Ordering.....	49
11.1.5 Burst Splitting.....	49
11.1.6 Backpressure.....	49
11.2 addr_mapper.....	50
11.2.1 Purpose.....	50
11.2.2 Interfaces.....	50
11.2.3 Mapping Function.....	50
11.2.4 Alternative Mappings.....	50
11.2.5 Relationship to the DFI BFM.....	51
12 Scheduler and Page Predictor.....	51
12.1 txn_queue.....	51
12.1.1 Purpose.....	51
12.1.2 Entry Format.....	51
12.1.3 Depth.....	52
12.1.4 Behavior.....	52
12.2 scheduler.....	52
12.2.1 Purpose.....	52
12.2.2 Inputs.....	52
12.2.3 Outputs.....	52
12.2.4 Priority Function.....	53
12.2.5 Strict In-Order Mode.....	53
12.2.6 Auto-Precharge Decision: Lookahead + Page Policy.....	54

12.2.7 Issue Slot Allocation.....	57
12.3 page_predictor (HAPPY hybrid, optional).....	57
12.3.1 Purpose.....	57
12.3.2 Algorithm.....	57
12.3.3 HAPPY Predictor Architecture.....	58
12.3.4 Storage.....	58
12.3.5 Reset Behavior.....	58
12.3.6 Observability.....	59
13 Bank Machines and Cross-Bank Timers.....	59
13.1 bank_machine.....	59
13.1.1 Purpose.....	59
13.1.2 Instantiation.....	59
13.1.3 FSM States.....	59
13.1.4 FSM Diagram.....	60
13.1.5 Per-Bank Registers.....	60
13.1.6 Outputs to Scheduler.....	61
13.1.7 Refresh Handshake Interface.....	61
13.1.8 Inputs from Cross-Bank Timers.....	62
13.1.9 Auto-Precharge Handling.....	62
13.2 xbank_timers.....	62
13.2.1 Purpose.....	62
13.2.2 Tracked Constraints.....	62
13.2.3 Implementation.....	62

13.2.4 Outputs.....	63
13.2.5 Why Shared Rather Than Per-Bank.....	63
13.2.6 Per-Rank vs. Cross-Rank Scope.....	63
14 Refresh Manager.....	64
14.1 refresh_mgr.....	64
14.1.1 Purpose.....	64
14.1.2 Internal State.....	64
14.1.3 Postponer Behavior (Deterministic Batching).....	65
14.1.4 Temperature-Compensated Refresh (LPDDR2).....	65
14.1.5 Controller-Level Refresh FSM.....	65
14.1.6 Periodic ZQCS Piggybacked on Refresh.....	67
14.1.7 Temperature-Compensated Refresh (LPDDR2).....	67
14.1.8 Refresh Decision Flow.....	68
14.2 REFab Path (DDR2 or LPDDR2 fallback).....	70
14.3 REFpb Path (LPDDR2, default).....	70
14.3.1 Bank Selection Policy.....	70
14.3.2 DARP Rationale.....	71
14.3.3 REFpb Issue Sequence.....	71
14.4 Refresh Priority to Scheduler.....	71
14.5 Self-Refresh Coordination.....	72
14.6 PASR Update Protocol.....	72
15 Init Engine and Power State.....	72
15.1 init_engine and Step Tables.....	73

15.1.1 Purpose.....	73
15.1.2 Microprogram Instruction Format.....	73
15.1.3 Step Tables.....	73
15.1.4 Init Sequence Outlines.....	73
15.1.5 Outputs.....	74
15.1.6 Init Engine Flowchart.....	74
15.1.7 SIM_INIT_SCALE.....	74
15.1.8 Restart on Warm Reset.....	75
15.1.9 Error Handling.....	75
15.1.10 MR Value Configurability.....	75
15.2 power_state (Top-Level FSM).....	75
15.2.1 Purpose.....	75
15.2.2 FSM States.....	75
15.2.3 Power State Diagram.....	76
15.2.4 Transitions.....	76
15.2.5 Step Tables for Power Transitions.....	76
15.2.6 DPD Entry Constraints.....	77
15.2.7 Self-Refresh Exit Latency.....	77
15.2.8 Power-State Observation.....	77
16 Command Encoder and Gear Output.....	77
16.1 cmd_encoder.....	77
16.1.1 Purpose.....	77
16.1.2 Input Record.....	78

16.1.3 DDR2 Encoder.....	78
16.1.4 LPDDR2 Encoder.....	78
16.1.5 Memtype Selection.....	79
16.1.6 Idle Defaults.....	79
16.1.7 Verification.....	79
16.2 gear_out.....	79
16.2.1 Purpose.....	79
16.2.2 Behavior.....	79
16.2.3 Multi-Cycle Commands.....	80
16.2.4 Phase Naming.....	80
16.2.5 Write / Read Data Phase Assignment.....	80
16.2.6 Idle Phase Drives.....	80
16.2.7 Verification.....	80
17 Write and Read Data Paths.....	81
17.1 wdata_path.....	81
17.1.1 Purpose.....	81
17.1.2 Inputs.....	81
17.1.3 Outputs.....	81
17.1.4 Behavior.....	81
17.1.5 Write Command Pipeline.....	82
17.1.6 B-Response Generation.....	82
17.1.7 Write Cancellation.....	82
17.2 rdata_path.....	82

17.2.1 Purpose.....	82
17.2.2 Inputs.....	83
17.2.3 Outputs.....	83
17.2.4 Behavior.....	83
17.2.5 Read Command Pipeline.....	84
17.2.6 Out-of-Order Completion.....	84
17.2.7 Read Error Handling.....	84
17.2.8 Read Data Capture Timing.....	84
17.2.9 Stale Read Suppression.....	85
18 DFI Master and CSR Slave.....	85
18.1 dfi_master.....	85
18.1.1 Purpose.....	85
18.1.2 Sub-Interfaces Driven.....	85
18.1.3 Sub-Interfaces Tied Idle.....	85
18.1.4 Signal Aggregation.....	86
18.1.5 Why a Dedicated Module.....	86
18.2 csr_slave.....	86
18.2.1 Purpose.....	86
18.2.2 Bus Protocol.....	86
18.2.3 Address Map Overview.....	87
18.2.4 Clock Domain Crossing.....	87
18.2.5 Read Latency.....	87
18.2.6 Configuration Quiet Period.....	87

18.2.7 Recommended Programming Sequence.....	87
19 AXI4 Slave Interface.....	88
19.1 Channels.....	88
19.1.1 AW (Write Address) — SoC master → controller slave.....	88
19.1.2 W (Write Data).....	89
19.1.3 B (Write Response).....	89
19.1.4 AR (Read Address).....	89
19.1.5 R (Read Data).....	90
19.2 Ordering Rules.....	90
19.3 Burst Type Support.....	90
19.4 Burst Length.....	90
19.5 Narrow Transfers.....	91
19.6 Posted Writes.....	91
19.7 Backpressure.....	91
19.8 AXI4 Reset.....	91
20 DFI v2.1 Master Interface.....	91
20.1 Per-Phase Signal Naming.....	91
20.2 Command Interface (MC → PHY).....	92
20.3 Write Data Interface (MC → PHY).....	92
20.4 Read Data Interface (PHY ↔ MC).....	92
20.5 Status Interface.....	93
20.6 Sub-Interfaces NOT Driven.....	93
20.7 Reset.....	93

20.8 Timing Parameters Honored.....	93
20.9 Clock Domain.....	94
21 APB CSR Slave.....	94
21.1 Signals.....	94
21.2 APB3 Conformance.....	94
21.3 Error Behavior.....	94
21.4 Clock Domain Crossing.....	95
21.5 Register Map.....	95
21.6 Configuration Sequencing.....	95
22 Build-Time vs. Runtime Configuration.....	96
22.1 Principle.....	96
22.2 Why This Distinction Matters.....	96
22.3 The Hybrid Pattern: “Build-Time MAX + Runtime Active”.....	97
22.4 The “Build-Time + Runtime Mux” Pattern.....	97
22.5 The “Runtime Only” Pattern.....	98
22.6 Configuration Quiet Points.....	98
22.7 Summary.....	98
23 Top-Level Parameter Table.....	99
23.1 Parameter Table.....	99
23.2 Memtype-Dependent Constraints.....	104
23.3 Timing-Configuration Sanity Checks.....	105
23.4 Parameter Categories.....	105
24 Characterization Knobs.....	106

24.1 Characterization Parameters.....	106
24.2 Sweep Plan.....	107
24.2.1 Benchmark Suite.....	107
24.2.2 Metrics Collected.....	108
24.2.3 Default Selection.....	108
24.3 Runtime Overrides via CSR.....	108
24.4 Observability for Sweep.....	109
25 Family-Wide Config Bits.....	109
25.1 Why a Family-Wide Registry.....	109
25.2 Applicability Notation.....	110
25.3 SCHED_TUNING (Scheduling Family Bits).....	110
25.4 REFRESH_TUNING (Refresh Family Bits).....	111
25.5 ADDR_MAP_TUNING (Address-Mapping Family Bits).....	111
25.6 INIT_TUNING (Init Family Bits).....	112
25.7 POWER_TUNING (Power-State Family Bits).....	112
25.8 ECC_TUNING (Future — Inline ECC).....	113
25.9 RANK_TUNING (Multi-Rank Family Bits).....	113
25.10 Quiet-Point Behavior.....	113
25.11 Bit Discovery via the Capability Vector.....	114
25.12 Per-Flavor Applicability Matrix.....	114
26 Clocks and Reset.....	115
26.1 Clock Domains.....	115
26.1.1 mc_clk.....	115

26.1.2	phy_clk.....	115
26.1.3	apb_pclk.....	116
26.2	Reset Signals.....	116
26.2.1	mc_rstn (active low).....	116
26.2.2	phy_rstn (active low).....	116
26.2.3	dfi_rstn (active low, MC → PHY).....	116
26.2.4	apb_presetn (active low).....	116
26.3	Reset Sequencing.....	116
26.3.1	Cold Boot.....	116
26.3.2	Warm Reset (Clock-Gated).....	117
26.4	Clock Domain Crossing Summary.....	117
26.5	Reset and Power Considerations.....	117
27	Init Sequences.....	118
27.1	DDR2 Cold Init.....	118
27.2	LPDDR2 Cold Init.....	119
27.3	Self-Refresh Entry (Both Memtypes).....	119
27.4	Self-Refresh Exit (Both Memtypes).....	120
27.5	LPDDR2 Deep Power Down Entry.....	120
27.6	LPDDR2 Deep Power Down Exit.....	120
27.7	Sim Scaling.....	120
28	CSR Register Map.....	121
28.1	Control and Status (0x000 – 0x00F).....	121
28.1.1	CTRL (0x000, R/W).....	121

28.1.2 STATUS (0x004, R only).....	121
28.1.3 STATUS_HISTORY (0x008, R only).....	121
28.2 Timing Parameters (0x010 – 0x01F).....	121
28.2.1 TIMINGS_RC_RCD_RP (0x010, R/W).....	122
28.2.2 TIMINGS_RAS_RFC_REFI (0x014, R/W).....	122
28.2.3 TIMINGS_RRD_FAW_WTR (0x018, R/W).....	122
28.2.4 TIMINGS_CL_CWL_WR (0x01C, R/W).....	122
28.3 Mode Register Values (0x020 – 0x02F).....	122
28.3.1 MR0, MR1, MR2, MR3 (0x020, 0x024, 0x028, 0x02C, R/W).....	122
28.4 LPDDR2-Specific (0x030 – 0x03F).....	123
28.4.1 PASR_BANK_MASK_RANK<R> (0x030 + R×4, R/W; R = 0 .. NUM_RANKS–1).....	123
28.4.2 PASR_SEG_MASK_RANK<R> (0x080 + R×4 within the multi-rank reserved area is reused — see register layout note).....	123
28.4.3 TEMP_DERATE_RANK<R> (0x038 + R×4, R only).....	123
28.5 Scheduler / Refresh / Page Tuning (0x040 – 0x04F).....	123
28.5.1 SCHED_TUNING (0x040, R/W).....	123
28.5.2 PAGE_PRED_TUNING (0x044, R/W, only when HAPPY enabled).....	124
28.5.3 REFRESH_TUNING (0x048, R/W).....	124
28.5.4 ADDR_MAP_TUNING (0x04C, R/W).....	125
28.5.5 INIT_TUNING (0x050, R/W).....	125
28.6 Per-(Rank, Bank) Observation (0x080 – 0x0FF).....	125
28.6.1 OBS_ROW_HIT_RANK<R>_BANK<N> (0x080 + (R × NUM_BANKS + N) × 4, R only).....	125

28.6.2 OBS_REF_LATENCY_RANK<R>_BANK<N> (0x0C0 + (R × NUM_BANKS + N) × 4, R only).....	126
28.7 System Observation (0x100 – 0x1FF).....	126
28.8 Module Identification (0xFF0 – 0xFFC).....	127
28.8.1 ID (0xFF0, R only).....	127
28.8.2 BUILD (0xFF4, R only).....	127
29 Verification Strategy.....	127
29.1 Layered Approach.....	127
29.1.1 Layer 1: Module-Level Unit Tests.....	127
29.1.2 Layer 2: Subsystem Tests.....	128
29.1.3 Layer 3: End-to-End with DFI BFM Slave.....	128
29.1.4 Layer 4: External-Reference Cross-Validation.....	129
29.2 Characterization Sweep.....	129
29.2.1 Sweep Matrix.....	129
29.2.2 Sweep Outputs.....	129
29.2.3 Out-of-Order Scheduler Trade-Offs.....	130
29.3 Verification Hooks.....	131
29.4 Coverage Targets.....	131
30 Open Issues and TODOs.....	132
30.1 Open Issues.....	132
30.1.1 1. AXI4 Narrow-Burst Handling Confirmation.....	132
30.1.2 2. AXI4 Read-Modify-Write Strobe Handling.....	132
30.1.3 3. APB CSR Address Space Allocation.....	132
30.1.4 4. Per-Bank Refresh Book-Keeping Cost.....	132

30.1.5 5. HAPPY Predictor Hash Function.....	132
30.1.6 6. Self-Refresh Exit Latency.....	132
30.1.7 7. Init Engine Retry on Error.....	133
30.1.8 8. CSR Write Atomicity.....	133
30.1.9 9. Burst Splitting at Row Boundaries.....	133
30.1.10 10. Multi-Rank.....	133
30.1.11 11. ECC.....	133
30.1.12 12. DFI Training Sub-Interface.....	133
30.1.13 13. AXI Quality of Service (QoS).....	133
30.1.14 14. Multi-Master AXI.....	133
30.1.15 15. Observation Counter Overflow.....	134
30.2 TODOs Before v0.2.....	134

List of Figures

No figures in this document.

List of Tables

No tables in this document.

1 Ddr2 Lpddr2 Has Index

Generated: 2026-06-14

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

2 Document Information

2.1 DDR2 / LPDDR2 Family Controller Hardware Architecture Specification

Document Number: DDR2-LPDDR2-HAS-001 **Version:** 0.2 **Status:** Draft - First Sketch

Classification: Open Source - Apache 2.0 License

2.2 Document Purpose

This Hardware Architecture Specification (HAS) provides a high-level architectural overview of the DDR2 / LPDDR2 unified memory controller family. It describes the system-level design, external interfaces, characterization parameters, and integration requirements without detailing internal implementation specifics.

Target Audience:

- System architects evaluating the controller for SoC integration
- Hardware engineers planning RTL implementation
- Verification engineers planning system-level testing
- Software engineers developing low-level memory configuration

Companion Documents:

- DDR2/LPDDR2 Family Controller Pre-Architecture Spec (`../pre-aspec.md`) — bullet-form architecture rationale
 - DDR2/LPDDR2 Paging and Refresh Notes (`../paging-refresh-notes.md`) — research-derived parameter sources
-

2.3 Revision History

Version	Date	Author	Notes
0.1	2026-06-13	RTL Design Sherpa	First sketch — module decomposition
0.2	2026-06-14	RTL Design Sherpa	Top-level interface port list (§2.4), FUB SWAG (§2.5), family-wide config-bit registry (§5.4), multi-rank support across §2 / §3.3 / §3.4 / §3.6 / §5.2 / §6.3

2.4 Scope of This Document

This HAS describes a single parameterized memory controller covering DDR2 and LPDDR2. It defines:

- External interfaces (AXI4 slave, DFI v2.1 master, APB CSR slave)
- Module-level decomposition and behavioral specification
- Build-time and run-time configuration parameters
- Initialization, power-state, and refresh policies
- Verification strategy and characterization plan

It does not define:

- Bit-level pin assignments (the SystemVerilog port list in `regs/ddr2_lpddr2_ctrl_ports.svh` is the canonical wire-level source)
 - Detailed timing diagrams (deferred to v0.3)
 - Verilog package skeletons or RTL stubs
 - Floorplan or layout guidance
-

2.5 Out of Scope

The following features are explicitly out of scope for this version of the controller and will be addressed in higher-generation family controllers or future revisions:

- Inline ECC
- DFI training, frequency-change, and low-power sub-interfaces

- AXI exclusive access semantics
- Bank groups (not present in DDR2 / LPDDR2)
- Command/Address parity, CRC, Data Bus Inversion (not present in DDR2 / LPDDR2)

3 Purpose and Scope

3.1 Document Purpose

This Hardware Architecture Specification (HAS) defines the high-level architecture of the DDR2 / LPDDR2 Family Memory Controller. It serves as the primary reference for:

- **System Integration** — understanding the controller’s external interfaces and system-level requirements
- **RTL Implementation Planning** — module decomposition, interface contracts, and parameter sweeps
- **Verification Planning** — system-level test scenarios and the characterization sweep matrix
- **Driver Development** — programming model and APB register interface

This document complements the upcoming Micro-Architecture Specification (MAS), which will provide detailed block-level implementation specifics, signal-level pinouts, and timing diagrams.

3.2 Scope

This HAS covers a single unified controller that supports both DDR2 and LPDDR2 memory via a build-time MEMTYPE parameter. The shared engine includes:

- AXI4 slave frontend
- Address mapper
- Transaction queue with row-hit caching
- FR-FCFS scheduler with parameterizable page policy

- Per-bank state machines with local timing enforcement
- Cross-bank timer pool (tRRD / tFAW / tWTR / tCCD)
- Refresh manager with Elastic deferral and DARP-style per-bank scheduling
- Top-level power-state FSM
- Microprogram-based initialization engine
- Pluggable command encoder (DDR2 ras/cas/we OR LPDDR2 CA bus)
- N-phase output gear stage ($N_PHASES \in \{1, 2, 4\}$)
- AXI write / read data paths with CWL / CL alignment
- DFI v2.1 master driver
- APB CSR slave for configuration and observation

The only differences between DDR2 and LPDDR2 build configurations are the command encoder, the init step table, the power-state FSM (specifically Self-Refresh vs Deep Power Down handling), and a small number of mode-register signals. Everything else is shared.

3.3 Design Philosophy

This controller is intended as a **characterization-first** design. Algorithmic choices that have meaningful research-backed alternatives — page policy, per-bank refresh policy, refresh deferral depth, transaction queue depth — are exposed as parameters rather than hardcoded. The verification plan includes a benchmark sweep over representative AXI traffic patterns to pick defaults from data, not assumption.

The two specific characterization knobs informed by external research are:

- **PAGE_POLICY** $\in \{\text{OPEN}, \text{CLOSE}, \text{HAPPY_HYBRID}\}$ — where HAPPY_HYBRID is the address-bit-based page-closure predictor from Ghasempour et al. (2015).
- **REFPB_POLICY** $\in \{\text{ROUND_ROBIN}, \text{OLDEST_FIRST}, \text{DARP}\}$ — where DARP is the dynamic access refresh parallelization scheme from Chang et al. (HPCA 2014).

These are described in detail in Chapter 5.

3.4 Audience

This document assumes the reader is familiar with:

- AXI4 protocol semantics (handshakes, burst types, ID-based ordering)
- DFI v2.1 specification (control / write-data / read-data sub-interfaces)
- JEDEC DDR2 (JESD79-2F) and LPDDR2 (JESD209-2F) command tables
- Basic memory-controller concepts (open vs closed page policy, refresh deadlines, bank state machines)

Readers new to these topics should first consult the companion `paging-refresh-notes.md` in the family working area for an entry-point bibliography.

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

4 Document Conventions

4.1 Typographic Conventions

Style	Meaning
monospace	Module names, parameter names, signal names, register names
bold	Defined terms on first introduction; important warnings
<i>italic</i>	Emphasis; placeholder values to be replaced
MEMTYPE	Top-level build-time parameter (all caps)
bank_machine	RTL module name (lowercase with underscores)
tRC, tRCD	JEDEC timing parameters (as in JESD79-2 / JESD209-2)

4.2 Numbering Conventions

- **Decimal** numbers have no prefix: 16, 256.
- **Hexadecimal** numbers are prefixed with 0x: 0x800, 0xFF.
- **Binary** literals use Verilog notation when needed: 8'b1011_0010.
- Bit ranges use the convention [MSB:LSB], e.g., dfi_address[19:0].

4.3 Signal Direction Conventions

For DFI signals, direction is from the **memory controller's** perspective:

- **MC → PHY** — driven by the controller
- **PHY → MC** — driven by the PHY (sampled by controller)

For AXI signals, direction follows the standard AXI4 convention with the controller as the **slave**.

4.4 Module Diagram Conventions

ASCII block diagrams in this document use the following conventions:

All diagrams in this document are rendered from Mermaid source files in `assets/mermaid/`. The Mermaid source files (`.mmd`) are linked alongside each rendered image so the diagrams can be edited and re-rendered.

- **Rectangular boxes with sharp corners** denote RTL modules.
- **Subgraph rectangles** group modules by functional layer.
- **Solid arrows** show primary data or control flow.
- **Dashed arrows** show configuration or observation paths (typically from the CSR slave to other modules).
- **Trapezoidal nodes** denote external interface ports (AXI, DFI, APB).

4.5 Cross-Reference Conventions

- References to other chapters: “see Chapter 4”
- References to specific sections: “see §4.1” or “see Section 4.1”
- References to other documents: “see `pre-aspec.md`” (relative path)
- References to external standards: “per JESD79-2F §x.y”

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

5 Definitions and Acronyms

5.1 Acronyms

Acronym	Expansion
AHB	Advanced High-performance Bus (ARM AMBA)

Acronym	Expansion
APB	Advanced Peripheral Bus (ARM AMBA)
AP	Auto-Precharge (DRAM column command modifier)
APD	Active Power Down
AXI	Advanced eXtensible Interface (ARM AMBA)
BL	Burst Length
CA	Command/Address (LPDDR2/3/4 multiplexed bus)
CDC	Clock Domain Crossing
CL	CAS Latency
CMD	Command
CSR	Control/Status Register
CWL	CAS Write Latency
DARP	Dynamic Access Refresh Parallelization
DBI	Data Bus Inversion
DDR	Double Data Rate
DDR2	Double Data Rate 2 (JESD79-2)
DFI	DDR PHY Interface
DPD	Deep Power Down (LPDDR2)
FSM	Finite State Machine
FR-FCFS	First-Ready, First-Come-First-Served
HAPPY	Hybrid Address-based Page Policy in DRAMs
HAS	Hardware Architecture Specification
LPDDR2	Low Power DDR 2 (JESD209-2)
MAS	Micro-Architecture Specification

Acronym	Expansion
MC	Memory Controller
MR / MRS	Mode Register / Mode Register Set command
EMR / EMRS	Extended Mode Register / Extended MRS
NOP	No Operation
ODT	On-Die Termination
OOO	Out-of-Order
PASR	Partial Array Self-Refresh (LPDDR2 feature)
PHY	Physical Layer (DRAM electrical interface)
PRE	Precharge command
PREA	Precharge All Banks
RD	Read column command
RDA	Read with Auto-Precharge
REF	Refresh command
REFab	All-Bank Refresh (LPDDR2)
REFpb	Per-Bank Refresh (LPDDR2)
SDRAM	Synchronous Dynamic RAM
SoC	System on Chip
SR	Self-Refresh
TCSR	Temperature-Compensated Self- Refresh (LPDDR2)
WR	Write column command
WRA	Write with Auto-Precharge
ZQ	Impedance Calibration (ZQ Calibration)

5.2 Defined Terms

Bank Machine — A finite state machine instance dedicated to one DRAM bank that enforces JEDEC timing constraints locally and tracks open-row state.

Characterization Knob — A build-time or runtime parameter intentionally exposed for performance sweeping during system characterization. Page policy and refresh policy are the two principal knobs in this controller.

Command Encoder — The module that translates the controller’s internal generic command record into DFI wire signals. Two implementations: `ddr2_encoder` and `lpddr2_encoder`. Selected at elaboration time via `MEMTYPE`.

FR-FCFS — First-Ready, First-Come-First-Served. The baseline DRAM scheduling policy: ready commands beat unready, row-hits beat row-misses, older entries beat younger on ties. Established by Rixner et al. (ISCA 2000).

Gear Output Stage — The module that replicates a 1-phase internal command bus into `N_PHASES` parallel DFI output phases.

Microprogram Engine — The init sequencer: a small instruction-decoded FSM that reads a memtype-specific ROM (step table) and issues the corresponding control bits, MR loads, and commands.

Page Hit / Row Hit — An access whose target row matches the currently open row in the target bank. Avoids the $t_{RP} + t_{RCD}$ penalty.

Page Conflict / Row Miss — An access whose target row differs from the currently open row in the target bank. Pays $t_{RP} + t_{RCD}$ before the access can begin.

Page Policy — The strategy for when to close an open row. See `PAGE_POLICY` parameter.

Row-Hit Caching — Storing the row-hit / row-miss decision on each transaction at queue insertion time, rather than recomputing each scheduler tick.

Transaction Queue — The pending-request buffer between the AXI frontend and the scheduler.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

6 Scope and Goals

6.1 High-Level Goals

This controller provides a single parameterized RTL implementation supporting both DDR2 and LPDDR2 memory devices via DFI v2.1. Primary design goals:

1. **Unified engine, swappable encoder.** DDR2 and LPDDR2 differ only at the wire-encoding layer (ras/cas/we vs CA bus); the bank machines, scheduler, refresh policy, init engine, and AXI frontend are shared between both targets.
2. **Embedded / mobile / mid-range SoC targets.** Not optimized for high-end server DRAM (DDR5, RDIMM, multi-rank). Optimized for power efficiency and area economy.
3. **Characterization-first parameterization.** Algorithmic choices with research-backed alternatives are exposed as build-time parameters so the design can be swept on real workloads before defaults are locked.
4. **Verification-friendly.** Mirrors the data structures and conventions of the DDR2 / LPDDR2 DFI BFM in the DV repository so the same address mapping, timing parameters, and command encodings can drive both RTL and simulation.

6.2 In Scope

The following features are explicitly within scope for this controller:

- **AXI4 slave** with 64 / 128 / 256-bit data width, 1–16-bit ID width, INCR burst type
- **DFI v2.1 master** driving the command, write-data, and read-data sub-interfaces
- **APB CSR slave** for configuration and observation
- **DDR2 and LPDDR2 device support** via MEMTYPE build parameter
- **DFI frequency ratio** (gear) of 1, 2, or 4 phases via N_PHASES parameter
- **Multi-rank operation** — 1, 2, or 4 ranks via NUM_RANKS parameter; per-rank CS_n, CKE, ODT, and refresh tracking; rank-aware ODT cross-termination rules per JEDEC
- **Page policies:** OPEN, CLOSE, HAPPY hybrid predictor (parameterized)

- **Per-bank refresh** for LPDDR2 with DARP-style scheduling (parameterized)
 - **All-bank refresh** for DDR2 (and as LPDDR2 fallback)
 - **Elastic refresh deferral** up to JEDEC ceiling of $8 \times t_{REFI}$ (parameterized)
 - **Partial-array self-refresh (PASR)** for LPDDR2 with CSR-configurable mask
 - **Temperature-compensated refresh** (LPDDR2 MR4 derating)
 - **Cold-boot initialization** via microprogram step tables
 - **Power state management** (Active, Active Power-Down, Self-Refresh, Deep Power Down)
 - **Bring-up observability** via CSR-exposed counters (row-hit rate, refresh latency, queue depth)
-

6.3 Out of Scope

Explicitly **not** addressed in this controller version:

Feature	Reason
Inline ECC	Out-of-controller responsibility (SoC sideband)
DFI training sub-interface	Not required for v2.1 DDR2 / LPDDR2 operation
DFI frequency-change protocol	No use case for this generation
DFI low-power sub-interface	Power-state FSM uses CKE only
AXI exclusive accesses	Embedded targets don't require
Bank groups	Not present in DDR2 / LPDDR2 (introduced in DDR4 / LPDDR4)
Command/Address parity	Not present in DDR2 / LPDDR2 (introduced in DDR4 / LPDDR4)
Write CRC	Not present in DDR2 / LPDDR2
Data Bus Inversion (DBI)	Not present in DDR2 / LPDDR2
DDR3 write leveling	Not present in DDR2 / LPDDR2 (LPDDR3 introduces)
DDR3 ZQCS / ZQCL on column	DDR2 uses OCD calibration; LPDDR2 uses MR10 ZQ
Higher DDR / LPDDR	Will be addressed in DDR3-LPDDR3 and DDR4-

Feature	Reason
generations	LPDDR4 controllers

6.4 Target Performance Envelope

Metric	Target
DRAM data rate	Up to DDR2-800 / LPDDR2-1066
AXI clock	Up to 200 MHz (typical embedded SoC)
Sustained read bandwidth	$\geq 70\%$ peak on stream workloads
99th-percentile read latency	≤ 200 ns (typical)
Refresh blocking overhead	$< 5\%$ (LPDDR2 with DARP); $< 8\%$ (DDR2)
Area	$< 50K$ gates (excluding HAPPY predictor)

These are target envelopes for v1; not guarantees. They will be validated and refined during the characterization sweep.

6.5 Differentiating Features

This controller is informed by published memory-controller literature and the open-source memory-controller ecosystem (LiteDRAM in particular — used as the cross-validation reference; see §6.4), but it is not a re-implementation of any existing controller. The design’s distinguishing features:

1. **Centralized FR-FCFS scheduler with full out-of-order issue.** A single scheduler scans the full transaction queue every cycle, considering all banks’ readiness and row-hit state in one decision. This is closer to the commercial-vendor pattern (Synopsys, Cadence, Xilinx MIG) than to most open-source RTL controllers — open-source controllers typically expose OoO only at the cross-bank level (a per-bank FIFO model where bank parallelism is the only source of reordering). True per-port within-stream reordering — where a younger row-hit request bypasses an older row-conflict request — is uncommon outside vendor IP. AXI4 per-ID ordering is honored at the response side regardless. **For real-time / safety-critical / formally-verifiable systems**, OoO reordering can be disabled at build time via `SCHEDULER_MODE = INORDER` (§3.2); the scheduler degrades to a first-ready FIFO that preserves arrival order at the cost of 10–25% sustained bandwidth on streaming workloads.
2. **Multi-rank support with JEDEC-compliant cross-rank ODT.** Most open-source DDR2/3 controllers are single-rank only — LiteDRAM’s default config, UberDDR3, and most

academic prototypes assume one rank per channel. This controller supports 1, 2, or 4 ranks via the NUM_RANKS build parameter. Per-rank CS_n, CKE, and ODT signals are driven; the rank-aware refresh manager tracks last_ref_age per (rank, bank); and the cross-rank ODT termination rules (read from rank N → ODT-high on rank ≠N during the read window; write to rank N → ODT-high on rank N during the write window) are handled by an ODT-control sub-module per JESD79-2 §x.y. Multi-rank is what makes the controller usable on real DIMM-class platforms rather than just on-board point-to-point DRAM.

3. **Unified DDR2 + LPDDR2 engine with a swappable command encoder.** A single controller core supports both memtypes via the MEMTYPE build parameter. The LPDDR2 path includes the 20-bit CA bus encoder (DFI v2.1 §3.1 Table 1), PASR mask handling (MR16/17), per-bank refresh (REFpb), temperature-compensated refresh (MR4 derating), and Deep Power Down — none of which are present in the open-source DDR2/3/4 reference controllers.
4. **DARP-style per-bank refresh scheduling** — picks the *idle* bank with the longest time since last refresh, falling back to oldest-first when no banks are idle. Based on Chang et al. (HPCA 2014); not present in available open-source controllers.
5. **Two-stage page management** — exact lookahead first, history-based fallback second.
 - Stage 1: lookahead window over the transaction queue (LOOKAHEAD_DEPTH ∈ {0, 1, 2, 4}) provides exact auto-precharge decisions when the next same-bank request is already queued.
 - Stage 2: when lookahead is inconclusive (shallow queue or bursty traffic), the HAPPY hybrid page predictor (Ghasempour et al. 2015) uses address-bit-hashed saturating counters to predict the next access. Open-source controllers typically have either lookahead OR a fixed page policy. The two-stage combination is original to this design.
6. **Characterization-first parameterization.** Page policy, lookahead depth, refresh policy, refresh deferral, queue depth, address mapping scheme, and HAPPY table size are all build-time parameters with a defined sweep matrix (see §5 and §6.4). Most published controllers expose only a subset of these as parameters; few expose them all.
7. **Closed-page policy as an explicit option.** PAGE_POLICY ∈ {OPEN, CLOSE, HAPPY_HYBRID}. Most open-source controllers expose only on/off auto-precharge (effectively OPEN with optional lookahead). The CLOSE option is useful for adversarial / security-sensitive workloads where row-state side channels matter.
8. **Three-scheme parameterized address mapping** — ADDR_MAP_SCHEME ∈ {ROW_MAJOR, BANK_INTERLEAVE, XOR_HASH}. The XOR_HASH scheme defeats pathological row-conflict patterns and is rarely available out-of-the-box.

9. **AXI4-first frontend** with ID-aware out-of-order completion. The AXI integration is a first-class design constraint rather than a wrapper on a native protocol.
10. **Mirrored verification address-mapping object.** The `addr_mapper` mirrors a Python `AddressMapping` class used by the DFI BFM, so the same mapping function drives both RTL elaboration and simulation. This eliminates a common source of address-decode bugs between RTL and verification.

6.5.1 Features Adopted from Standard Memory-Controller Patterns

These design patterns are widespread in the memory-controller literature and the open-source controllers we examined. Adopting them is the right call; they are noted here so the design rationale is transparent:

- **Per-bank state machines with local timing enforcement** + central cross-bank timer pool (tRRD / tFAW / tWTR / tCCD) — standard.
- **FR-FCFS scheduler baseline** (Rixner et al. ISCA 2000) — the academic baseline.
- **Elastic Refresh deferral** (Stuecheli et al. MICRO 2010) — postponer with deterministic back-to-back drain.
- **Refresh-grant handshake** (`refresh_req` / `refresh_gnt`) between refresh manager and bank machines — established pattern.
- **Periodic ZQCS piggybacked on the refresh window** — established pattern for systems with thermal drift.
- **Microprogram-driven init engine** with per-memtype step-table ROMs — established pattern.

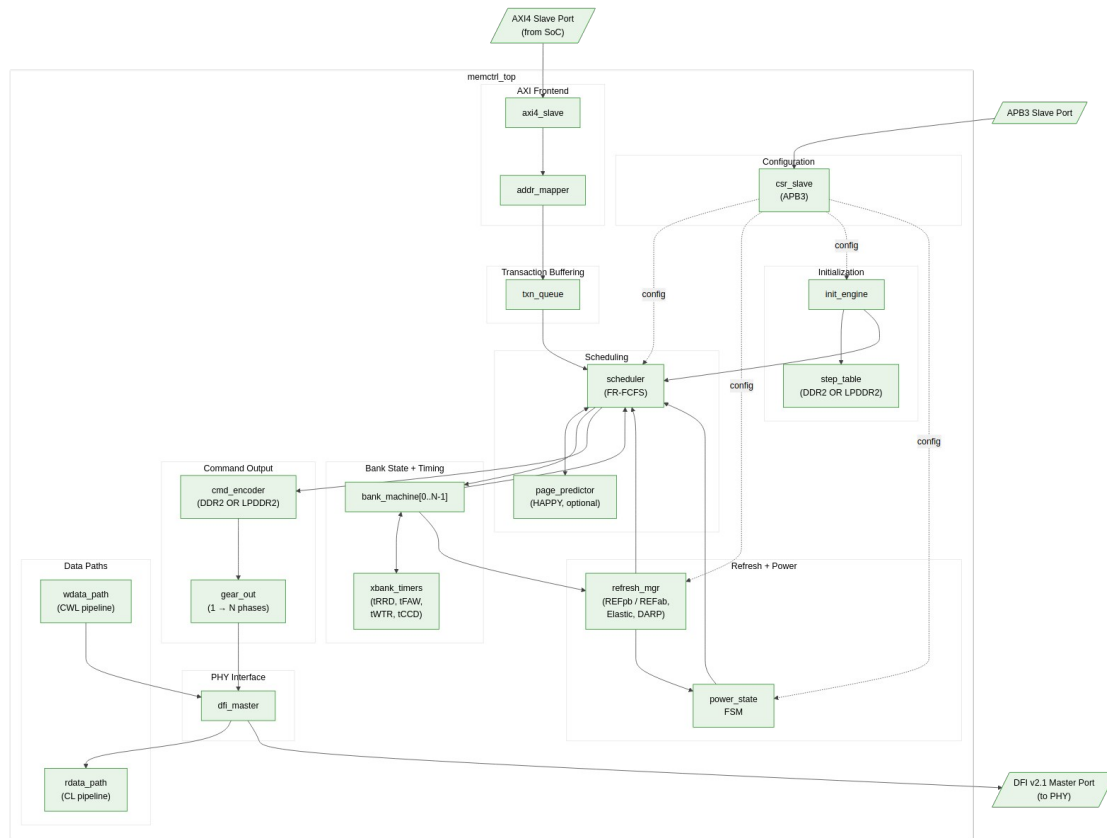
The design rationale for adopting versus extending each is documented in the relevant section.

7 Block Diagram

7.1 Top-Level Block Diagram

The controller's top-level data and control flow is shown below. AXI4 traffic enters at top-left, flows through the address mapper and transaction queue into the scheduler, gets converted to

per-bank commands and routed through the encoder and gear stage out to the DFI PHY interface. The APB CSR slave (bottom) provides configuration and observation in parallel.



Top-Level Block Diagram

Source: [01_block_diagram.mmd](#)

7.2 Data Flow Summary

Write path: 1. AXI master issues an AW transaction; AXI4 slave accepts and queues. 2. The address mapper translates the flat AXI address into (bank, row, column) coordinates. 3. The transaction queue caches the request with row-hit metadata determined from the bank machines' current state. 4. The scheduler picks the next eligible transaction; consults the page predictor if enabled. 5. The scheduler issues ACT, then WR (or WRA) through the command encoder. 6. The gear output stage replicates the 1-phase command onto N parallel DFI phases. 7. CWL cycles after the WR command, the write data path pulls W-channel beats and drives them onto DFI wrdata. 8. A B-response is generated to the AXI master after the WR is scheduled (posted-write semantics).

Read path: 1. AXI master issues an AR transaction; AXI4 slave accepts and queues. 2. Address mapping and queue caching identical to write path. 3. The scheduler issues ACT, then RD (or RDA) through the command encoder. 4. CL cycles after the RD command, the read data path samples

DFI rddata for the burst duration. 5. Read beats are tagged with the original AXI ID and streamed back on the R channel.

Refresh path: 1. The refresh manager's tREFI counter elapses, incrementing `refresh_pending`. 2. When `refresh_pending` exceeds the soft threshold, a refresh request enters the scheduler queue. 3. For DDR2 or LPDDR2 REFab: scheduler waits for all banks to be idle, then issues REF; all bank machines enter REFRESHING state. 4. For LPDDR2 REFpb under DARP: scheduler picks the idle bank with the oldest last-refresh timestamp; issues REFpb to that bank only; other banks remain available for column commands.

Init / power path: 1. On cold reset, the power-state FSM transitions to INIT and starts the init engine. 2. The init engine executes the memtype-specific step table, driving CKE/ODT/RESET_N and issuing MRS commands via the command encoder. 3. On completion (`init_done`), the FSM transitions to ACTIVE and the scheduler begins servicing AXI traffic. 4. Power-down / self-refresh entry is requested by the SoC via CSR and orchestrated by the FSM.

Detailed per-module behavior follows in Chapter 3.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

8 Module Hierarchy

The controller is decomposed into 17 RTL modules under a single top-level wrapper. Each is described in detail in Chapter 3.

8.1 Hierarchy Tree



Module Hierarchy

Source: [02_module_hierarchy.mmd](#)

8.2 Module Functional Groupings

For documentation and review purposes, the modules group naturally into seven functional layers:

Layer	Modules
AXI Frontend	axi4_slave, addr_mapper
Transaction Buffering	txn_queue
Scheduling	scheduler, page_predictor
Bank State + Timing	bank_machine, xbank_timers
Refresh + Power	refresh_mgr, power_state
Initialization	init_engine (with embedded step_table)
Command Output	cmd_encoder, gear_out
Data Paths	wdata_path, rdata_path
PHY Interface	dfi_master
Configuration	csr_slave

Chapter 3 follows this grouping (one section per functional layer).

8.3 Memtype-Conditional Modules

Three modules contain memtype-conditional logic that is selected at elaboration:

- **cmd_encoder** — instantiates either `ddr2_encoder` or `lpddr2_encoder` based on MEMTYPE.
- **init_engine** — its `step_table` ROM is loaded from either `ddr2_init_steps_pkg` or `lpddr2_init_steps_pkg`.
- **power_state** — handles Self-Refresh (DDR2) or Deep Power Down (LPDDR2) entry sequences.

Synthesis dead-code-eliminates the unused paths. The same RTL source supports both targets.

8.4 Optional Modules

One module is parameter-gated and may be entirely absent from the synthesized design:

- **page_predictor** — only instantiated when `PAGE_POLICY == HAPPY_HYBRID`. For OPEN or CLOSE policies, the predictor module is not synthesized.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

9 Top-Level Interfaces

This section is the integrator's quick-reference for the controller's top-level module port list. Detailed protocol semantics for each interface are in Chapter 4 (Interfaces); this is the **wire-level** view — names, directions, widths, parameter dependencies.

The top module is `ddr2_lpddr2_ctrl` and has four external interfaces:

1. AXI4 slave (host request path)
2. DFI v2.1 master (DRAM PHY path)
3. APB CSR slave (configuration and observation)
4. Clocks, resets, and miscellaneous (interrupts, debug, scan)

All four are exposed at the top level as flat per-signal ports. SystemVerilog interfaces are **not** used at the top boundary because the framework's interface convention (see `CocoTBFramework` BFM components) drives flat-port pin-level handshakes. Internal sub-blocks use interface objects, but the top-of-design is flat for synthesis and BFM compatibility.

9.1 Parameter-Dependent Widths

Several port widths depend on top-level parameters. Throughout this section:

- $AW = AXI_ADDR_WIDTH$ (default 32)
- $DW = AXI_DATA_WIDTH$ (default 64)
- $SW = DW/8$ (byte-strobe width)
- $IDW = AXI_ID_WIDTH$ (default 4)
- $DFI_DW = DFI_DATA_WIDTH$ (per-phase; default 32)
- $DFI_AW = DFI_ADDR_WIDTH$ (default 14 for DDR2, 20 for LPDDR2)
- $NP = N_PHASES$ (default 4)
- $NR = NUM_RANKS$ (default 1)
- $NB = NUM_BANKS$ (default 8)
- $BA = \lceil \log_2(NUM_BANKS) \rceil$

9.2 1. AXI4 Slave (Host-side)

9.2.1 Write Address Channel (AW)

Signal	Direction	Width	Description
awid	input	IDW	Write transaction ID
awaddr	input	AW	Write start address
awlen	input	8	Burst length – 1
awsize	input	3	Beat size encoding
awburst	input	2	Burst type (INCR mandatory; FIXED / WRAP optional)
awlock	input	1	Lock (ignored — exclusives not supported)
awcache	input	4	Cache hint (observed; no behavior)
awprot	input	3	Protection bits (observed)
awqos	input	4	QoS hint (boosts scheduler priority — see §3.2)
awregion	input	4	Region (observed)
awvalid	input	1	Address-valid
awready	output	1	Address-ready

9.2.2 Write Data Channel (W)

Signal	Direction	Width	Description
wdata	input	DW	Write data
wstrb	input	SW	Byte enables
wlast	input	1	Last beat of burst
wvalid	input	1	Data-valid
wready	output	1	Data-ready

9.2.3 Write Response Channel (B)

Signal	Direction	Width	Description
bid	output	IDW	Response ID
bresp	output	2	Response code

Signal	Direction	Width	Description
			(OKAY / SLVERR)
bvalid	output	1	Response-valid
breedy	input	1	Response-ready

9.2.4 Read Address Channel (AR)

Signal	Direction	Width	Description
arid	input	IDW	Read transaction ID
araddr	input	AW	Read start address
arlen	input	8	Burst length – 1
arsize	input	3	Beat size encoding
arburst	input	2	Burst type
arlock	input	1	Lock (ignored)
arcache	input	4	Cache hint
arprot	input	3	Protection
arqos	input	4	QoS hint
arregion	input	4	Region
arvalid	input	1	Address-valid
arready	output	1	Address-ready

9.2.5 Read Data Channel (R)

Signal	Direction	Width	Description
rid	output	IDW	Read data ID
rdata	output	DW	Read data
rresp	output	2	Response code
rlast	output	1	Last beat
rvalid	output	1	Data-valid
rready	input	1	Data-ready

9.3 2. DFI v2.1 Master (PHY-side)

The DFI v2.1 sub-interfaces present on this controller are limited to **command**, **write-data**, **read-data**, **status**, and **update**. Training, frequency-change, low-power, and CRC sub-interfaces are not driven — see §2.1 Out of Scope.

9.3.1 DFI Command Sub-Interface (Per-Phase)

The command bus is replicated N_PHASES times. Each per-phase index $p \in \{0 \dots NP-1\}$ carries its own command operand set; the gear logic in `gear_dfi` aligns them within the MC clock cycle.

Signal	Direction	Width	Per-Phase	Description
<code>dfi_address</code>	output	<code>DFI_AW</code>	$\times NP$	Address operand (row, column, or CA-bus)
<code>dfi_bank</code>	output	BA	$\times NP$	Bank operand (per-rank bank, see <code>dfi_cs_n</code> for rank)
<code>dfi_cs_n[r]</code>	output	1	$\times NP \times NR$	Active-low chip-select, one per rank
<code>dfi_ras_n</code>	output	1	$\times NP$	RAS strobe (DDR2 only; tied / unused for LPDDR2)
<code>dfi_cas_n</code>	output	1	$\times NP$	CAS strobe (DDR2 only)
<code>dfi_we_n</code>	output	1	$\times NP$	WE strobe (DDR2 only)
<code>dfi_cke[r]</code>	output	1	$\times NR$	Clock-enable, one per rank (cross-phase; held)
<code>dfi_odt[r]</code>	output	1	$\times NR$	ODT, one per rank (cross-phase; driven by <code>odt_ctrl</code>)
<code>dfi_reset_n</code>	output	1		DRAM reset (init engine)

9.3.2 DFI Write-Data Sub-Interface

Signal	Direction	Width	Description
<code>dfi_wrdata_en</code>	output	NP	Per-phase write-data valid
<code>dfi_wrdata</code>	output	$NP \times DFI_DW$	Concatenated per-phase write data
<code>dfi_wrdata_mask</code>	output	$NP \times DFI_DW/8$	Per-byte write mask
<code>dfi_wrdata_cs_n[</code>	output	$NP \times NR$	Per-rank, per-phase data-bus

Signal	Direction	Width	Description
r]			chip-select; the encoder asserts the matching rank when issuing a write

9.3.3 DFI Read-Data Sub-Interface

Signal	Direction	Width	Description
dfi_rddata_en	output	NP	Per-phase read-data enable
dfi_rddata	input	NP × DFI_DW	Per-phase read data
dfi_rddata_valid	input	NP	Per-phase read data valid
dfi_rddata_cs_n[r]	output	NP × NR	Per-rank, per-phase data-bus chip-select; the encoder asserts the matching rank when issuing a read

9.3.4 DFI Status and Update Sub-Interface

Signal	Direction	Width	Description
dfi_init_start	output	1	Init handshake — start of DRAM init
dfi_init_complete	input	1	PHY signals init complete
dfi_dram_clk_disable	output	1	PHY clock disable during power-down
dfi_ctrlupd_req	output	1	Controller-update request
dfi_ctrlupd_ack	input	1	Controller-update acknowledge
dfi_phyupd_req	input	1	PHY-initiated update request
dfi_phyupd_ack	output	1	PHY-update acknowledge
dfi_phyupd_type	input	2	Update type (training vs. ctrl-recal)

9.4 3. APB CSR Slave (Configuration / Observation)

Single APB3 slave (4 KB region; 12-bit paddr). Detailed register map in §6.3.

Signal	Direction	Width	Description
psel	input	1	Slave select
penable	input	1	Enable strobe
pwrite	input	1	Write enable
paddr	input	12	Address
pwrdata	input	32	Write data
pstrb	input	4	Byte strobe (APB4 only)
pready	output	1	Slave-ready
prdata	output	32	Read data
pslverr	output	1	Slave error

9.5 4. Clocks, Resets, Interrupts, Debug

Signal	Direction	Width	Description
mc_clk	input	1	MC clock (drives core, DFI command, scheduler)
mc_rst_n	input	1	MC asynchronous reset, active low
apb_pclk	input	1	APB clock (independent of mc_clk for CDC)
apb_presetn	input	1	APB reset, active low
irq_init_done	output	1	One-cycle pulse when DRAM init completes
irq_init_error	output	1	Latched on init failure (cleared by CSR write)
irq_overflow	output	1	AXI queue overflow / refresh-deadline miss
dbg_state	output	4	Encoded power-state for board-level probe
dbg_refresh	output	1	Asserted during any refresh sequence

The interrupts are wired to a downstream interrupt controller; the dbg outputs are typically routed to a test header or chipset SignalTap / ILA.

9.6 Hierarchical Top-Level View

The top-level pinout in shorthand:

```

ddr2_lpddr2_ctrl
├── AXI4 slave          (host)          awid, awaddr, ..., bvalid,
bready, ...
├──                      arid, araddr, ..., rvalid,
rready, ...
├── DFI v2.1 master    (PHY)          dfi_address,
dfi_cs_n[r], ..., dfi_cke[r], dfi_odt[r], ...
├── APB CSR slave      (config)       psel, penable, paddr, ...,
prdata, pslverr
├── Clocks + IRQs
apb_presetn, irq_*, dbg_*          mc_clk, mc_rst_n, apb_pclk,

```

The full SystemVerilog port list — with every parameter substituted — is generated by the build system into `regs/ddr2_lpddr2_ctrl_ports.svh` and is the canonical source. This section is the **integrator's overview**, not the build-time canonical list.

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

10 FUB Breakdown (First-Pass SWAG)

This section is a **first-pass SWAG** — Some Wild-Ass Guess — at the controller's Functional Unit Block decomposition. It follows the RTLDesignSherpa stream convention of organizing RTL as small, independently-verifiable blocks under `rtl/fub/`, with a single top-level integration in `rtl/top/`. The boundary lines below will move during detailed design; the purpose here is to establish the SWAG so that file layout, filelist generation, and per-FUB verification plans can start in parallel with the architectural detail.

10.1 Stream FUB Convention

Per the standard component layout:

```

ddr2-lpddr2/
├── bin/                # build / regression scripts
├── coverage_combined/  # aggregated coverage reports
├── coverage_data/      # raw cocotb coverage drops
├── docs/               # this HAS, generated PDFs
├── dv/                 # cocotb tests, BFM glue
├── regs/               # CSR JSON/YAML → generated SV/headers
├── reports/            # synth/timing reports
├── rtl/
│   └── top/            # ddr2_lpddr2_ctrl.sv (integration)

```

```

only)
├── fub/                # leaf FUBs (this section)
├── includes/          # shared `defines, parameter packages
├── macro/             # SRAMs, latches, hard-IP wrappers
└── filelists/         # *.f files per FUB and per top

```

Each FUB has: - One top SystemVerilog file with the same name as the FUB - An optional `_pkg.sv` for parameter typedefs / config structs - A filelist `filelists/<fub_name>.f` - A dv/ testbench that drives that FUB standalone

10.2 FUB Inventory

The 14 leaf FUBs proposed below are grouped by major architectural role. Each entry gives the FUB name, primary inputs/outputs (informal), the parameters that govern its size, and a one-line purpose. These FUBs map directly to the sub-modules called out in §3 (Architecture) but are the level at which the **filelist** and **synthesis boundary** are drawn.

10.2.1 Group A — AXI Frontend

10.2.1.1 *axi4_slave_fub*

- **Purpose:** AXI4 slave protocol engine; AW / W / B and AR / R channel handshake; ID-aware out-of-order completion buffer.
- **Key params:** AXI_DATA_WIDTH, AXI_ID_WIDTH, AXI_ADDR_WIDTH, AXI_000_ACROSS_IDS, SUPPORT_FIXED_BURST, SUPPORT_WRAP_BURST.
- **Upstream:** top-level AXI ports.
- **Downstream:** `addr_mapper_fub` (decoded address), `txn_queue_fub` (request push), data-path FUBs (`wr_data_path_fub`, `rd_data_path_fub`).

10.2.1.2 *addr_mapper_fub*

- **Purpose:** Decode flat AXI address into (rank, bank, row, column) per the active address-mapping scheme.
- **Key params:** NUM_RANKS, NUM_BANKS, ROW_WIDTH, COL_WIDTH, ADDR_MAP_SCHEMES_SYNTH, ADDR_MAP_SCHEME_DEFAULT.
- **Notable:** Mirrors the Python AddressMapping class in the DV repo; the same decode function is used by RTL and BFM.

10.2.2 Group B — Transaction Buffering and Scheduling

10.2.2.1 *txn_queue_fub*

- **Purpose:** Pending-transaction queue with `row_hit` caching and saturating age counters.

- **Key params:** TXN_QUEUE_DEPTH, AGE_MAX.
- **Storage:** Distributed registers; for TXN_QUEUE_DEPTH $\geq$ 32 consider promoting to a small SRAM in rtl/macro/.

10.2.2.2 *scheduler_fub*

- **Purpose:** FR-FCFS priority function, lookahead window, refresh-priority gating, runtime force_inorder collapse.
- **Key params:** SCHEDULER_MODE, LOOKAHEAD_DEPTH_MAX.
- **Notable:** The single hardest synthesis-timing FUB in the design; expected to dominate the path from txn_queue to command issue.

10.2.2.3 *page_predictor_fub (conditional)*

- **Purpose:** HAPPY hybrid page-conflict predictor; synthesized only when PAGE_POLICY == HAPPY_HYBRID.
- **Key params:** PAGE_PREDICTOR_TABLE_BITS.
- **Storage:** One small SRAM (rtl/macro/page_pred_table_sram) for table size $\geq$ 12 bits; distributed registers below.

10.2.3 Group C — Per-Bank State and Cross-Bank Timing

10.2.3.1 *bank_machine_fub*

- **Purpose:** Per-(rank, bank) FSM; per-bank timing counters; refresh handshake.
- **Key params:** ROW_WIDTH. Instantiated NUM_RANKS $\times$ NUM_BANKS times by the top.
- **Synthesis note:** Replicated; an attempt at sharing across banks is rejected because per-bank state must be queryable in parallel by the scheduler.

10.2.3.2 *xbank_timers_fub*

- **Purpose:** Cross-bank timing constraints (tRRD, tFAW, tCCD, tWTR, tRTW, tRTRS, tCS); per-rank vs. global scope as documented in §3.3.
- **Key params:** NUM_RANKS, NUM_BANKS.
- **Storage:** Per-rank 4-entry tFAW FIFOs; remaining state is small down-counters.

10.2.4 Group D — Refresh, Init, Power

10.2.4.1 *refresh_mgr_fub*

- **Purpose:** tREFI postponer; REFab / REFpb dispatch; round-robin rank dispatch; DARP bank selection; periodic ZQCS piggyback; PASR mask propagation.
- **Key params:** NUM_RANKS, NUM_BANKS, REFPB_POLICY, REFRESH_DEFER_MAX.

10.2.4.2 *init_engine_fub*

- **Purpose:** Microprogram-driven cold-boot init sequence per MEMTYPE; step-table ROM lookup; sub-step timing.
- **Key params:** MEMTYPE, SIM_INIT_SCALE.
- **Storage:** Step-table ROMs in rtl/macro/init_steps_<memtype>_rom.

10.2.4.3 *power_state_fub*

- **Purpose:** Active / Active-Power-Down / Self-Refresh / Deep-Power-Down FSM; CKE per-rank control; SR-entry / SR-exit coordination with refresh_mgr_fub.
- **Key params:** NUM_RANKS, MEMTYPE (DPD is LPDDR2 only).

10.2.5 Group E — Command Encoding and Output Gear

10.2.5.1 *cmd_encoder_fub*

- **Purpose:** Swap between DDR2 (ras/cas/we) and LPDDR2 (CA-bus packed) encoding; selected at elaboration by MEMTYPE.
- **Key params:** MEMTYPE, DFI_ADDR_WIDTH, N_PHASES.

10.2.5.2 *gear_dfi_fub*

- **Purpose:** Pack scheduler-issued commands and data into per-phase DFI slots according to N_PHASES, WRPHASE, RDPHASE.
- **Key params:** N_PHASES, WRPHASE, RDPHASE, DFI_DATA_WIDTH.

10.2.5.3 *odt_ctrl_fub*

- **Purpose:** Drive per-rank dfi_odt[r] per the multi-rank termination rule (ODT_RULE_MULTIRANK). Cross-rank read/write turn-on/turn-off windows.
- **Key params:** NUM_RANKS, ODT_RULE_MULTIRANK, MEMTYPE.
- **Notable:** This FUB is the home of the famous “ODT-on-other-rank during read” rule — see §3.6.

10.2.6 Group F — Data Paths

10.2.6.1 *wr_data_path_fub*

- **Purpose:** AXI W-channel → write-data buffer → DFI wrdata. Width-conversion (AXI DW → NP × DFI_DW). Byte-mask propagation.
- **Key params:** AXI_DATA_WIDTH, N_PHASES, DFI_DATA_WIDTH.

10.2.6.2 *rd_data_path_fub*

- **Purpose:** DFI rddata → read-data buffer → AXI R-channel; CL / CWL-aware capture; ID-tagged completion buffer for OoO return.
- **Key params:** AXI_DATA_WIDTH, N_PHASES, DFI_DATA_WIDTH, AXI_000_ACROSS_IDS.

10.2.7 Group G — CSR

10.2.7.1 *csr_apb_fub*

- **Purpose:** APB3 slave; register file; CDC between apb_pclk and mc_clk; runtime override propagation with quiet-point sync.
- **Key params:** derived from CSR YAML in regs/.
- **Notable:** The register fields themselves are generated; the FUB wrapper provides the APB protocol and the CDC.

10.3 FUB Count Summary

Group	FUBs	Notes
A — AXI Frontend	2	axi4_slave, addr_mapper
B — Transaction / Scheduling	3	txn_queue, scheduler, page_predictor*
C — Bank state	2	bank_machine, xbank_timers
D — Refresh / Init / Power	3	refresh_mgr, init_engine, power_state
E — Encoding / Output	3	cmd_encoder, gear_dfi, odt_ctrl
F — Data paths	2	wr_data_path, rd_data_path
G — CSR	1	csr_apb
Total	16	(page_predictor conditional; effective floor 15)

The number is a SWAG; expect ± 2 during detailed design as the boundary between scheduler ↔ xbank_timers and between gear_dfi ↔ cmd_encoder is re-litigated.

10.4 Integration

rtl/top/ddr2_lpddr2_ctrl.sv is integration-only — pure structural wiring across the FUBs above, with **no behavioral logic**. Every behavioral statement belongs in a FUB.

The top has three principal wiring concerns:

- 1. **Per-rank, per-bank fan-out** of bank_machine_fub instances; their state aggregation to scheduler_fub.
- 2. **Per-rank fan-out** of dfi_cs_n, dfi_cke, dfi_odt from odt_ctrl_fub and power_state_fub.
- 3. **Per-phase fan-out** to gear_dfi_fub and wr_data_path_fub / rd_data_path_fub.

The integration file is mechanically generated by a build-time wrapper-generator script from the FUB filelists; hand-editing is only for the leaf FUB instantiation block.

11 AXI4 Frontend

The AXI frontend comprises two modules: axi4_slave (channel handshake and decoupling) and addr_mapper (flat-address-to-DRAM-coordinates translation).

11.1 axi4_slave

11.1.1 Purpose

Absorb AXI4 channel pressure and decouple SoC-side handshake behavior from the controller’s internal clock-aligned pipeline. The slave terminates the five AXI channels (AW, W, B, AR, R) and presents a simplified internal interface to the rest of the controller.

11.1.2 Internal State

Element	Description
AW skid buffer	2-deep; absorbs back-pressure timing

Element	Description
AR skid buffer	2-deep
W skid buffer	2-deep
B response FIFO	Indexed by AXI ID; preserves per-ID order
R response staging	Per-ID burst beat buffer

11.1.3 Behavior

- Accepts AW transactions; emits an internal `txn_alloc` request to `addr_mapper` and `txn_queue`. Acknowledges AW once both downstream slots are reserved.
- AR transactions are handled identically.
- W beats stream into a write-data FIFO with the originating transaction ID attached.
- B and R responses come back from the controller core and are routed to their channels with the correct ID.

11.1.4 Per-ID Ordering

AXI4 requires reads from the same ID to return in order, and writes from the same ID to commit in order. The slave preserves order **per ID**. Across IDs, out-of-order completion is permitted by default (controlled by the `AXI_000_ACROSS_IDS` parameter — defaults to `true`).

11.1.5 Burst Splitting

AXI4 permits up to 256 beats per burst; a DRAM row contains $2^{\text{COL_WIDTH}}$ columns. If an AXI burst exceeds the remaining columns in the current row, `axi4_slave` splits the burst at the row boundary. Splitting is transparent to the AXI master — internally, the burst is recorded as two transactions with a “continuation” flag that prevents the second half from being scheduled before the first completes.

11.1.6 Backpressure

- `AW .ready` deasserts when the transaction queue is full or has no available slots for the requested burst size.
- `AR .ready` follows the same rule.
- `W .ready` deasserts when the write-data FIFO is full.
- `R` and `B .valid` follow standard AXI protocol; consumers may stall the channels indefinitely without affecting the controller core.

11.2 addr_mapper

11.2.1 Purpose

Translate a flat AXI address into DRAM coordinates (*bank*, *row*, *col*). The mapping is parameterizable so the same module supports different memory geometries and different scheduling-friendly bank interleavings.

11.2.2 Interfaces

Inputs:

- *axi_addr* — AXI_ADDR_WIDTH bits wide

Outputs:

- *bank* — $\$clog2(\text{NUM_BANKS})$ bits
- *row* — ROW_WIDTH bits
- *col* — COL_WIDTH bits

11.2.3 Mapping Function

The default mapping is row-major (column in the low bits, bank above, row above that):

```
axi_addr[AXI_ADDR_WIDTH-1 : ROW_WIDTH + BANK_WIDTH + COL_WIDTH] =  
unused  
axi_addr[ROW_WIDTH + BANK_WIDTH + COL_WIDTH - 1 : BANK_WIDTH +  
COL_WIDTH] = row  
axi_addr[BANK_WIDTH + COL_WIDTH - 1 : COL_WIDTH] = bank  
axi_addr[COL_WIDTH - 1 : 0] = col
```

11.2.4 Alternative Mappings

A parameter ADDR_MAP_SCHEME selects from three predefined mappings:

Scheme	Layout	When to use
ROW_MAJOR	row bank col	Default; good for streaming workloads
BANK_INTERLEAVE	row col bank	Improves bank-parallelism for random workloads
XOR_HASH	row bank $\oplus$ row_low col	Breaks pathological row-conflict patterns

11.2.5 Relationship to the DFI BFM

This module mirrors the `AddressMapping` class in the DFI BFM (`src/CocoTBFramework/components/dfi/dfi_signals.py`). The same mapping object can drive both RTL elaboration and BFM verification, ensuring address-decode consistency between simulation and silicon.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

12 Scheduler and Page Predictor

This section covers two tightly-coupled modules: the FR-FCFS scheduler that picks the next request to issue, and the optional HAPPY page predictor that informs the scheduler's auto-precharge decisions.

12.1 txn_queue

12.1.1 Purpose

Hold pending memory transactions with the metadata the scheduler needs to make priority decisions.

12.1.2 Entry Format

Each slot in the queue carries:

Field	Width	Purpose
valid	1	Slot occupied
axi_id	AXI_ID_WIDTH	Originating AXI transaction ID
is_write	1	Write (1) vs read (0)
bank	$\$clog2(NUM_BANKS)$	Target bank
row	ROW_WIDTH	Target row
col	COL_WIDTH	Target starting column
burst_len	8	AXI len + 1 (max 256)
row_hit	1	Cached at insert
age	$\$clog2(AGE_MAX)$	Saturating age counter

Field	Width	Purpose
state	2	{PENDING, ISSUED, COMPLETING}

12.1.3 Depth

TXN_QUEUE_DEPTH (default 16; parameterizable 4–64).

12.1.4 Behavior

- New entries are inserted by `axi4_slave` via `addr_mapper`.
- `row_hit` is computed at insert time by querying the relevant bank machine's current `open_row` register. Caching this avoids per-cycle recomputation in the scheduler.
- `age` increments every cycle and saturates at `AGE_MAX` (default 256).
- The scheduler reads the entire queue snapshot each cycle to choose the next issue candidate.
- Entries transition `PENDING` → `ISSUED` when the scheduler picks them, then → `COMPLETING` when the corresponding data beats are flowing, then deallocate when complete.

12.2 scheduler

12.2.1 Purpose

Pick the next request to issue and dispatch its corresponding DRAM command. Implements FR-FCFS with characterization knobs for page policy and refresh prioritization.

12.2.2 Inputs

- Queue snapshot (all entries' metadata)
- Per-bank state from each `bank_machine` (`idle` / `activating` / `active` / `precharging`)
- Refresh request from `refresh_mgr` with priority level
- Init-in-progress signal (pauses normal scheduling)
- Page predictor signal (when `HAPPY` enabled)

12.2.3 Outputs

- `issue_valid` — 1 when a command is being issued this cycle

- `issue_op` — {ACT, RD, RDA, WR, WRA, PRE, PREA, REF, REFPB, MRS, ZQCS, ZQCL, NOP}
- `issue_bank`, `issue_row`, `issue_col`, `issue_axi_id` — operands

12.2.4 Priority Function

Evaluated each cycle (descending priority):

1. **Init in progress** — block all normal traffic; init commands take a parallel path.
2. **Critical refresh** (`refresh_pending` $\geq$ `critical_threshold`) — REF / REFPB wins all ties.
3. **Soft refresh** (`refresh_pending` $\geq$ `soft_threshold`) — boost refresh above row-misses but allow row-hits.
4. **Row-hit ready columns** — same bank's open row matches request row; bank machine accepts column command.
5. **Age-ordered FCFS among row-hits** — oldest wins.
6. **Row-miss requests** — bank needs PRE then ACT first.
7. **Idle (NOP)**.

12.2.5 Strict In-Order Mode

For systems that require deterministic, in-order issue — real-time / safety-critical / formally-verifiable timing targets — the scheduler can be forced into a first-ready FIFO mode. Two routes:

Runtime debug switch (recommended). When `SCHEDULER_MODE` = 000 is synthesized (the default), setting `SCHED_TUNING.force_inorder` = 1 collapses the priority function to first-ready FIFO at runtime. The OoO comparator logic is still in silicon but its outputs are masked. Switching is a single CSR write, takes effect at the next configuration quiet point, and is fully reversible.

Build-time omission. Setting `SCHEDULER_MODE` = INORDER at elaboration omits the FR-FCFS comparator logic entirely, saving area. Once chosen, the design cannot switch back to OoO at runtime — `SCHED_TUNING.force_inorder` becomes a tied-high observation bit. Pick this only if the area savings are critical and OoO will never be needed.

In either form, the in-order behavior is:

- Only the **first-ready filter** survives. The scheduler still skips requests whose target bank is blocked by timing constraints (otherwise it would waste cycles on requests that can't issue), but it does not reorder requests for row-hit benefit.

- **Row-hit prioritization is disabled.** Requests are considered strictly in `txn_queue` arrival order.
- **Age-based ties become moot** — the queue is already age-ordered by construction.
- **Refresh priority still applies** (the refresh path is a JEDEC requirement, not a performance choice).
- **HAPPY and lookahead are unaffected at elaboration** but can be independently disabled via runtime CSR (`SCHED_TUNING.happy_enable = 0`, `SCHED_TUNING.lookahead_active = 0`). For pure first-ready FIFO with neither, set all three CSR bits.
- **AXI per-ID ordering is honored regardless.** The change here is only at the DRAM-issue layer, not at the AXI completion layer.

Trade-off: in-order mode reduces sustained bandwidth on streaming workloads typically 10–25% (the cost of not reordering row-conflicts behind row-hits), but issue latency becomes deterministic and the design is much easier to formally verify. The runtime-switchable form is preferred for shipping designs because it preserves debug flexibility at the cost of the silicon area for the unused OoO logic.

12.2.6 Auto-Precharge Decision: Lookahead + Page Policy

The decision to issue a column command as bare RD / WR (keep row open) versus the auto-precharge variant RDA / WRA (close row in flight) follows a two-stage rule. Lookahead runs first; the page policy is the fallback when lookahead is inconclusive.

12.2.6.1 Stage 1: Lookahead

If `LOOKAHEAD_DEPTH > 0`, the scheduler peeks the next `LOOKAHEAD_DEPTH` pending entries in `txn_queue` that target the same bank as the current command:

- If a same-bank pending request is found AND its target row equals the current `open_row`, the next access is a guaranteed row-hit. Issue the bare RD / WR so the row stays open.
- If a same-bank pending request is found AND its target row differs, the next access is a guaranteed row-conflict. Issue RDA / WRA so the precharge happens in flight (saves an explicit PRE command + tRP later).
- If no same-bank pending request is found within `LOOKAHEAD_DEPTH`, fall through to Stage 2.

When the transaction queue is deep — which is the common case for streaming / DMA workloads — lookahead provides an **exact** auto-precharge decision rather than a predicted one. The

LOOKAHEAD_DEPTH $\in \{0, 1, 2, 4\}$ parameter sweeps lookahead window size against comparator cost during characterization (covered in §5).

12.2.6.2 *Stage 2: Page Policy Fallback*

Triggered when lookahead is disabled (LOOKAHEAD_DEPTH = 0) or inconclusive (no same-bank request in the queue, typical of shallow-queue or bursty CPU traffic). Dispatched per PAGE_POLICY:

- **OPEN** — issue bare RD / WR. Rows stay open until evicted by a row-conflict or refresh. Best for streaming workloads where lookahead is usually conclusive anyway, so the OPEN fallback rarely fires.
- **CLOSE** — issue RDA / WRA. Always close on every access. Best for random access without temporal locality.
- **HAPPY_HYBRID** — query the page_predictor with the current request's row address. If the predictor says “next access to this bank will conflict,” issue RDA / WRA; otherwise issue bare RD / WR. Recommended for bursty mixed workloads where neither lookahead nor a fixed policy is reliable.

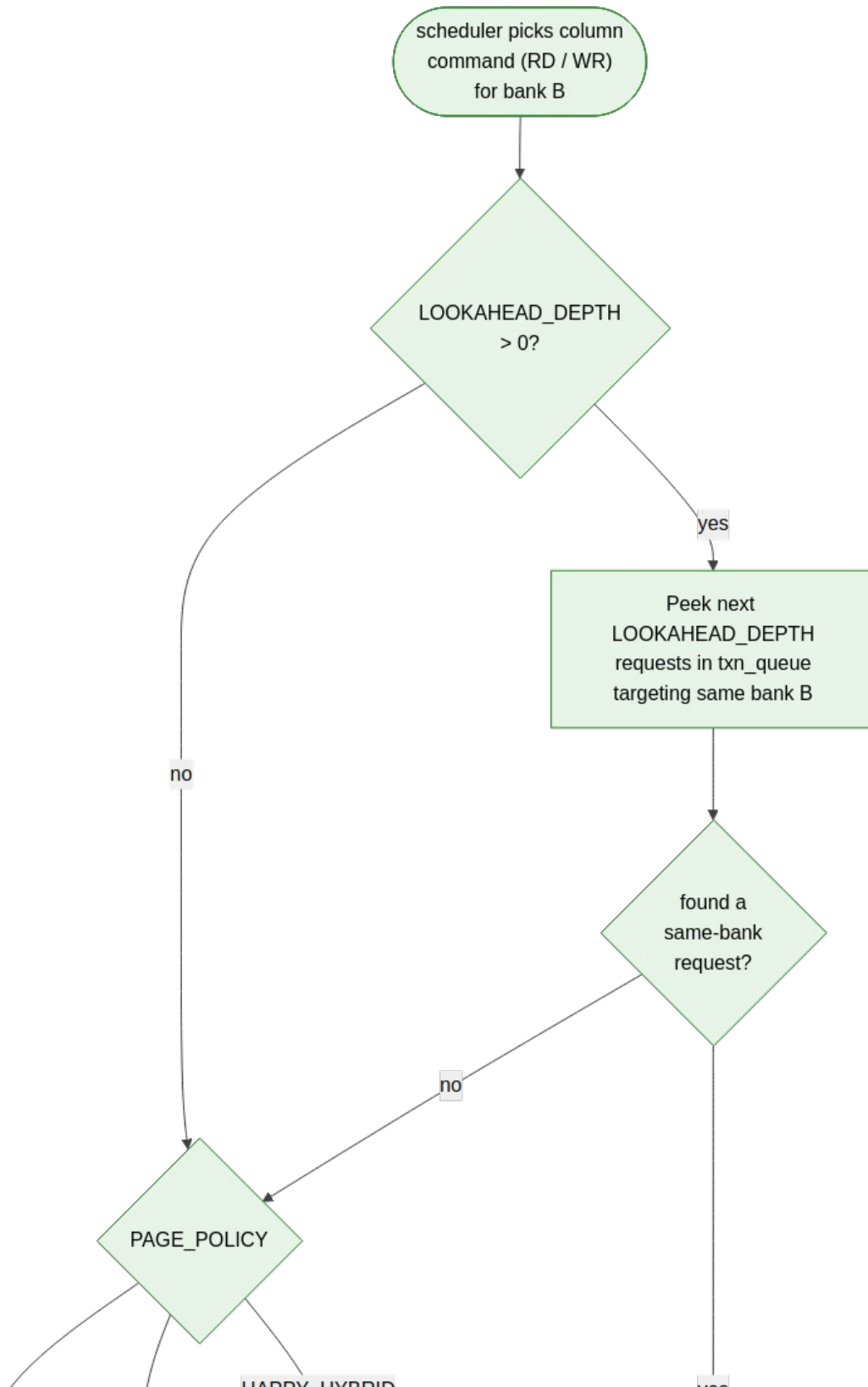
12.2.6.3 *Why Both*

Lookahead is **provably correct** when the queue contains a same-bank request: we know exactly what the next access will be. HAPPY (and the simple OPEN / CLOSE policies) operate without that information.

The two mechanisms are complementary:

- **Deep queue, repetitive locality** (streaming, DMA): lookahead almost always resolves; the page-policy choice barely matters.
- **Shallow queue, sporadic access** (bursty CPU traffic, multi-tenant): lookahead frequently misses; the page-policy choice dominates.

LOOKAHEAD_DEPTH is a characterization knob in its own right — see §5.



Lookahead + Page-Policy Decision Tree

Source: [10_lookahead_auto_pre.mmd](#)

12.2.7 Issue Slot Allocation

One command issue per MC clock cycle. Multi-cycle LPDDR2 commands are handled inside the `cmd_encoder`, so the scheduler still sees them as a single issue.

12.3 page_predictor (HAPPY hybrid, optional)

12.3.1 Purpose

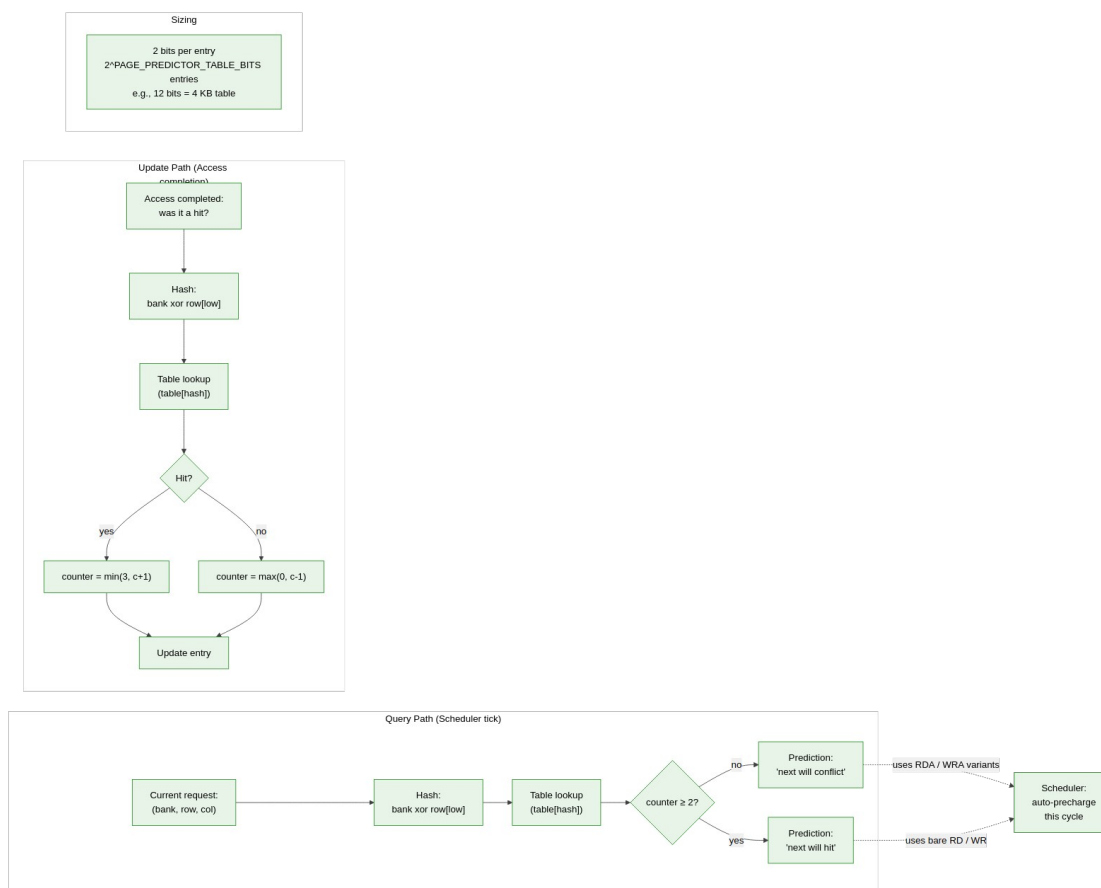
Predict whether the next access to a bank will hit the currently open row or conflict with it. Synthesized only when `PAGE_POLICY == HAPPY_HYBRID`.

12.3.2 Algorithm

Address-bit-based two-level predictor following Ghasempour et al. (2015):

- Table indexed by hash of bank and a small selection of row-address bits
- Each entry: 2-bit saturating counter (hit-likely vs conflict-likely)
- Update on access completion: was the next access a hit or a conflict?
- Query at scheduler decision time: “for the request I’m about to issue, will the next access to this bank be a hit?”

12.3.3 HAPPY Predictor Architecture



HAPPY Predictor Query + Update Paths

Source: [09_happy_predictor.mmd](#)

12.3.4 Storage

Total storage: $2 \times 2^{\text{PAGE_PREDICTOR_TABLE_BITS}}$ bits.

- For PAGE_PREDICTOR_TABLE_BITS = 12 (default): 8 Kbit (about 1 KB)
- For PAGE_PREDICTOR_TABLE_BITS = 8: 512 bit
- For PAGE_PREDICTOR_TABLE_BITS = 16: 128 Kbit (around 16 KB; large but rarely needed)

12.3.5 Reset Behavior

All entries reset to “weakly hit-likely” (binary 01) so the controller starts in open-page-like behavior. The predictor adapts to workload over the first few hundred accesses.

12.3.6 Observability

A debug output exposes the rolling prediction accuracy (% correct) per bank, which the SoC can read via CSR. This is critical for tuning during bring-up and for the characterization sweep.

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

13 Bank Machines and Cross-Bank Timers

Per-bank state machines enforce JEDEC timing locally; a shared cross-bank timer pool enforces the constraints that span banks.

13.1 bank_machine

13.1.1 Purpose

One FSM instance per bank. Enforces per-bank JEDEC timing constraints; tracks open-row state.

13.1.2 Instantiation

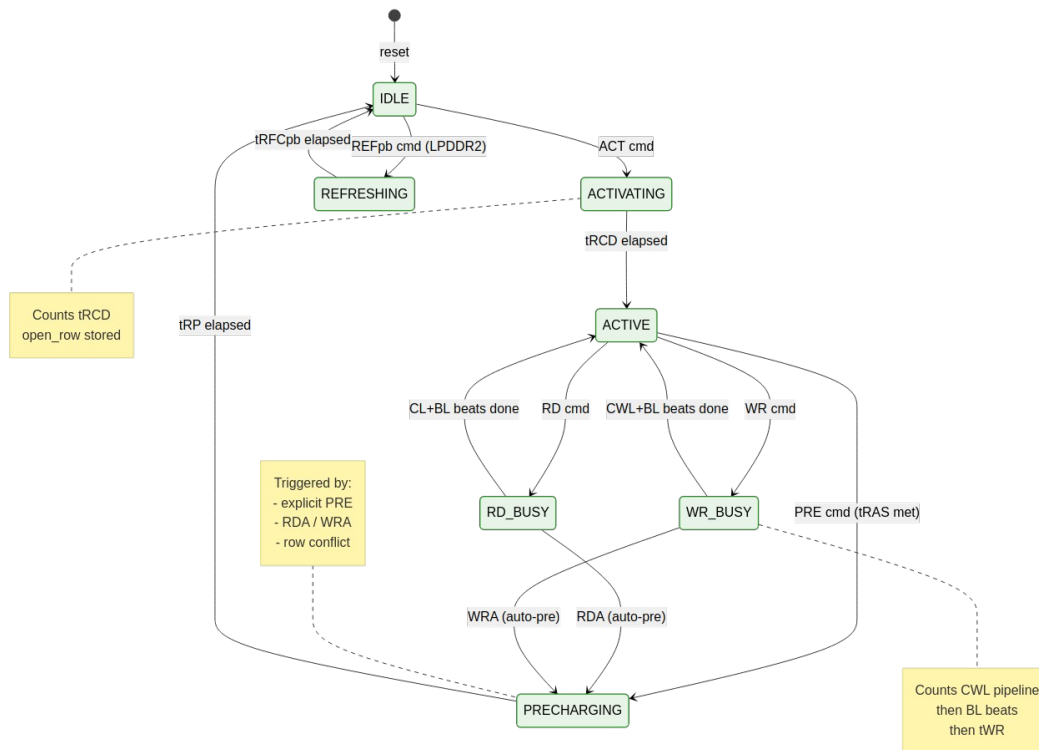
NUM_RANKS × NUM_BANKS instances — one FSM per (rank, bank) pair. For the default (NUM_RANKS=1, NUM_BANKS=8) this is 8 instances; for a 2-rank DIMM with 8-bank devices it is 16; for a 4-rank configuration it is 32. The (rank, bank) tuple is the bank machine's index; all per-bank state (open_row, timing counters, last_ref_age) is independent across ranks because JEDEC tracks every rank's banks separately.

13.1.3 FSM States

State	Description
IDLE	Bank closed; ready to accept ACT
ACTIVATING	Counting tRCD; row is being opened
ACTIVE	Row open; ready for RD / WR
RD_BUSY	Read in progress; counting CL for data return
WR_BUSY	Write in progress; counting

State	Description
PRECHARGING	CWL + tWR window
REFRESHING	Counting tRP
REFRESHING	Counting tRFCpb (per-bank refresh)

13.1.4 FSM Diagram



Per-Bank FSM

Source: [03_bank_machine_fsm.mmd](#)

13.1.5 Per-Bank Registers

Register	Width	Purpose
open_row	ROW_WIDTH	Valid in {ACTIVE, RD_BUSY, WR_BUSY}
t_rcd_cnt	small	tRCD down-counter (loaded on ACT issue)
t_ras_cnt	small	tRAS down-counter
t_rp_cnt	small	tRP down-counter

Register	Width	Purpose
t_rc_cnt	small	tRC down-counter
t_wr_cnt	small	tWR down-counter
t_rfcpb_cnt	small	tRFCpb down-counter
last_ref_age	wider	Cycles since last refresh (for DARP / OLDEST_FIRST)

All counters saturate at zero.

13.1.6 Outputs to Scheduler

- state — current FSM state
- open_row — row register (valid when state ∈ active set)
- accepts_act — combinational: 1 when state == IDLE and t_rp_cnt == 0
- accepts_rd, accepts_wr — 1 when state == ACTIVE and constraints met
- accepts_pre — 1 when state == ACTIVE and t_ras_cnt == 0
- accepts_ref — 1 when state == IDLE
- last_ref_age — exposed to refresh manager

13.1.7 Refresh Handshake Interface

In addition to the scheduler-side signals above, each bank machine exposes a dedicated **refresh handshake** pair to the refresh_mgr:

Signal	Direction	Purpose
refresh_req	refresh_mgr → bank	Refresh manager wants to issue REF; bank should drain to a quiescent state
refresh_gnt	bank → refresh_mgr	Bank acknowledges it is in IDLE (or REFRESHING for a per-bank refresh) and the controller may proceed

The handshake is the explicit version of “wait for all banks idle.” The refresh manager asserts refresh_req to all bank machines; each bank machine, on completing its current activity and landing in IDLE, asserts refresh_gnt. The refresh manager waits until **all** bank-machine grants are high (for REFab) or the **selected** bank’s grant is high (for REFpb) before issuing the actual sequence. When the refresh manager deasserts refresh_req, the bank machine drops refresh_gnt and resumes normal operation.

The handshake gives the refresh manager deterministic acknowledgment timing rather than relying on a race-prone “wait for state == IDLE” poll.

13.1.8 Inputs from Cross-Bank Timers

The cross-bank timer pool feeds blocking signals:

- `xbank_blocks_act` — `tRRD` or `tFAW` prevents new ACT
- `xbank_blocks_wr_after_rd` — `tCCD` or `tWTR` prevents WR after recent RD
- `xbank_blocks_rd_after_wr` — `tWTR` prevents RD after recent WR data beat

These gate the per-bank `accepts_*` signals.

13.1.9 Auto-Precharge Handling

RDA / WRA commands trigger an automatic transition `RD_BUSY` → `PRECHARGING` (or `WR_BUSY` → `PRECHARGING`) without a separate PRE command from the scheduler. Internal book-keeping handles the timing.

13.2 `xbank_timers`

13.2.1 Purpose

Enforce cross-bank timing constraints. Implemented as a single shared module rather than per-bank to share the FIFO buffer for `tFAW`.

13.2.2 Tracked Constraints

Constraint	Description
<code>tRRD</code>	Minimum gap between any two ACT commands
<code>tFAW</code>	At most 4 ACTs in any rolling <code>tFAW</code> window
<code>tWTR</code>	Write-to-read column-to-column on different banks
<code>tCCD</code>	Column-to-column (any column command to any other)

13.2.3 Implementation

Constraint	Storage
<code>tRRD</code>	Single saturating down-counter loaded on any ACT issue
<code>tFAW</code>	4-entry FIFO of recent ACT timestamps; new ACT blocked if oldest is younger than (<code>now</code> – <code>tFAW_cycles</code>)
<code>tCCD</code>	Small down-counter loaded on any RD / WR issue
<code>tWTR</code>	Down-counter loaded on the last WR data beat completion

13.2.4 Outputs

- Per-bank `blocks_act` / `blocks_rdw` signals that the bank machines AND into their respective `accepts_*` outputs.

13.2.5 Why Shared Rather Than Per-Bank

Cross-bank constraints are global: a single tRRD counter can be checked by all banks simultaneously, and the tFAW FIFO needs to see all recent ACTs across all banks. Replicating these per bank would be wasteful.

13.2.6 Per-Rank vs. Cross-Rank Scope

When `NUM_RANKS > 1`, each cross-bank constraint must be evaluated at the correct scope:

Constraint	Scope	Rationale
tRRD	per-rank	tRRD limits how fast a single device can stagger ACTs; different ranks have independent activate stagger budgets
tFAW	per-rank	tFAW is a device-local thermal/power limit; each rank has its own 4-ACT rolling window
tCCD	global (all ranks)	The DQ bus is shared across ranks; column commands to different ranks still share the data window
tWTR	global (all ranks)	Same reason — data-bus turnaround applies regardless of rank
tRTW	global (all ranks)	Same reason — data-bus turnaround applies regardless of rank
tRTRS	cross-rank only	Rank-to-rank read switching delay (1 cycle DDR2/3); applies when consecutive RDs target different ranks
tCS	cross-rank only	Chip-select setup; only relevant when switching ranks mid-burst

The implementation therefore has a per-rank tRRD counter (`NUM_RANKS` copies) and a per-rank tFAW FIFO (`NUM_RANKS` copies, 4 entries each), with the global tCCD / tWTR / tRTW counters shared as before. Two additional cross-rank counters (`tRTRS_cnt`, `tCS_cnt`) are added when `NUM_RANKS > 1` and gate `accepts_rd` / `accepts_wr` whenever the candidate's rank differs from the most recent rank seen on the data bus.

14 Refresh Manager

The refresh manager owns tREFI timing, refresh request queuing, the choice between REFab and REFpb, and the DARP-based per-bank refresh scheduling policy.

14.1 refresh_mgr

14.1.1 Purpose

Issue REF / REFpb commands on schedule. Honor the PASR mask. Implement Elastic-Refresh deferral up to REFRESH_DEFER_MAX × tREFI. Implement DARP-style idle-bank-first per-bank refresh scheduling for LPDDR2.

14.1.2 Internal State

Element	Description
t_refi_cnt	Central tREFI down-counter
postpone_cnt	Down-counter from REFRESH_DEFER_MAX – 1 to 0 (postponer batch)
refresh_owed	Non-saturating counter of refreshes still owed (deterministic; replaces saturating refresh_pending)
t_zqcs_cnt	ZQCS interval counter (cycles between periodic ZQCS)
wants_zqcs	One-cycle pulse when periodic ZQCS is due
pasr_bank_mask[NUM_RANKS]	Per-rank NUM_BANKS-bit mask (set via CSR; mirrors each rank's MR16/17). LPDDR2 PASR is a per-rank construct because each rank has its own mode-register file.
pasr_seg_mask[NUM_RANKS]	Per-rank segment mask (set via CSR)
last_ref_cycle[R][B]	Per-(rank, bank) timestamp (lives in each bank_machine)
round_robin_rank	Cyclic rank pointer for REFab dispatch; advanced after each refresh issue so all ranks are serviced

Element	Description
	fairly

14.1.3 Postponer Behavior (Deterministic Batching)

The postponer is a deterministic countdown that gathers REFRESH_DEFER_MAX deadlines, then triggers exactly that many back-to-back refreshes via the sequencer.

t_refi_cnt decrements from tREFI_cycles to zero each cycle. On reaching zero:

1. postpone_cnt decrements (from REFRESH_DEFER_MAX – 1 down to 0)
2. Reload t_refi_cnt = tREFI_cycles
3. When postpone_cnt reaches 0, emit a one-cycle wants_refresh pulse, reload postpone_cnt = REFRESH_DEFER_MAX – 1, and add REFRESH_DEFER_MAX to refresh_owed

The sequencer (next subsection) then drains refresh_owed to zero by issuing that many back-to-back REF / REFpb commands.

Why deterministic, not saturating. Earlier revisions used a saturating refresh_pending counter that silently dropped refresh deadlines when the system stayed bandwidth-starved past 8× tREFI. JEDEC requires the controller to catch up on all missed refreshes; saturation is technically out-of-spec. The deterministic postponer counts exact refresh events over time and never drops.

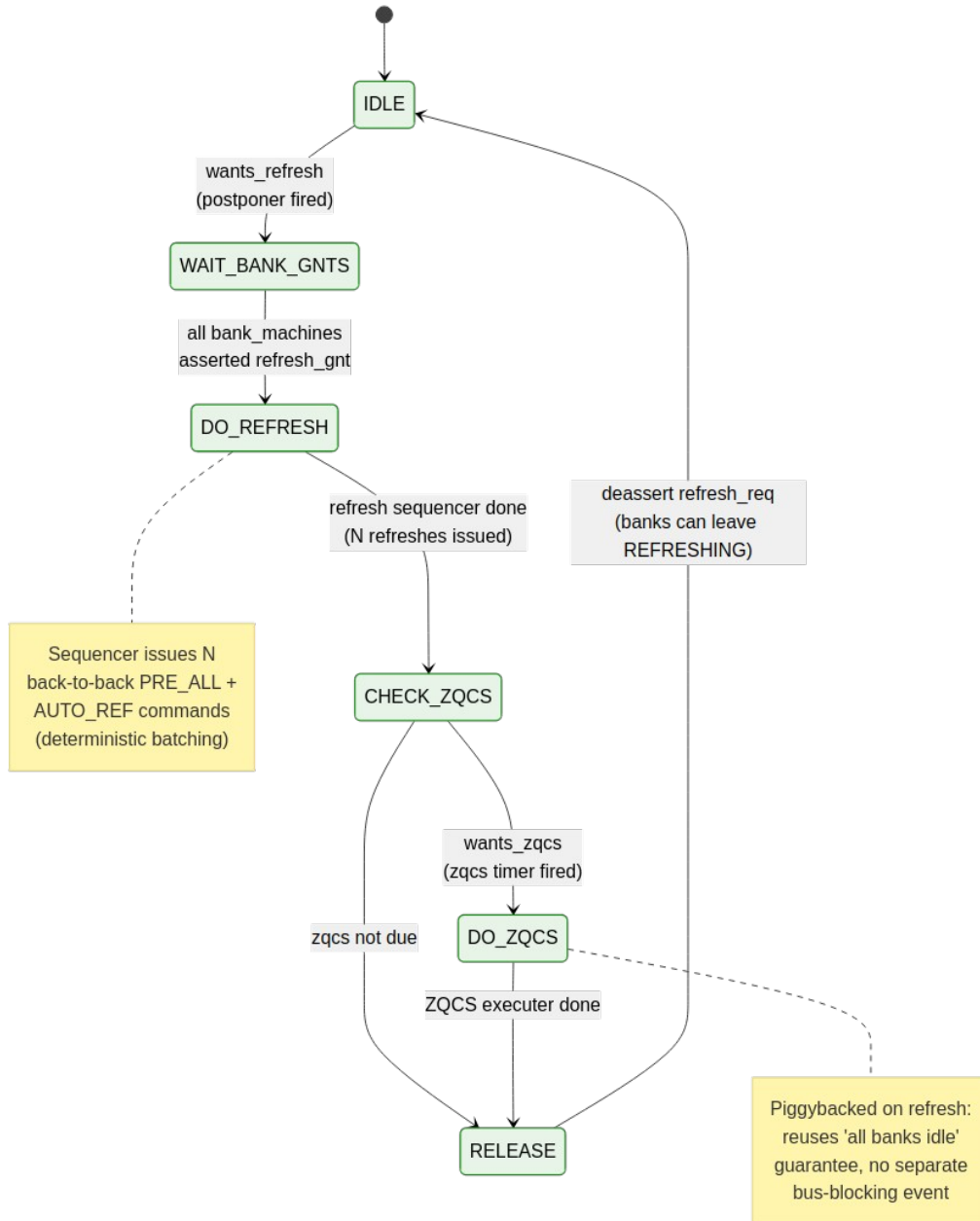
Setting REFRESH_DEFER_MAX = 1 disables batching (every tREFI fires one refresh; equivalent to no postponer).

14.1.4 Temperature-Compensated Refresh (LPDDR2)

LPDDR2 part reports a temperature classification in MR4 which the SoC reads via CSR. The CSR sideband programs a tREFI_scale factor (1x, 0.5x, 0.25x); tREFI_cycles is scaled accordingly. Higher temperature → shorter tREFI → more frequent refresh.

14.1.5 Controller-Level Refresh FSM

The refresh manager hosts a small top-level FSM that orchestrates the bank-grant handshake, the refresh sequencer, and the (piggybacked) periodic ZQCS:



Refresh-Manager Top-Level FSM

Source: [11_refresh_controller_fsm.mmd](#)

State transitions:

- **IDLE** — waits for `wants_refresh` from the postponer

- **WAIT_BANK_GNTS** — asserts refresh_req to all bank machines (REFab) or the selected bank (REFpb); waits for the required refresh_gnt signals
- **DO_REFRESH** — runs the refresh sequencer that issues REFRESH_DEFER_MAX back-to-back PRE_ALL+AUTO_REF cycles, decrementing refresh_owed per issue
- **CHECK_ZQCS** — if wants_zqcs is asserted (periodic ZQCS due), transition to DO_ZQCS; otherwise go to RELEASE
- **DO_ZQCS** — execute ZQCS sequence (PRE_ALL → tRP → ZQCS command → wait tZQCS)
- **RELEASE** — deassert refresh_req; bank machines drop their grants and resume

14.1.6 Periodic ZQCS Piggybacked on Refresh

Periodic ZQ Short Calibration is required for long-running silicon to compensate for thermal drift in DRAM impedance. Earlier revisions of this document put ZQCS only in the init engine step table — meaning ZQCS ran exactly once at boot and never again. That is incorrect for systems that operate for hours or longer.

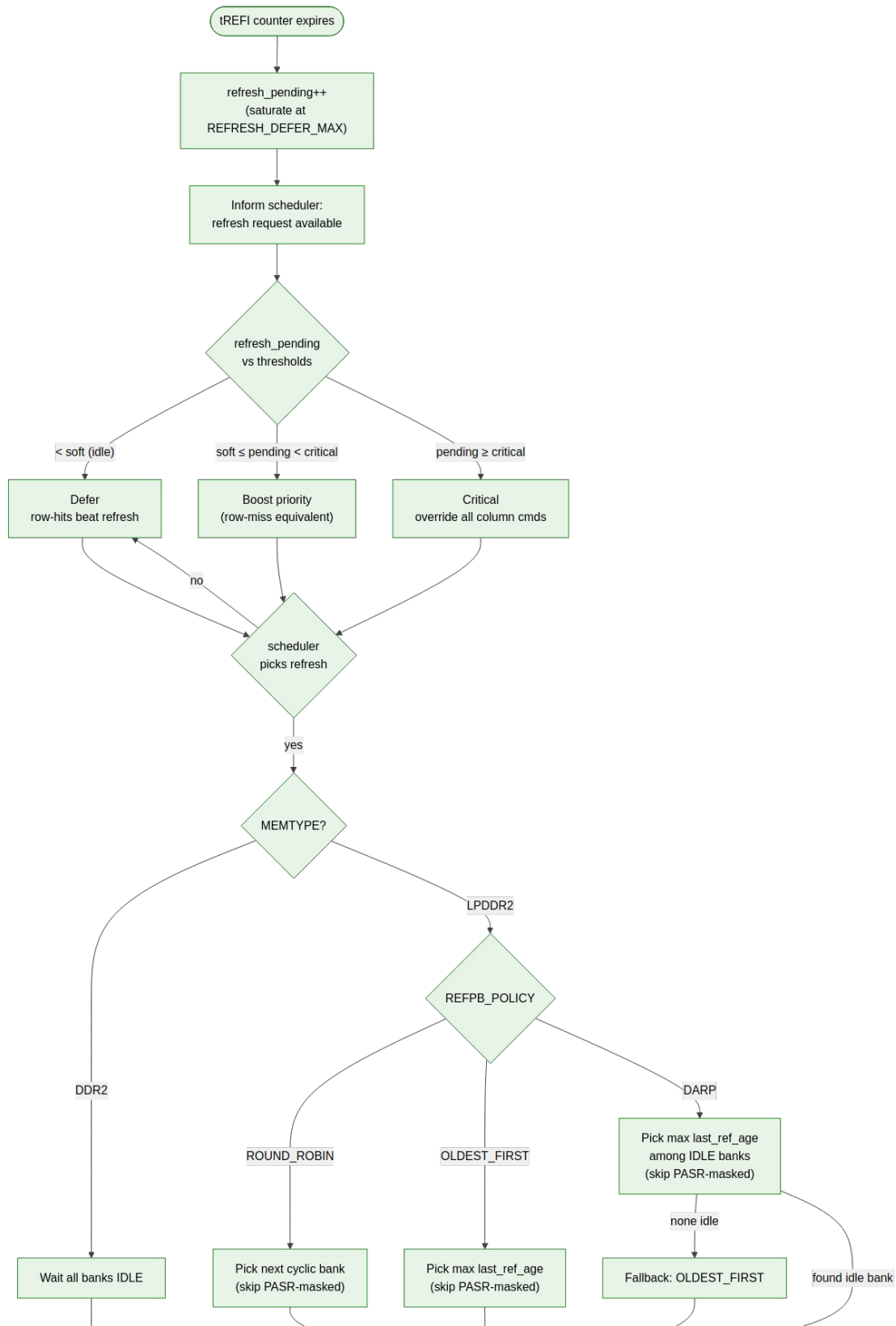
Design: a separate t_zqcs_cnt counter triggers wants_zqcs at the rate set by ZQCS_FREQ_HZ (default 1 Hz). The FSM checks wants_zqcs at the end of a refresh sequence and runs ZQCS while the banks are still in the refresh handshake. This piggybacks on the “all banks idle” guarantee that refresh already provides — no separate bus-blocking event is needed for ZQCS.

ZQCS frequency is independent of refresh frequency. Setting ZQCS_FREQ_HZ = 0 disables periodic ZQCS entirely (init-only behavior, for short-running systems or systems where ZQCS is managed externally).

14.1.7 Temperature-Compensated Refresh (LPDDR2)

LPDDR2 part reports a temperature classification in MR4 which the SoC reads via CSR. The CSR sideband programs a tREFI_scale factor (1x, 0.5x, 0.25x); tREFI_cycles is scaled accordingly. Higher temperature → shorter tREFI → more frequent refresh.

14.1.8 Refresh Decision Flow



Source: [06_refresh_decision.mmd](#)

14.2 REFab Path (DDR2 or LPDDR2 fallback)

1. Pick the target rank $r = \text{round_robin_rank}$ (advanced post-issue).
2. Assert `refresh_req` to **all** bank machines on rank r .
3. Wait for all bank machines on rank r to assert `refresh_gnt` (each does so on reaching IDLE).
4. Issue REF command via the scheduler with $\text{CS}_n[r] = 0$ and all other ranks deselected.
5. Load `tRFCab_cnt` in all bank machines on rank r ; all transition to REFRESHING. Other ranks continue normal operation.
6. Decrement `refresh_owed`. If `refresh_owed > 0`, advance `round_robin_rank` and the sequencer issues another back-to-back PRE_ALL+AUTO_REF cycle (to the next rank).
7. When `refresh_owed == 0`, deassert `refresh_req`; bank machines drop their grants and resume.

REFab blocks all banks **on the targeted rank** for `tRFCab` cycles per refresh; with batching, REFRESH_DEFER_MAX refreshes run back-to-back, each on the next rank in round-robin order. This is the DDR2 default and the LPDDR2 fallback when many banks need refresh simultaneously. For `NUM_RANKS=1` this collapses to the original single-rank flow.

Why per-rank rather than all-rank REFab. Driving REF with all $\text{CS}_n[*] = 0$ would force every rank into REFRESHING simultaneously, wasting bandwidth on ranks that aren't yet due. The round-robin per-rank dispatch keeps the unused ranks available for normal column commands during the multi-rank-batched refresh window.

14.3 REFpb Path (LPDDR2, default)

The REFpb path selects one bank at a time, leaving the others available. Selection policy is parameterized.

14.3.1 Bank Selection Policy

Controlled by `REFPB_POLICY`. The selection space is the full **(rank, bank)** product set across all ranks:

Policy	Selection rule
ROUND_ROBIN	Cyclic through (rank, bank) tuples, skipping per-rank PASR-

Policy	Selection rule
	masked banks
OLDEST_FIRST	Max last_ref_age among non-masked (rank, bank) tuples
DARP (default)	Max last_ref_age among idle non-masked (rank, bank) tuples; if none idle, fall back to OLDEST_FIRST

Per-rank PASR masks are honored independently — bank 3 of rank 0 may be masked while bank 3 of rank 1 is active, and DARP will only pick from the unmasked tuples.

14.3.2 DARP Rationale

DARP (Chang et al., HPCA 2014) wins over ROUND_ROBIN because real workloads have uneven bank activity. The cyclic baseline picks the next bank in sequence regardless of whether it's idle; if that bank happens to be open with a pending column command, the refresh blocks it. DARP picks among the idle banks instead, preserving column-command parallelism.

14.3.3 REFpb Issue Sequence

1. Pick the target bank per REFPB_POLICY.
2. Assert refresh_req to the **selected** bank only.
3. Wait for that bank's refresh_gnt (it asserts on reaching IDLE; PRE if necessary — but DARP minimizes this).
4. Issue REFpb command via scheduler.
5. Load tRFCpb_cnt in that bank → state REFRESHING.
6. Other banks remain in their current states and continue to service column commands.
7. Decrement refresh_owed.
8. When refresh_owed == 0, deassert refresh_req for the selected bank; it drops its grant and resumes.

14.4 Refresh Priority to Scheduler

With deterministic batching, the refresh manager exposes a binary priority to the scheduler:

Condition	Effect
wants_refresh == 0	No request; column commands flow normally
wants_refresh == 1	High priority: scheduler asserts refresh_req immediately on the next cycle

The scheduler always honors `wants_refresh` because the postponer has already absorbed the bandwidth flexibility (up to `REFRESH_DEFER_MAX × tREFI` worth of deferral before any refresh issues). Once the postponer fires, the refresh must complete to keep up with JEDEC's catch-up requirement.

Bandwidth flexibility is concentrated in the deferred-batch window rather than being smeared across a soft-threshold curve. This produces deterministic refresh latency profiles, preferred for real-time-sensitive systems.

14.5 Self-Refresh Coordination

The refresh manager and the power-state FSM coordinate during self-refresh entry / exit:

- The power-state FSM requests self-refresh entry; the refresh manager signals when no auto-refresh is currently in progress.
- During self-refresh, the DRAM handles its own refresh internally; the controller's tREFI counter is paused.
- On self-refresh exit, the refresh manager observes the SR-exit completion signal from the power-state FSM and resumes tREFI counting.

14.6 PASR Update Protocol

The PASR mask is written by the SoC via CSR. The refresh manager treats the mask as advisory until the next self-refresh entry. The Linux PASR Framework (LWN.net) documents the typical update pattern: the kernel writes the mask when it allocates / frees DRAM regions; the controller propagates the mask to the DRAM via MR16/17 writes on the next opportunity (typically during the SR-entry sequence).

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

15 Init Engine and Power State

This section covers the cold-boot initialization engine and the top-level power-state FSM that owns transitions between Active, power-down, and self-refresh / deep-power-down modes.

15.1 init_engine and Step Tables

15.1.1 Purpose

Sequence the cold-boot initialization: CKE control, MR and EMR loads, ZQ calibration. Implemented as a small microprogram sequencer reading a memtype-specific step table ROM.

15.1.2 Microprogram Instruction Format

Each step in the table is a fixed-width record:

Field	Width	Purpose
step_type	4	Opcode: SET_CTRL_BITS, DELAY, MRS_LOAD, ISSUE_CMD, WAIT_FOR_BIT, END
payload_bank	3	Used by MRS and ISSUE_CMD
payload_addr	16	MR address / value, or command address
post_delay	24	Cycles to wait after step completes (scaled by SIM_INIT_SCALE in sim)

15.1.3 Step Tables

Two compile-time-included Verilog packages:

- **ddr2_init_steps_pkg.sv** — DDR2 cold-init sequence
- **lpddr2_init_steps_pkg.sv** — LPDDR2 cold-init sequence

The engine reads its table sequentially. On encountering END, asserts `init_done`.

15.1.4 Init Sequence Outlines

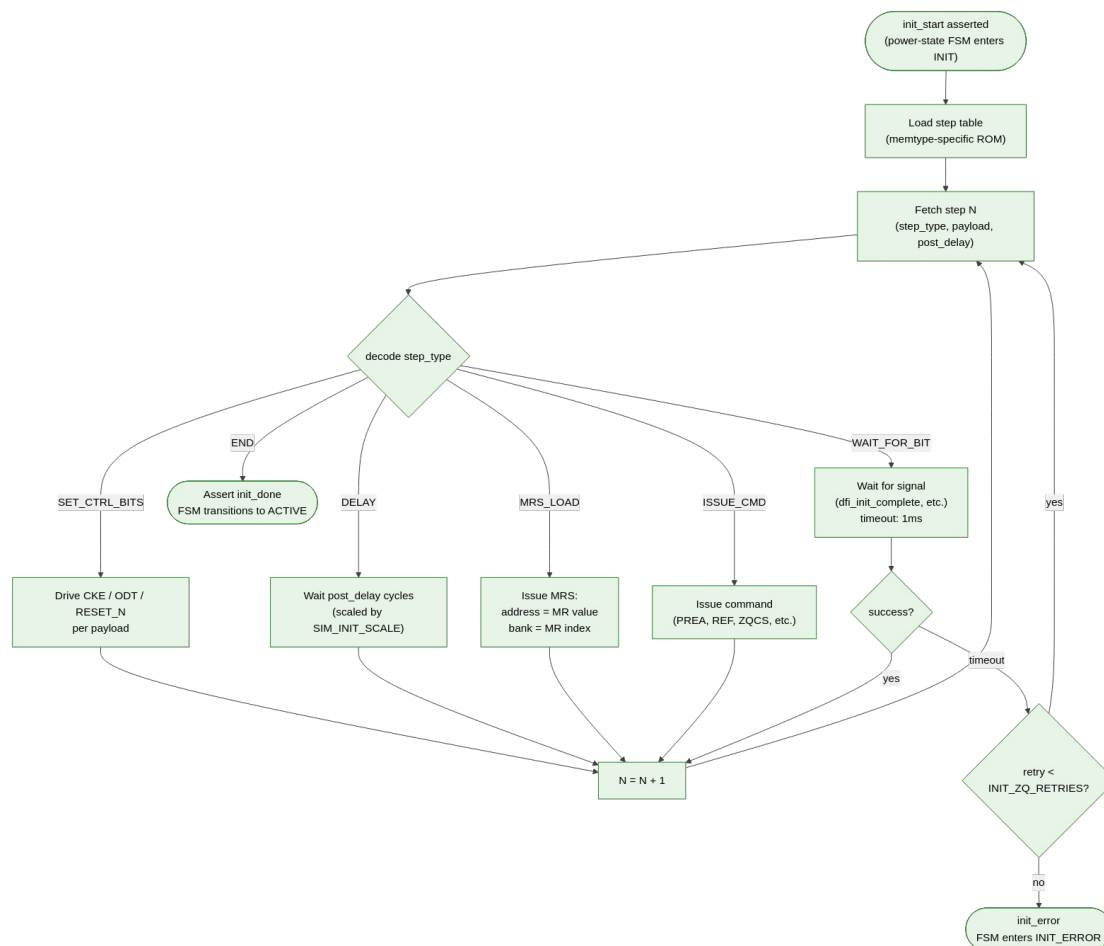
See §6.2 for the full step lists. Summary:

- **DDR2:** 13 steps — power-up wait, CKE high, NOP, PRECHARGE all, EMRS(2/3) defaults, EMRS(1) enable DLL, MRS reset DLL, PRECHARGE all, AUTO REFRESH $\times 2$, MRS final config, EMRS(1) OCD default, EMRS(1) OCD exit, END.
- **LPDDR2:** 13 steps — tINIT1 CKE low, tINIT2 CKE high, tINIT3 NOP, MR63 reset, tINIT5 wait, MR10 ZQ + wait for done, MR1 BL/WRAP/BT, MR2 RL/WL, MR3 drive strength, MR4 read for temp class, MR16 PASR bank mask, MR17 PASR segment mask, END.

15.1.5 Outputs

Signal	Width	Purpose
init_done	1	Asserted after END
init_error	1	Asserted on step failure (timeout, ZQ failure)
init_step_dbg	8	Current step number (for bring-up observability)

15.1.6 Init Engine Flowchart



Init Engine Flow

Source: [05_init_engine_flow.mmd](#)

15.1.7 SIM_INIT_SCALE

Real init sequences include multi-microsecond delays (200 μ s power-up wait, etc.) that are impractical to simulate at cycle granularity. The `SIM_INIT_SCALE` parameter scales delays during

simulation: a value of 1 uses real values; a value of 1000 divides all delays by 1000. Silicon builds always use `SIM_INIT_SCALE = 1`.

15.1.8 Restart on Warm Reset

The engine restarts from step zero on warm-reset deassertion. Cold reset (DRAM content lost) and warm reset (CKE gated only) take the same code path; the step table is identical. This simplifies the FSM and avoids subtle bring-up bugs.

15.1.9 Error Handling

If a `WAIT_FOR_BIT` step times out (e.g., ZQ done never asserts), the engine retries up to 3 times before raising `init_error` and halting. The SoC can clear via CSR and re-issue `init_start`.

15.1.10 MR Value Configurability

MR and EMR values are CSR-configurable before `init_start` is asserted, with sensible defaults baked into the step table. The step table references named CSRs (MR0, MR1, etc.), not literals, so the SoC can override CL, BL, WL, and PASR masks without rebuilding the controller.

15.2 power_state (Top-Level FSM)

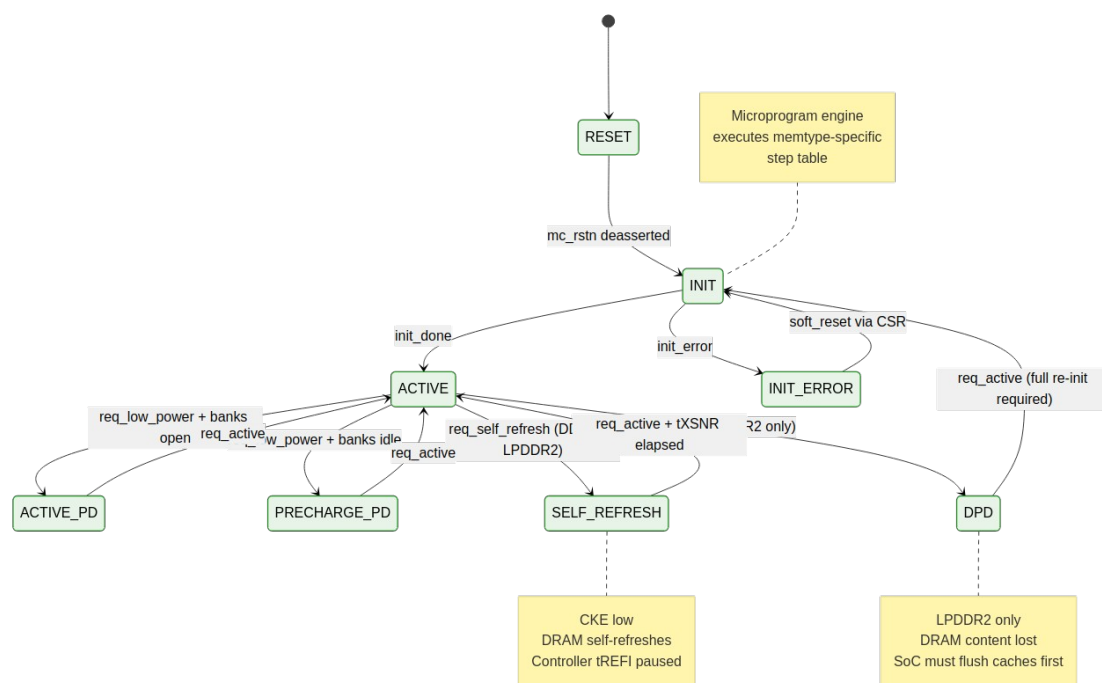
15.2.1 Purpose

Own the top-level power-state transitions. Coordinate with the init engine, refresh manager, and scheduler when entering / exiting low-power states.

15.2.2 FSM States

State	Description
RESET	mc_rstn asserted
INIT	init_engine running
ACTIVE	Normal operation
ACTIVE_PD	Active Power-Down (CKE low, rows kept open)
PRECHARGE_PD	Precharge Power-Down (CKE low, all banks idle)
SELF_REFRESH	DDR2 self-refresh (CKE low, internal refresh)
SR_LP2	LPDDR2 self-refresh (same role, different exit FSM)
DPD	LPDDR2 Deep Power Down (content lost, requires re-init)

15.2.3 Power State Diagram



Power-State FSM

Source: [04_power_state_fsm.mmd](#)

15.2.4 Transitions

Transitions are driven by SoC requests via CSR:

- req_low_power — request entry into APD or PRECHARGE_PD
- req_self_refresh — request entry into SR
- req_dpd — request entry into DPD (LPDDR2 only)
- req_active — wake up to ACTIVE

The FSM defers entry until the refresh manager signals that no auto-refresh is in progress.

15.2.5 Step Tables for Power Transitions

Power-state transitions reuse the microprogram engine pattern:

- sr_entry_steps_pkg.sv — common entry sequence (drain pending, CKE low)

- `sr_exit_steps_pkg.sv` — common exit sequence (CKE high, wait `tXSNR`, resume)
- `dpd_entry_steps_pkg.sv` — LPDDR2 DPD entry
- `dpd_exit_steps_pkg.sv` — LPDDR2 DPD exit (which re-runs the full init sequence)

15.2.6 DPD Entry Constraints

Deep Power Down loses all DRAM content. The controller does not enforce that the SoC has flushed dirty caches before requesting DPD — that’s the SoC’s responsibility. The controller simply executes the transition. Misuse will cause data corruption, which is the price of the deepest-sleep mode.

15.2.7 Self-Refresh Exit Latency

JEDEC requires a self-refresh-exit-to-next-command latency `tXSNR` (~200 ns for DDR2). The power-state FSM honors this with an internal counter; the scheduler is gated until the counter expires.

15.2.8 Power-State Observation

Current state is exposed in the STATUS CSR (bits [7:4]), and the recent transition history (last 8 transitions) is exposed in STATUS_HISTORY for bring-up debugging.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

16 Command Encoder and Gear Output

This section covers the two modules that translate the controller’s internal generic command bus into DFI wire signals: `cmd_encoder` (memtype-specific wire mapping) and `gear_out` (1-phase to N-phase replication).

16.1 `cmd_encoder`

16.1.1 Purpose

Translate the controller’s internal generic command record into the DFI wire signals appropriate for the target memory. The encoder is the only place where DDR2 and LPDDR2 differ at the wire layer.

16.1.2 Input Record

The internal generic command is:

```
{
  cmd      : { NOP, DESEL, ACT, RD, WR, RDA, WRA,
               PRE, PREA, REF, REFPB, MRS, ZQCS, ZQCL },
  bank     : log2(NUM_BANKS),
  row      : ROW_WIDTH,
  col      : COL_WIDTH,
  ap       : 1,           // auto-precharge bit
  all_banks : 1,           // all-banks variant of PRE / REF
  mr_addr  : 8,           // MR / EMR address
  mr_data  : 16           // MR / EMR data (LPDDR2 only; DDR2 packs
into addr)
}
```

16.1.3 DDR2 Encoder

Outputs the standard DDR2 command-bus signals per JESD79-2 Table 67:

- `cs_n` — asserted (low) on command issue; high in idle
- `ras_n`, `cas_n`, `we_n` — set per the JESD79-2 lookup table
- `bank` — driven from the command record's bank field
- `address` — driven from `row` for ACT; from `col` for RD / WR (with `addr[10]` carrying the AP bit for column commands or the AB bit for PRE)
- `cke` — held high during normal operation; lowered by the power-state FSM
- `odt` — driven from a small lookup based on transaction type
- `reset_n` — DDR2 has no `reset_n`; held high

16.1.4 LPDDR2 Encoder

Per DFI v2.1 Table 1, LPDDR2 carries the command on the `dfi_address` 20-bit CA word:

- `cs_n` — asserted on command issue
- `ras_n`, `cas_n`, `we_n`, `bank` — held at their idle values (1, 1, 1, 0)
- `address[19:0]` — packed CA word via the algorithm in our existing `lpddr_ca` encoder (`src/CocoTBFramework/components/dfi/lpddr_ca.py`):
 - `address[9:0]` = CA cycle 1
 - `address[19:10]` = CA cycle 2
- `cke` — used by the power-state FSM (LPDDR2 has the richer CKE state machine)

- odt — LPDDR2 has no ODT; held low
- reset_n — held high during normal operation

16.1.5 Memtype Selection

At elaboration, the appropriate encoder is instantiated based on MEMTYPE. Only one is synthesized; the other path is dead code.

16.1.6 Idle Defaults

When the controller is not issuing a command, the encoder drives the bus to its deselected idle state:

- DDR2: cs_n=1, ras_n=1, cas_n=1, we_n=1, bank=0, address=0, odt=0
- LPDDR2: cs_n=1, address=0

This convention matches the DFI v2.1 Table 2 default values.

16.1.7 Verification

Per-encoder unit tests:

- DDR2 encoder: round-trip table test against the JESD79-2 truth table
 - LPDDR2 encoder: leverages our existing 30 round-trip tests for the CA bus encoder (tests/unit/test_lpddr_ca.py in the DV repo)
-

16.2 gear_out

16.2.1 Purpose

Replicate the 1-phase internal command bus onto N parallel DFI output phases, where N_PHASES $\in \{1, 2, 4\}$.

16.2.2 Behavior

- Phase 0 by convention carries the active command on the cycle it's issued.
- Other phases idle (drive their deselected defaults).
- For N_PHASES == 1: pass-through. Synthesis will optimize the module away.
- For N_PHASES == 2 or 4: the output stage tracks which phase a command should land on. Some implementations assign the phase based on

workload; ours uses the simpler “always phase 0” rule and relies on the PHY to serialize correctly per the DFI frequency-ratio spec.

16.2.3 Multi-Cycle Commands

LPDDR2’s 2-cycle commands are **not** split here. They are packed into a single 20-bit `dfi_address` word by the encoder; the PHY’s job is to split them into two DRAM cycles per DFI v2.1 §3.1.

16.2.4 Phase Naming

Per-phase signals follow the DFI naming convention with `_p0` through `_pN_PHASES - 1` suffixes. For `N_PHASES == 1`, the suffix may be elided per DFI v2.1 §3.1 (“phase 0 may exclude the suffix”), but this controller always includes the suffix for naming clarity.

16.2.5 Write / Read Data Phase Assignment

The write-data and read-data sub-interfaces also replicate per phase. Two additional parameters control phase placement:

- `WRPHASE` — which phase carries the active write-data beat (default: phase 0)
- `RDPHASE` — which phase carries the active read-data beat (default: phase 0)

These are PHY-vendor-specific in real silicon; we make them parameters so a real-PHY integration can override.

16.2.6 Idle Phase Drives

The gear output stage drives the per-phase signals to their deselected idle values per DFI v2.1 Table 2 when no command is being issued on that phase. This includes both the inactive phases when a single-phase command is issued, and all phases when the scheduler issues NOP.

16.2.7 Verification

Reuses the DFI BFM’s `DFIPhaseAdapter` (Python) as a reference model — the same gear ratios, the same conventions. The RTL output passes through the adapter and round-trips via the BFM monitor.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

17 Write and Read Data Paths

The write and read data paths bridge the AXI W / R channels and the DFI wrdata / rddata sub-interfaces with the JEDEC-required CWL / CL timing alignment.

17.1 wdata_path

17.1.1 Purpose

Stream AXI W beats onto the DFI write-data bus with correct CWL alignment relative to the issued WR command.

17.1.2 Inputs

- AXI W channel beats (after `axi4_slave` accepts them into the internal FIFO)
- Issued WR command notification from `scheduler` (includes AXI ID and burst length)
- CWL value from `csr_slave` (programmed at init)
- `WRPHASE` parameter from elaboration

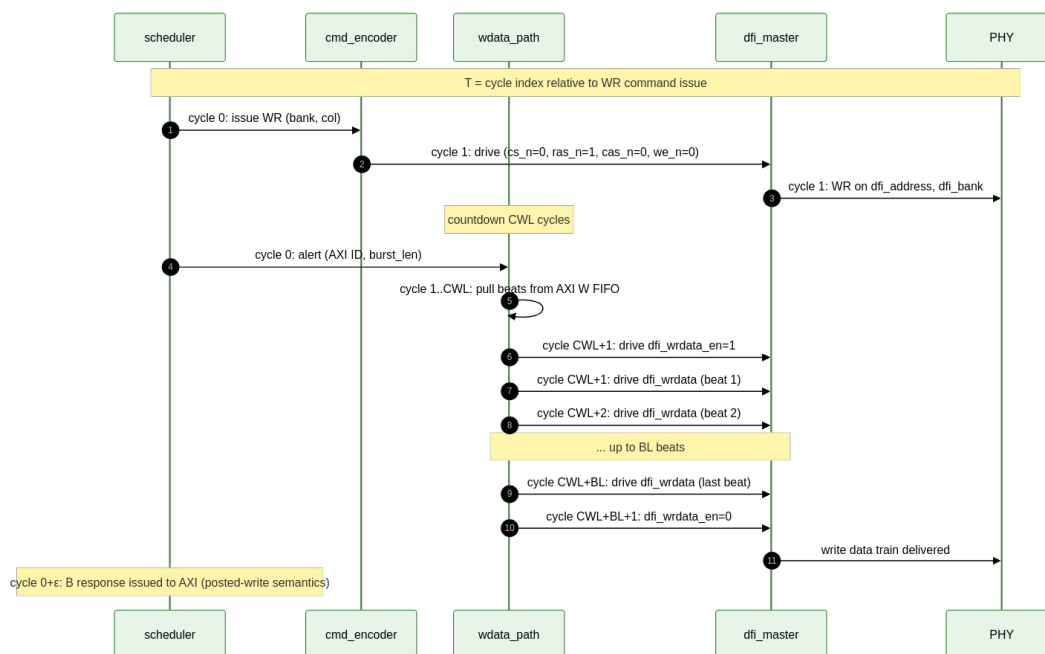
17.1.3 Outputs

- `dfi_wrdata` — write data, driven on the correct phase
- `dfi_wrdata_en` — driven high CWL cycles after the WR command for the burst duration
- `dfi_wrdata_mask` — byte mask, driven from inverted AXI strobes (DFI convention: 1 = mask, 0 = write)

17.1.4 Behavior

1. On WR command issue, the scheduler tags the AXI ID and burst length.
2. The write-data path begins a CWL-cycle countdown.
3. After CWL cycles, the path pulls beats from the AXI W FIFO matching the tagged ID and drives them onto `dfi_wrdata` for the burst duration.
4. `dfi_wrdata_mask` is computed from the AXI `strb` field with bit inversion (AXI: 1 = byte enable; DFI: 1 = mask).
5. After the last beat, `dfi_wrdata_en` deasserts.

17.1.5 Write Command Pipeline



WR Command Pipeline with CWL Alignment

Source: [07_wr_pipeline.mmd](#)

17.1.6 B-Response Generation

A B-channel response is generated when the WR command is **scheduled**, not when the data commits to the DRAM array. This matches AXI4 posted-write semantics and decouples B latency from DRAM-side delays.

17.1.7 Write Cancellation

Not supported. Once a WR is committed by the scheduler, it must drain. The AXI master must not retract a posted write.

17.2 rdata_path

17.2.1 Purpose

Sample DFI rddata with correct CL alignment relative to the issued RD command, and stream the beats back on the AXI R channel with the correct ID and burst tagging.

17.2.2 Inputs

- DFI `rddata`, `rddata_valid` — driven by the PHY
- Issued RD command notification from scheduler (AXI ID, burst length)
- CL value from `csr_slave`
- `RDPHASE` parameter

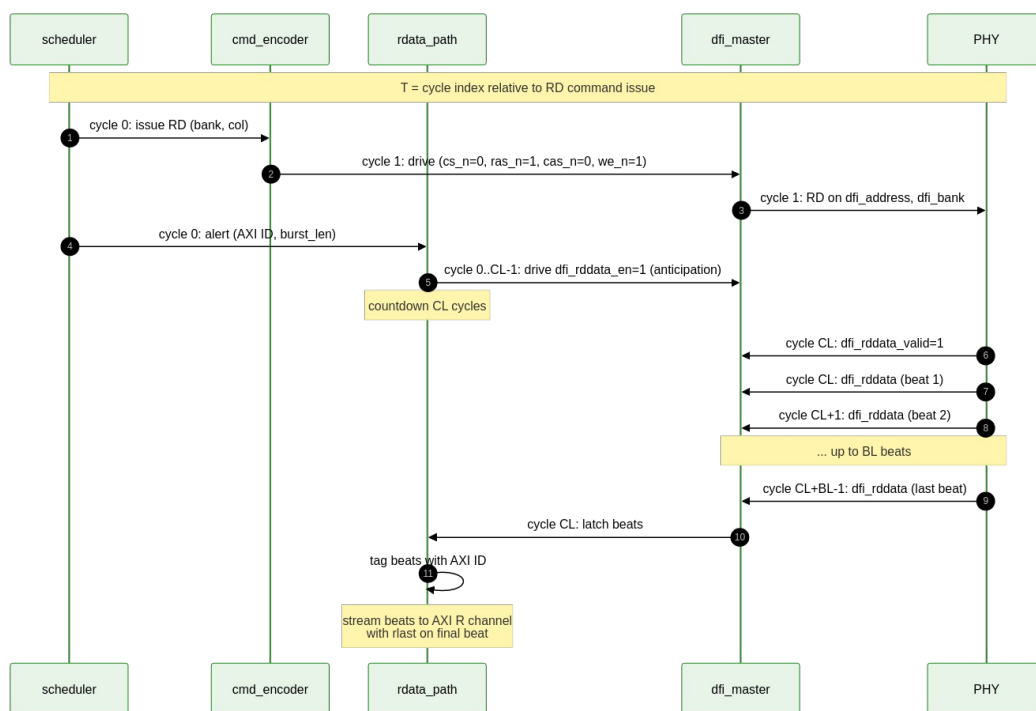
17.2.3 Outputs

- AXI R channel — `rdata`, `rid`, `rresp`, `rlast`, `rvalid`

17.2.4 Behavior

1. On RD command issue, the scheduler tags the AXI ID and burst length.
2. The read-data path begins a CL-cycle countdown.
3. After CL cycles, the path samples `rddata` and `rddata_valid` for the burst duration.
4. Valid beats are tagged with the original AXI ID and streamed onto the AXI R channel.
5. `rlast` is asserted on the final beat of each AXI transaction.

17.2.5 Read Command Pipeline



RD Command Pipeline with CL Alignment

Source: [08_rd_pipeline.mmd](#)

17.2.6 Out-of-Order Completion

The path supports out-of-order R-channel completion across AXI IDs (when AXI_000_ACROSS_IDS is enabled). Within an ID, ordering is preserved.

17.2.7 Read Error Handling

If the burst's underlying PHY reports an issue (e.g., uncorrected parity error — though DDR2 / LPDDR2 don't have CA parity), rresp is set to SLVERR for the affected beats. In v1 of this controller, all reads return OKAY.

17.2.8 Read Data Capture Timing

CL is programmed via CSR at init. The path uses the programmed value directly; it does not learn CL from the PHY. This works because LPDDR2 init sequence has the controller program MR2 with the desired CL / CWL values, and the same value is mirrored in CSR.

17.2.9 Stale Read Suppression

If the AXI R channel is back-pressured for an extended period, the path will eventually report a timeout. The default timeout is 1024 cycles; configurable via CSR. The timeout indicates a downstream consumer is stuck, which is typically a system-level bug rather than a controller bug.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

18 DFI Master and CSR Slave

The two final modules in the controller hierarchy: `dfi_master` (final DFI signal aggregation) and `csr_slave` (APB3 configuration interface).

18.1 `dfi_master`

18.1.1 Purpose

Aggregate command, write-data, read-data, and init-control signals into the final DFI v2.1 signal set. Tie off the unused DFI sub-interfaces to their idle values. Provide a single point of contact with the PHY for clean integration.

18.1.2 Sub-Interfaces Driven

This controller drives only the three core DFI sub-interfaces required for v2.1 LPDDR2 operation:

- **Command Interface** (MC → PHY): all command-bus signals
- **Write Data Interface** (MC → PHY): `wrdata`, `wrdata_en`, `wrdata_mask`
- **Read Data Interface** (MC ↔ PHY): `rddata`, `rddata_valid`, `rddata_en`, and LPDDR2's `rddata_dnv`

18.1.3 Sub-Interfaces Tied Idle

The remaining DFI v2.1 sub-interfaces are present in the port list (for ABI compatibility) but driven to their idle values:

- **Status Interface** — only `init_complete` is consumed; others tied off
- **Update Interface** — MC drives `ctrlupd_req` = 0; PHY's `phyupd_req` is acked but no action taken
- **Training Interface** — held idle; no training in DDR2 / LPDDR2

- **Low Power Control Interface** — held idle; we use CKE-based power management directly

This means the controller is compatible with PHYs that support full DFI v2.1 but doesn't require them to.

18.1.4 Signal Aggregation

All per-phase signals from `cmd_encoder` (through `gear_out`), `wdata_path`, and `rdata_path` arrive at `dfi_master` and are routed to the appropriate output ports. The module is essentially a wiring stub plus tie-offs.

18.1.5 Why a Dedicated Module

Putting the tie-offs in a single module keeps the rest of the controller PHY-detail-free. If we later need to enable a sub-interface (e.g., the training interface for a real PHY that requires it), the change is localized.

18.2 `csr_slave`

18.2.1 Purpose

APB3 slave for configuration and observation. All controller parameters that need to be programmable at runtime (rather than build time) are exposed here.

18.2.2 Bus Protocol

Standard APB3 slave. Signals:

Signal	Width	Direction	Purpose
<code>psel</code>	1	input	Slave selected by APB master
<code>penable</code>	1	input	Access phase 2 (data transfer)
<code>pwrite</code>	1	input	Write (1) or read (0)
<code>paddr</code>	12	input	Address (4 KB region)
<code>pwdata</code>	32	input	Write data
<code>prdata</code>	32	output	Read data
<code>pready</code>	1	output	Slave ready to complete
<code>pslverr</code>	1	output	Error response

18.2.3 Address Map Overview

A 4 KB region with the following high-level partitioning:

Range	Purpose
0x000 – 0x00F	Top-level control and status
0x010 – 0x01F	Packed timing parameters
0x020 – 0x02F	Mode-register configuration values
0x030 – 0x03F	LPDDR2-specific (PASR, etc.)
0x040 – 0x04F	Scheduler / refresh / page-policy tuning
0x080 – 0x0FF	Per-bank observation counters
0x100 – 0x1FF	System-wide observation counters
0xFF0 – 0xFFC	Module identification

The full register map is documented in Section 6.3.

18.2.4 Clock Domain Crossing

The CSR slave runs on `apb_pclk`, which is independent of `mc_clk`. A 2-flop synchronizer with handshake protocol crosses configuration writes into the `mc_clk` domain. The handshake guarantees atomic register update from the controller’s perspective.

18.2.5 Read Latency

APB read latency is implementation-defined. This slave uses single-cycle reads for control / status registers (the `paddr ≤ 0x07F` range) and 2-cycle reads for observation counters (which require CDC of the live counter value into the APB domain). `pready deasserts` during the second cycle of an observation-counter read.

18.2.6 Configuration Quiet Period

Certain parameters (e.g., MR values, timings) can only take effect during a “quiet period” — when no DRAM commands are in flight. The CSR slave does not enforce quiet-period writes; the SoC is responsible for sequencing. Writing during normal operation may cause subtle protocol violations.

18.2.7 Recommended Programming Sequence

1. Hold `mc_rstn` asserted.

2. Program all timing parameters and MR values via CSR.
3. Configure PASR mask (LPDDR2 only).
4. Configure characterization parameters (PAGE_POLICY runtime override, REFPB_POLICY runtime override, etc.).
5. Release mc_rstn.
6. Wait for STATUS.init_done == 1.
7. Begin AXI traffic.

Skipping CSR programming uses the build-time defaults, which are conservative (HAPPY_HYBRID for LPDDR2 page policy, DARP for REFPB policy).

19 AXI4 Slave Interface

The controller exposes a standard AXI4 slave for SoC connectivity.

19.1 Channels

Standard AXI4 — all five channels (AW, W, B, AR, R) are mandatory. No exclusive access support; AxLOCK is ignored.

19.1.1 AW (Write Address) — SoC master → controller slave

Signal	Width	Notes
awid	AXI_ID_WIDTH	Transaction ID
awaddr	AXI_ADDR_WIDTH	Flat byte address
awlen	8	Burst length minus 1
awsize	3	Beat size; up to bus width
awburst	2	INCR mandatory; FIXED/WRAP optional
awcache	4	Ignored; treated as 0010 (modifiable)
awprot	3	Ignored
awqos	4	Forwarded to scheduler as priority hint
awvalid	1	Master asserts

Signal	Width	Notes
awready	1	Controller asserts when slot available

19.1.2 W (Write Data)

Signal	Width	Notes
wdata	AXI_DATA_WIDTH	Beat data
wstrb	AXI_DATA_WIDTH/8	Byte enables; inverted to wrdata_mask
wlast	1	Final beat of burst
wvalid	1	Master asserts
wready	1	Controller asserts on FIFO space

Note: AXI4 W channel has no wid; beats are presumed in the order their AW transactions were issued. The controller tracks IDs internally based on AW arrival order.

19.1.3 B (Write Response)

Signal	Width	Notes
bid	AXI_ID_WIDTH	Matches awid
bresp	2	OKAY or SLVERR
bvalid	1	Controller asserts on response ready
bready	1	Master asserts

19.1.4 AR (Read Address)

Signal	Width	Notes
arid	AXI_ID_WIDTH	Transaction ID
araddr	AXI_ADDR_WIDTH	Flat byte address
arlen	8	Burst length minus 1
arsize	3	Beat size
arburst	2	INCR mandatory; others optional
arcache	4	Ignored
arprot	3	Ignored
arqos	4	Forwarded to scheduler as priority
arvalid	1	Master asserts

Signal	Width	Notes
aready	1	Controller asserts when slot avail

19.1.5 R (Read Data)

Signal	Width	Notes
rdata	AXI_DATA_WIDTH	Beat data
rid	AXI_ID_WIDTH	Matches arid
resp	2	OKAY (v1 always)
rlast	1	Final beat of burst
rvalid	1	Controller asserts when beat ready
ready	1	Master asserts

19.2 Ordering Rules

- **Within an ID:** AXI4 ordering preserved. Reads from arid = X return in AR order; writes to awid = X commit in AW order.
- **Across IDs:** Out-of-order completion permitted by default (AXI_000_ACROSS_IDS = true). Disabling forces in-order across IDs at the cost of bandwidth.

19.3 Burst Type Support

Burst	Mandatory	Notes
FIXED	optional	Enable with SUPPORT_FIXED_BURST; rarely useful for DRAM
INCR	mandatory	Standard incrementing burst
WRAP	optional	Enable with SUPPORT_WRAP_BURST

19.4 Burst Length

AXI4 allows up to 256 beats. If a burst exceeds the remaining columns in its target row, axi4_slave splits internally; the SoC sees a single AW with the expected response time.

19.5 Narrow Transfers

AXI4 supports `awsz < bus width`. The narrow case is handled via `wstrb` — the unused byte lanes are masked. The DRAM commits only the unmasked bytes.

19.6 Posted Writes

B-channel responses are generated when the WR command is **scheduled**, not when `wrdata` commits. This is AXI4-compliant (OKAY indicates the transaction was accepted, not committed) and reduces master-side latency.

19.7 Backpressure

- `AW / AR .ready` deassert when `txn_queue` is full
- `W .ready` deasserts when the write-data FIFO is full
- `R / B .valid` follow standard protocol; consumers may stall indefinitely without controller misbehavior

19.8 AXI4 Reset

`axi_aresetn` is the AXI reset. When asserted, all AXI state is cleared. The internal controller pipeline is also reset (the two reset signals are tied together in v1 of this controller).

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

20 DFI v2.1 Master Interface

The controller drives DFI v2.1 master signals to the PHY. DFI v2.1 is the first DFI revision that supports LPDDR2; this controller targets that revision.

20.1 Per-Phase Signal Naming

DFI v2.1 §3.1 defines per-phase signals with `_pN` suffix where `N` is the phase index. For `N_PHASES = 4`, signals exist as `_p0`, `_p1`, `_p2`, `_p3`. For `N_PHASES = 1`, signals can elide the suffix; this controller always emits the suffix for clarity.

20.2 Command Interface (MC → PHY)

Signal	Width	Notes
dfi_address	DFI_ADDR_WIDTH	DDR2: row/col address; LPDDR2: 20-bit packed CA word
dfi_bank	BANK_WIDTH	DDR2: bank; LPDDR2: held idle (per Table 1)
dfi_cas_n	1	DDR2: CAS encoding; LPDDR2: held idle
dfi_cke	1	Clock enable
dfi_cs_n	1	Chip select; asserted on command issue
dfi_odt	1	DDR2 only; LPDDR2 held low
dfi_ras_n	1	DDR2: RAS encoding; LPDDR2: held idle
dfi_reset_n	1	DDR2 only (DDR2 has no reset); LPDDR2 driven
dfi_we_n	1	DDR2: WE encoding; LPDDR2: held idle

20.3 Write Data Interface (MC → PHY)

Signal	Width	Notes
dfi_wrdata	DFI_DATA_WIDTH	Write data; driven CWL cycles after WR
dfi_wrdata_en	1	High during valid write data beats
dfi_wrdata_mask	DFI_DATA_WIDTH/8	1 = mask byte; 0 = write (DFI convention)

20.4 Read Data Interface (PHY ↔ MC)

Signal	Width	Direction	Notes
dfi_rddata	DFI_DATA_WIDTH	PHY → MC	Read data, CL cycles after RD
dfi_rddata_valid	1	PHY → MC	High when beat is valid
dfi_rddata_en	1	MC → PHY	High when MC expects to capture read
dfi_rddata_notvalid	1	PHY →	LPDDR2 only: Data-Not-Valid

Signal	Width	Direction	Notes
		MC	

20.5 Status Interface

Only `init_complete` is consumed:

Signal	Width	Direction	Notes
<code>dfi_init_complete</code>	1	PHY → MC	High when PHY has finished its init

Other status signals (`dfi_data_byte_disable`, etc.) are not consumed.

20.6 Sub-Interfaces NOT Driven

The following DFI v2.1 sub-interfaces exist in the port list but are tied to idle by `dfi_master`:

- **Update Interface** — `ctrlupd_req`, `phyupd_req`, etc.
- **Training Interface** — all `rdlvl` / `wrlvl` signals
- **Low Power Control Interface** — handled via CKE in command interface instead

20.7 Reset

Signal	Width	Direction	Notes
<code>dfi_rstn</code>	1	MC → PHY	DFI reset; controlled by <code>init_engine</code>

20.8 Timing Parameters Honored

The controller respects the following DFI v2.1 timing parameters:

- `tCTRL_DELAY` — PHY-reported command-launch delay (read from PHY's reported value; programmed via CSR if PHY has no advertise mechanism)
- `tPHY_WRDATA` — write data launch delay relative to WR command (default 0; programmable)
- `tPHY_RDLAT` — read data return latency relative to RD command (default = CL; programmable)

20.9 Clock Domain

DFI signals are launched on `mc_clk` (the controller side). The PHY consumes them on `phy_clk` which runs at $N_PHASES \times mc_clk$ rate. The gear output stage handles the phase replication; the PHY handles the actual serialization to DRAM cycles.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

21 APB CSR Slave

The controller exposes an APB3 slave port for configuration and observation.

21.1 Signals

Signal	Width	Direction	Notes
<code>apb_pclk</code>	1	input	APB bus clock (independent of <code>mc_clk</code>)
<code>apb_presetn</code>	1	input	APB reset (active low)
<code>psel</code>	1	input	Slave selected
<code>penable</code>	1	input	Access phase 2
<code>pwrite</code>	1	input	Write (1) or read (0)
<code>paddr</code>	12	input	4 KB address space
<code>pwrdata</code>	32	input	Write data
<code>prdata</code>	32	output	Read data
<code>pready</code>	1	output	Slave ready
<code>pslverr</code>	1	output	Error response

21.2 APB3 Conformance

Standard APB3 — single-cycle accesses for the typical case, multi-cycle for observation-counter reads (which require CDC).

21.3 Error Behavior

`pslverr` is asserted in these cases:

- Write to a read-only register (e.g., STATUS)
- Read from a write-only register (none defined in v1)
- Access to an unimplemented address (any address not listed in the register map)

Errors do not affect controller state — the APB transaction simply terminates with `pslverr = 1`.

21.4 Clock Domain Crossing

The CSR slave operates on `apb_pclk`, independent of `mc_clk`. A 2-flop synchronizer handshake transfers configuration writes into the `mc_clk` domain. The handshake guarantees atomic register update from the controller's perspective.

Observation reads use a parallel CDC mechanism: snapshot the live counter in the `mc_clk` domain, then transfer to `apb_pclk`. `pready` deasserts during the transfer (typically 2 `apb_pclk` cycles).

21.5 Register Map

The full address map is documented in Section 6.3.

21.6 Configuration Sequencing

See `csr_slave` description in §3.8 for the recommended programming sequence:

1. Hold `mc_rstn` asserted
2. Program timing parameters and MR values
3. Configure PASR mask (LPDDR2)
4. Configure characterization parameters
5. Release `mc_rstn`
6. Wait for `STATUS.init_done`
7. Begin AXI traffic

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

22 Build-Time vs. Runtime Configuration

This controller distinguishes between **build-time parameters** that set the silicon footprint and **runtime CSR registers** that select behavior within that footprint. The two layers serve different purposes and should not be confused. This chapter applies the principle to every configurable knob in the design.

22.1 Principle

Layer	What it controls	Frozen at	Cost of changing
Build-time parameter	Synthesized silicon footprint: signal widths, table depths, queue sizes, instantiation counts, encoder selection, MAX bounds of runtime values	Elaboration time	Full RTL rebuild + place-and-route + tape-out (silicon) or bitstream rebuild (FPGA)
Runtime CSR register	Behavior selection <i>within</i> the synthesized footprint: policy choices, threshold values, on/off toggles, active counts up to the build-time MAX	Programmable any time the configuration quiet point is reached (typically during init or under SoC orchestration)	A single APB write

The architectural rule is straightforward: **if a knob is changing silicon area, it must be build-time. If a knob is only changing behavior of already-synthesized hardware, it must be runtime.**

22.2 Why This Distinction Matters

A controller designed without this distinction tends to collapse one of two ways:

- **Too much build-time** — every algorithmic choice is a parameter that requires a new bitstream / silicon revision to evaluate. Characterization

becomes impossibly slow; post-silicon debug becomes impossible. Most academic prototype controllers fall here.

- **Too much runtime** — every option is always synthesized, including ones that will rarely be used. Area balloons; timing closure suffers; the design becomes harder to verify because every code path is live. This is the failure mode of “ultra-configurable” commercial IP.

This controller targets the middle: **build-time picks what’s physically present; runtime picks what’s active.**

22.3 The Hybrid Pattern: “Build-Time MAX + Runtime Active”

Many of this controller’s knobs follow a hybrid pattern where the build-time parameter sets a synthesized MAX, and a paired runtime CSR field selects the actually-used value within that MAX.

Concrete example for LOOKAHEAD_DEPTH:

Build-time: LOOKAHEAD_DEPTH_MAX = 4
 → synthesizes 4 row-address comparators per bank
machine

Runtime CSR: sched_tuning.lookahead_active [3:0]
 → scheduler peeks min(LOOKAHEAD_DEPTH_MAX,
lookahead_active)
 same-bank requests per cycle

For debug: Write lookahead_active = 0 to force pure HAPPY/page-
policy mode
 Write lookahead_active = 4 to use the full synthesized
window
 No rebuild required.

The same pattern applies to TXN_QUEUE_DEPTH, AGE_MAX, REFRESH_DEFER_MAX, INIT_ZQ_RETRIES, ZQCS_FREQ_HZ, and others — area is paid once at synthesis for the MAX; runtime selects how much of the synthesized hardware is actively gated.

22.4 The “Build-Time + Runtime Mux” Pattern

For algorithmic choices where multiple alternatives have similar area cost, the build-time parameter selects **which** code paths are synthesized, and a runtime CSR field picks **which** of the synthesized paths is active.

Concrete example for PAGE_POLICY:

Build-time: `PAGE_POLICY = HAPPY_HYBRID`
 → synthesizes OPEN, CLOSE, and HAPPY decision logic
 (plus the HAPPY predictor table)

Runtime CSR: `refresh_tuning.page_policy_or [1:0]`
 00 = use build-time default (HAPPY_HYBRID here)
 01 = force OPEN
 10 = force CLOSE
 11 = force HAPPY_HYBRID

For debug: Sweep policies on real workloads with no rebuilds.

The build-time choice `PAGE_POLICY = OPEN` would have omitted the HAPPY predictor table from the synthesized design entirely, saving area but losing the ability to switch to HAPPY at runtime. The build-time choice `PAGE_POLICY = HAPPY_HYBRID` synthesizes everything, paying the area cost for the predictor table, in exchange for the ability to switch among any of the three at runtime.

The same pattern applies to `REFPB_POLICY`, `ADDR_MAP_SCHEME`, `SCHEDULER_MODE`, and HAPPY enable/disable.

22.5 The “Runtime Only” Pattern

A small number of knobs have zero area cost and are runtime-only: simple counter compare values, timeout thresholds, retry counts. Examples include `ZQCS_FREQ_HZ`, `INIT_ZQ_RETRIES`, and the various observation-counter clear-on-read controls.

These have no build-time presence at all.

22.6 Configuration Quiet Points

Runtime CSR writes that change scheduler or refresh-manager behavior take effect at the next **configuration quiet point** — defined as: no DRAM commands in flight, no refresh sequence in progress, and the relevant FSM in its idle state. The CSR slave does not enforce quiet-point writes; the SoC firmware is responsible for sequencing. Writing during normal operation is well-defined but the change does not apply until the next quiet point — which can be enforced by writing to the `CTRL.config_apply` strobe to force a brief drain.

Writes to register-file CSRs that have no FSM impact (timing parameter overrides, MR override values, observation counters) take effect immediately and have no quiet-point requirement.

22.7 Summary

The following two chapters document this controller’s complete configuration interface:

- **§5.2 Top-Level Parameter Table** — every build-time parameter, with its type, range, governing section, and (where applicable) the paired runtime CSR field.
- **§5.3 Characterization Knobs** — the subset of build-time + runtime knobs intentionally exposed for performance sweeping during silicon / FPGA characterization.

The full runtime CSR register map is in §6.3.

23 Top-Level Parameter Table

All build-time parameters for the controller, with type, default, valid range, the section that governs their use, and a one-line purpose summary.

23.1 Parameter Table

Parameter	Type	Default	Range	Section	Purpose
MEMTYPE	string	"DDR2"	{"DDR2", "LPDDR2"}	§3.6	Picks encoder + step table at build time
N_PHASES	integer	4	{1, 2, 4}	§3.6	DFI frequency-ratio gear
AXI_DATA_WIDTH	integer	64	{64, 128, 256}	§3.1	AXI4 data bus width
AXI_ID_WIDTH	integer	4	1–16	§3.	AXI ID tag width

Parameter	Type	Default	Range	Section	Purpose
AXI_ADDR_WIDTH	integer	32	24–40	§ 3.1	AXI flat address width
AXI_000_ACROSS_IDS	boolean	true	{true, false}	§ 3.1	Allow out-of-order completion across AXI IDs
SCHEDULE_R_MODE	enum	000	{000, INORDER}	§ 3.2	000 = synthesize the full FR-FCFS scheduling logic (default; debug-friendly — INORDER mode is then available via runtime CSR). INORDER = omit FR-FCFS at elaboration; smaller area but permanent in-order operation. Runtime override: SCHED_TUNING.force_inorder forces FIFO at runtime without rebuild when 000 is synthesized.
SUPPORT_FIXED_BURST	boolean	false	{true, false}	§ 3.1	AXI4 FIXED burst type support
SUPPORT_WRAP_BURST	boolean	false	{true, false}	§ 3.1	AXI4 WRAP burst type support
NUM_RANKS	integer	1	{1, 2, 4}	§ 3.	Number of physical ranks (per-rank CS_n, CKE, ODT). 1

Parameter	Type	Default	Range	Section	Purpose
	t			3	= single-rank point-to-point; 2 / 4 = DIMM-class. Rank bits are placed in the address map (see §3.1) and refresh state is tracked per (rank, bank).
NUM_BANKS	integer	8	{4, 8}	§ 3.3	Per-device bank count (per rank)
ODT_RULE_MULTIRANK	enum	JEDEC_DDR2	{JEDEC_DDR2, JEDEC_LPDDR2, OFF}	§ 3.6	Multi-rank ODT termination policy. JEDEC_DDR2 = read→ODT-high on the non-accessed rank during the read window, write→ODT-high on the accessed rank during the write window. JEDEC_LPDDR2 = LPDDR2's simpler always-off-during-read rule. OFF = no ODT driving (point-to-point only; valid only for NUM_RANKS=1). Forced to OFF when NUM_RANKS=1.
ROW_WIDTH	integer	14	12–16	§ 3.3	Row-address width
COL_WIDTH	integer	10	9–12	§ 3.3	Column-address width
DFI_DATA_WIDTH	integer	32	16–128	§ 3.	Per-phase DFI data width

Parameter	Type	Default	Range	Section	Purpose
	t			6	
DFI_ADDR_WIDTH	integer	14 (DDR2) / 20 (LPDDR2)	≥ 14 (DDR2), ≥ 20 (LPDDR2)	§ 3.6	DFI address bus width
WRPHASE	integer	0	0 .. N_PHASES - 1	§ 3.6	Which DFI phase carries write data
RDPHASE	integer	0	0 .. N_PHASES - 1	§ 3.6	Which DFI phase carries read data
TXN_QUEUE_DEPTH	integer	16	4–64	§ 3.2	Pending-transaction queue depth
AGE_MAX	integer	256	32–1024	§ 3.2	FR-FCFS anti-starvation cap
LOOKAHEAD_DEPTH_MAX	integer	4	{0, 1, 2, 4}	§ 3.2	Synthesized MAX number of same-bank lookahead comparators per bank. 0 omits lookahead at elaboration. Runtime active depth: SCHED_TUNING.lookahead_active picks how many of the synthesized comparators are gated on (0 .. MAX) without rebuild. Reset value of lookahead_active matches the build's recommended default (typically 1).

Parameter	Type	Default	Range	Section	Purpose
PAGE_POLICY	enum	HAPPY_HYBRID (LPDDR2) / OPEN (DDR2)	{OPEN, CLOSE, HAPPY_HYBRID}	§ 3.2	Fallback policy when lookahead is inconclusive
PAGE_PREDICTOR_TABLE_BITS	int	12	8–16	§ 3.2	HAPPY predictor table size (synthesized only when PAGE_POLICY is HAPPY_HYBRID). Runtime kill-switch: SCHED_TUNING.happy_enable disables predictor lookup without rebuild — useful for A/B'ing HAPPY's contribution to perf.
ADDR_MAP_SCHEMES_SYNTH	set	{ROW_MAJOR}	subset of {ROW_MAJOR, BANK_INTERLEAVE, XOR_HASH}	§ 3.1	Schemes synthesized for runtime mux. Including more schemes adds a small amount of address-decode logic. Runtime mux: ADDR_MAP_TUNING.scheme_or picks the active scheme among the synthesized set.
ADDR_MAP_SCHEME_DEFAULT	enum	ROW_MAJOR	one of ADDR_MAP_SCHEMES_SYNTH	§ 3.1	Reset-state choice among the synthesized schemes (used when CSR override is 00).
REFPB_POLICY	enum	DARP	{ROUND_ROBIN, OLDEST_FIRST, DARP}	§ 3.4	Per-bank refresh scheduling policy
REFRESH_DEFER_MAX	int	8	1–8	§	Synthesized MAX refresh-

Parameter	Type	Default	Range	Section	Purpose
X	uint			3.4	postponement count (counter width is $\text{clog}_2(\text{MAX})$). Runtime active: <code>REFRESH_TUNING.refresh_defer_active</code> picks any value 1..MAX without rebuild. 1 = no batching (every tREFI fires one refresh).
SIM_INIT_SCALE	uint	1	1–10000	§3.5	Init-delay divider for sim runs (silicon always uses 1)

Runtime-only knobs (no build-time presence — purely register-file values):

- `REFRESH_TUNING.zqcs_freq_hz` (16-bit) — Periodic ZQCS interval in Hz. 0 disables periodic ZQCS (init-only). Reset = 1 Hz.
- `INIT_TUNING.zq_retries` (4-bit) — ZQ calibration retry count before raising `init_error`. Reset = 3.
- `INIT_TUNING.init_timeout_ms` (8-bit) — Per-step init timeout. Reset = 10.

These cost zero silicon area beyond the register flops; making them build-time parameters would have been wrong (and was, in earlier revisions of this document).

23.2 Memtype-Dependent Constraints

Some parameters have memtype-dependent constraints checked at elaboration:

- `MEMTYPE == "LPDDR2" ⇒ DFI_ADDR_WIDTH ≥ 20` (CA bus packing requirement)
- `MEMTYPE == "DDR2" ⇒ DFI_ADDR_WIDTH ≥ ROW_WIDTH`
- `PAGE_POLICY == HAPPY_HYBRID ⇒ PAGE_PREDICTOR_TABLE_BITS` must be in range; otherwise the predictor module is not instantiated and the parameter is ignored

23.3 Timing-Configuration Sanity Checks

The controller performs the following elaboration-time assertions on the loaded timing CSR values:

- $t_{\text{REFI_cycles}} \geq 100$ — the MC clock must be fast enough relative to t_{REFI} that the refresh timer is well-resolved. If not, elaboration fails with a clear error.
- $t_{\text{RCD_cycles}} \geq 2$, $t_{\text{RP_cycles}} \geq 2$, $t_{\text{RC_cycles}} \geq t_{\text{RAS}} + t_{\text{RP}}$ — basic JEDEC consistency checks.
- $t_{\text{RFC_cycles}} > t_{\text{RP_cycles}}$ — the refresh-cycle time must exceed the precharge time.

These are not preventable run-time errors — they’re configuration bugs that should be caught at synthesis or boot time before silicon ships.

23.4 Parameter Categories

Category	Parameters
Memtype	MEMTYPE
Geometry	NUM_RANKS, NUM_BANKS, ROW_WIDTH, COL_WIDTH
Multi-rank	NUM_RANKS, ODT_RULE_MULTIRANK
Bus widths	AXI_DATA_WIDTH, AXI_ID_WIDTH, AXI_ADDR_WIDTH, DFI_DATA_WIDTH, DFI_ADDR_WIDTH
Gear	N_PHASES, WRPHASE, RDPHASE
AXI features	AXI_000_ACROSS_IDS, SUPPORT_FIXED_BURST, SUPPORT_WRAP_BURST
Capacity	TXN_QUEUE_DEPTH, AGE_MAX
Characterization	LOOKAHEAD_DEPTH_MAX, PAGE_POLICY, PAGE_PREDICTOR_TABLE_BITS, ADDR_MAP_SCHEMES_SYNTH, REFPB_POLICY, REFRESH_DEFER_MAX (all paired with runtime CSR overrides — see §6.3)
Init	SIM_INIT_SCALE

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

24 Characterization Knobs

A subset of the build-time parameters is intentionally left configurable for **characterization sweeping**. These are the algorithmic choices that have research-backed alternatives where the best choice depends on workload. Rather than baking in a one-size-fits-all default, we expose them as parameters so the verification plan can sweep them on representative AXI traffic and pick defaults from data.

24.1 Characterization Parameters

Parameter	Choices	Recommended default	Rationale
LOOKAHEAD_DEPTH	0 / 1 / 2 / 4	1	Same-bank lookahead window for exact auto-precharge decisions when the next same-bank request is already in the queue (typical for streaming / DMA). Falls back to PAGE_POLICY when the queue is shallow. 0 disables; 2 or 4 buys diminishing returns at modest comparator cost.
PAGE_POLICY	OPEN / CLOSE / HAPPY_HYBRID	HAPPY_HYBRID (LPDDR2), OPEN (DDR2)	Fallback for when lookahead is inconclusive. Streaming workloads favor OPEN; mobile-mixed favors HAPPY_HYBRID. Ghasempour 2015 reports near-optimal accuracy with small storage.
PAGE_PREDICTOR_TABLE_SIZE	8 – 16	12	4 KB predictor is sufficient for typical mobile workloads. Larger tables marginal improvement.
REFPB_POLICY	ROUND_ROBIN / OLDEST_FIRST / DARP	DARP	Chang HPCA 2014: DARP wins over round-robin on uneven bank activity (real workloads).
REFRESH_DEFER_MAX	1 – 8	8 (JEDEC ceiling)	Stuecheli MICRO 2010: deferring up to 8 maximizes Elastic Refresh benefit. Set to 1 for real-time

Parameter	Choices	Recommended default	Rationale
TXN_QUEUE_DEPTH	4 – 64	16	predictability. Larger queues help bandwidth on multi-master AXI; smaller queues reduce area.
AGE_MAX	32 – 1024	256	Caps FR-FCFS starvation; larger values let row-hits dominate longer at the cost of tail latency.
N_PHASES	1 / 2 / 4	depends on PHY	4 typical for FPGA SoCs; 2 for slower mobile platforms; 1 for simulation-only.
ADDR_MAP_SCHEME	ROW_MAJOR / BANK_INTERLEAVE / XOR_HASH	ROW_MAJOR OR	Default works for streaming; BANK_INTERLEAVE for random; XOR_HASH for adversarial.

24.2 Sweep Plan

The verification plan (see §6.4) includes a characterization sweep that runs each parameter through its choices on a small benchmark suite:

24.2.1 Benchmark Suite

A key open question for the sweep is the **interaction between LOOKAHEAD_DEPTH and PAGE_POLICY**. With deep queues and conclusive lookahead, the choice of PAGE_POLICY barely matters because the fallback rarely fires. With shallow queues, PAGE_POLICY dominates. The sweep matrix should cover both extremes.

Workload	Pattern
stream-read	Sequential 4 KB read bursts
stream-write	Sequential 4 KB write bursts
stream-mixed	Interleaved sequential read and write
random-narrow	Random small AXI bursts (1–4 beats)
random-wide	Random large AXI bursts (16–64 beats)
mobile-	Bursty traffic with periodic large writes (modeled on

Workload	Pattern
mixed	smartphone GPU + display)
multi-master	Two AXI masters issuing concurrent traffic with different patterns

24.2.2 Metrics Collected

For each (workload, parameter) combination, the sweep records:

- **Sustained read bandwidth** (% peak)
- **Sustained write bandwidth** (% peak)
- **Average read latency**
- **99th-percentile read latency**
- **Refresh blocking time** (% of cycles where the bus was blocked by refresh)
- **Row-hit rate** (per-bank, averaged)
- **Refresh deferral utilization** (avg refresh_pending over time)

24.2.3 Default Selection

After the sweep, each parameter's default is set to the value that:

1. Maximizes sustained bandwidth across the benchmark suite
2. Subject to: 99th-percentile latency does not regress by more than 20%
3. Subject to: area cost is within budget

The defaults shown in the parameter table above are the **expected** outcomes based on the cited research. The actual defaults will be confirmed by the sweep.

24.3 Runtime Overrides via CSR

Some characterization parameters are also accessible at runtime via CSR (see §6.3). This allows in-system tuning without rebuild:

- PAGE_POLICY runtime override
- REFPB_POLICY runtime override
- Page predictor warm-up cycles and hysteresis

Runtime overrides take effect at the next “quiet point” (no commands in flight). The SoC is responsible for ensuring no DRAM traffic during the transition.

24.4 Observability for Sweep

The CSR exposes the following observation counters (described in §6.3):

- Per-bank row-hit count
- Average refresh latency
- Max transaction queue depth
- Page-predictor accuracy (HAPPY mode)
- Refresh deferral histogram

These feed back into the sweep tool for in-loop default selection.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

25 Family-Wide Config Bits

This section is the **family-wide config-bit registry** — a catalog of runtime configuration bits used across the DDR2 / LPDDR2, DDR3 / LPDDR3, and DDR4 / LPDDR4 memory controllers. Some bits apply to all generations; some are introduced in one generation and absent in earlier ones; some are *defined* across the family but ignored in flavors where the underlying mechanism does not exist. **That last case is intentional.** Allocating the bit family-wide — even when unused — keeps the CSR map mechanically compatible across controllers and lets software treat the family as a single product line.

25.1 Why a Family-Wide Registry

Each controller in the family has its own CSR map (per §6.3 of each generation's HAS). But the field encodings, bit positions within shared registers, and the *names* are aligned across the family. The intent:

- **Software portability:** a single device driver template understands the bit layout and only needs per-flavor capability checks.
- **Bring-up portability:** bring-up software written for DDR3 should mostly work on DDR2 with the same register reads.
- **Verification portability:** the cocotb test suite's CSR-poke helpers can be reused.

- **Debug consistency:** the same “config dump” script produces a parseable output regardless of which flavor is running.

A bit may be marked N/A in a flavor; reads return 0 and writes are ignored (no `pslverr`, just silently ignored — because software portability requires that writing a fully-portable config word does not error on flavors that don’t implement every bit).

25.2 Applicability Notation

The Apply column uses this shorthand:

- **all** — applies to every flavor in the family
- **DDR-only** — applies to DDR2/3/4 only; LPDDR variants ignore
- **LP-only** — applies to LPDDR2/3/4 only; DDR variants ignore
- **DDR3+** — applies to DDR3, DDR4, and forward; DDR2 ignores
- **DDR4+** — applies to DDR4 only (introduced for bank groups); earlier ignore
- **LPDDR3+** — LPDDR3 and LPDDR4 only
- **LPDDR4-only** — LPDDR4 only

25.3 SCHED_TUNING (Scheduling Family Bits)

Register block at the same offset (0x040) in every controller. These bits govern the FR-FCFS scheduler’s runtime behavior.

Bit Field	Width	Apply	Description
lookahead_active	4	all	Runtime lookahead window (0 .. build-time MAX). 0 disables lookahead.
force_inorder	1	all	Forces FR-FCFS into first-ready FIFO mode for debug / real-time.
happy_enable	1	all	Runtime kill-switch for the HAPPY page predictor. Reads as 0 in flavors built without HAPPY.
age_max_runtime	8	all	Anti-starvation age cap; runtime override of AGE_MAX.
txn_queue_high_water	8	all	Backpressure threshold; AXI awready deasserts when queue $\geq$ threshold.
lookahead_max_obs	4	all (R)	Echo of build-time LOOKAHEAD_DEPTH_MAX.

Bit Field	Width	Apply	Description
		only)	Software discovery.
qos_high_priority	1	all	When 1, scheduler boosts requests with $awqos/arqos \geq 8$.
bank_group_balance	1	DDR4+	Round-robin across bank groups in scheduler tie-break (DDR4/LPDDR4 only); N/A elsewhere.

25.4 REFRESH_TUNING (Refresh Family Bits)

Bit Field	Width	Apply	Description
refpb_policy_or	2	LP-only	REFpb selection-policy override (LPDDR2/3/4); N/A for DDR-only flavors
page_policy_or	2	all	Runtime override for fallback page policy
refresh_defer_active	4	all	Active deferral count (1 .. build-time MAX). 1 = no batching.
zqcs_freq_hz	16	all	Periodic ZQCS interval. 0 = init-only.
fgr_mode	2	DDR3+	Fine-Granularity Refresh mode (DDR3 1x/2x/4x; DDR4 fixed-rate, on-the-fly). N/A in DDR2.
refresh_per_rank	1	all	Force per-rank dispatch even when NUM_RANKS=1 (debug). Reset = 1 always when multi-rank.

25.5 ADDR_MAP_TUNING (Address-Mapping Family Bits)

Bit Field	Width	Apply	Description
scheme_or	2	all	Runtime scheme mux (ROW_MAJOR, BANK_INTERLEAVE, XOR_HASH)
synth_mask_obs	4	all (R only)	Bitmask of synthesized schemes. Software discovery.
bg_field_position	3	DDR4+	Bit position of bank-group field in flat address. N/A pre-DDR4.
xor_seed_runtime	8	all	Runtime seed for the XOR_HASH scheme

Bit Field	Width	Apply	Description
			(changes hash without rebuild).

25.6 INIT_TUNING (Init Family Bits)

Bit Field	Width	Apply	Description
zq_retries	4	all	ZQ calibration retry count (DDR2 uses OCD; LPDDR2 uses MR10 ZQ).
init_timeout_ms	8	all	Per-step init timeout (ms; scaled by SIM_INIT_SCALE in sim).
wl_retries	4	DDR3+	Write-leveling retry count. N/A in DDR2 (no write-leveling).
mpr_enable	1	DDR3+	Multi-Purpose Register read-back during init. N/A pre-DDR3.
ca_train_enable	1	LPDDR3+	LPDDR3/4 CA-bus training. N/A in LPDDR2 (no CA training).
cbt_enable	1	LPDDR4-only	Command-Bus-Training. N/A pre-LPDDR4.

25.7 POWER_TUNING (Power-State Family Bits)

Bit Field	Width	Apply	Description
apd_idle_threshold	16	all	Cycles of bank-idleness before APD entry
srf_idle_threshold	24	all	Cycles before Self-Refresh entry
dpd_enable	1	LP-only	Deep-Power-Down enable. N/A for DDR-only flavors.
pasr_active	1	LP-only (R)	Reflects whether any rank has a non-zero PASR mask. R-only.
ckedis_after_sr	1	DDR3+	Drop CKE after SR-entry for sub-mW power floor. N/A in DDR2.
low_freq_mode	1	DDR3+	Reduced-frequency operation hint to PHY. N/A in DDR2.

25.8 ECC_TUNING (Future — Inline ECC)

This block is reserved for future inline-ECC controllers. The bits are **N/A in all current controllers** (DDR2/3/4 and LPDDR2/3/4 v1) because inline ECC is out of scope for the current generation. Future revisions of any flavor that adopt inline ECC will populate this block.

Bit Field	Width	Apply	Description
<code>ecc_enable</code>	1	(reserved)	Enable inline ECC. Reads as 0 in all v1 controllers.
<code>ecc_correct_mode</code>	2	(reserved)	SECEDED / Chipkill / off

25.9 RANK_TUNING (Multi-Rank Family Bits)

These are present in any flavor with `NUM_RANKS > 1`. For `NUM_RANKS = 1` builds, the bits read as 0 and writes are silently ignored.

Bit Field	Width	Apply	Description
<code>rank_enable_mask</code>	4	all	Per-rank enable; clearing a bit suppresses all commands to that rank. Up to 4 ranks.
<code>odt_rule_or</code>	2	all	Runtime override for <code>ODT_RULE_MULTIRANK</code> (00 = build-time default; 01 = JEDEC_DDR2; 10 = JEDEC_LPDDR2; 11 = OFF).
<code>num_ranks_obs</code>	3	all (Read only)	Echo of build-time <code>NUM_RANKS</code> . Software discovery.
<code>cs_assert_cycles</code>	4	all	Number of cycles to hold <code>CS_n</code> asserted per command; tunable for PHY timing margin.

25.10 Quiet-Point Behavior

All runtime config bits take effect at the **next configuration quiet point** — no DRAM commands in flight, no refresh sequence in progress. The CSR slave does not enforce quiet points automatically; software requests one by writing `CTRL.config_apply = 1`, which triggers a controlled drain. Reads of `STATUS.config_settled` indicate when the override propagation has completed.

The quiet-point requirement is family-wide because every flavor has hand-off state between scheduler / refresh / power-state that would be corrupted by mid-flight reconfiguration.

25.11 Bit Discovery via the Capability Vector

At 0xFF8 of each controller's CSR map, a **capability vector** is exposed:

Bits	Field	Description
0	cap_happy	1 = HAPPY predictor synthesized
1	cap_bg_balance	1 = bank-group balance scheduler logic present (DDR4+)
2	cap_fgr	1 = Fine-Granularity Refresh supported (DDR3+)
3	cap_pasr	1 = PASR mask is implemented (LP-only)
4	cap_dpd	1 = Deep-Power-Down supported (LP-only)
5	cap_wl	1 = Write-leveling implemented (DDR3+)
6	cap_cbt	1 = CBT implemented (LPDDR4-only)
7	cap_xor_hash	1 = XOR_HASH address scheme synthesized
11:8	cap_max_ranks	Build-time NUM_RANKS (echo of geometry)
15:12	cap_n_phases	Build-time N_PHASES
19:16	cap_memtype	0 = DDR2, 1 = DDR3, 2 = DDR4, 4 = LPDDR2, 5 = LPDDR3, 6 = LPDDR4

Software queries this once at boot to decide which family-wide bits are meaningful on the current flavor.

25.12 Per-Flavor Applicability Matrix

Bit	DDR2	DDR3	DDR4	LPDDR2	LPDDR3	LPDDR4
lookahead_active	✓	✓	✓	✓	✓	✓
force_inorder	✓	✓	✓	✓	✓	✓
happy_enable	✓	✓	✓	✓	✓	✓
bank_group_balance	—	—	✓	—	—	✓
refpb_policy_or	—	—	—	✓	✓	✓
fgr_mode	—	✓	✓	—	—	—
bg_field_position	—	—	✓	—	—	✓
wl_retries	—	✓	✓	—	—	—
ca_train_enable	—	—	—	—	✓	✓

Bit	DDR2	DDR3	DDR4	LPDDR2	LPDDR3	LPDDR4
cbt_enable	—	—	—	—	—	✓
dpd_enable	—	—	—	✓	✓	✓
ckedis_after_sr	—	✓	✓	—	—	—
rank_enable_mask	✓	✓	✓	✓	✓	✓
odt_rule_or	✓	✓	✓	✓	✓	✓
xor_seed_runtime	✓	✓	✓	✓	✓	✓

The “—” entries are present in the CSR map at the documented offsets but read 0 and ignore writes in that flavor. Software treats these as “soft-N/A” — not an error condition, just an absent feature.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

26 Clocks and Reset

26.1 Clock Domains

The controller uses three independent clocks.

26.1.1 mc_clk

Primary controller clock. All scheduler, bank machines, command pipeline, and AXI4-side logic run on mc_clk. The DFI master output stage launches its signals on mc_clk.

Typical frequency: 100 – 200 MHz on FPGA SoCs; up to 533 MHz on silicon for DDR2-1066 / LPDDR2-800.

26.1.2 phy_clk

PHY-side clock. Runs at $N_PHASES \times mc_clk$ rate. The PHY consumes DFI signals on phy_clk.

For $N_PHASES = 4$, $phy_clk = 4 \times mc_clk$. For $N_PHASES = 2$, $phy_clk = 2 \times mc_clk$. For $N_PHASES = 1$, $phy_clk = mc_clk$ (they may be the same signal in this case).

The relationship is asynchronous from the controller’s perspective — the PHY handles the rate conversion. The DFI v2.1 specification’s frequency-ratio scheme is what makes this work without explicit FIFO crossings.

26.1.3 apb_pclk

APB CSR bus clock. Independent of mc_clk — typically a slower SoC management clock. CDC inside csr_slave transfers configuration writes into the mc_clk domain via 2-flop synchronizer plus handshake.

26.2 Reset Signals

26.2.1 mc_rstn (active low)

Primary controller reset. Synchronously deasserted on mc_clk. Resets:

- AXI4 slave state machines
- Transaction queue
- Scheduler and bank machines
- Cross-bank timers
- Refresh manager (clears tREFI counter and refresh_pending)
- Power-state FSM (transitions to RESET state)
- Init engine (resets to step 0)
- All observation counters

26.2.2 phy_rstn (active low)

Controlled by init_engine during the reset-release phase of the init sequence. Asserted at startup; deasserted as the first step of the init step table.

26.2.3 dfi_rstn (active low, MC → PHY)

DFI reset signal driven to the PHY. Controlled by init_engine per the init step table; allows the PHY to perform its own reset sequence during DRAM init.

26.2.4 apb_presetn (active low)

APB CSR bus reset. Synchronously deasserted on apb_pclk. Resets the CSR slave state machine.

26.3 Reset Sequencing

26.3.1 Cold Boot

1. All resets asserted (mc_rstn, phy_rstn, dfi_rstn, apb_presetn)
2. Clocks stable
3. apb_presetn deasserted (SoC programs CSR if needed; otherwise defaults)
4. mc_rstn deasserted

5. Init engine starts; runs step table; drives phy_rstn and dfi_rstn per the sequence
6. On init_done, power-state FSM transitions to ACTIVE
7. Controller services AXI traffic

26.3.2 Warm Reset (Clock-Gated)

The SoC can warm-reset the controller without losing DRAM content:

1. SoC requests self-refresh entry via CSR
2. Controller acknowledges; DRAM is in self-refresh
3. SoC asserts mc_rstn; clocks may be gated
4. SoC deasserts mc_rstn; clocks resume
5. Controller re-runs init from the table (the table is idempotent — it does not corrupt DRAM in self-refresh)
6. Controller resumes normal operation

The init step table is designed to be cold-boot-correct **and** warm-boot-safe. The step list for LPDDR2 includes a check on MR4 for the current temperature; if the temperature changed during the warm reset, the controller updates its tREFI scale accordingly.

26.4 Clock Domain Crossing Summary

Crossing	Mechanism	Latency
apb_pclk → mc_clk	2-flop sync + handshake	~3-4 mc_clk cycles
mc_clk → apb_pclk	2-flop sync (observation reads)	~3-4 apb_pclk cycles
mc_clk → phy_clk	DFI frequency-ratio scheme (no explicit CDC)	0 (combinational replication)

26.5 Reset and Power Considerations

Note that DFI v2.1 §4.1 requires dfi_init_start to be asserted by the controller as part of init; the controller does this during the reset-release step. The PHY's dfi_init_complete signal indicates that the PHY has finished its own initialization; the controller's init engine waits on this before proceeding to MRS loads.

27 Init Sequences

The full cold-boot init sequences for DDR2 and LPDDR2. These are the step lists embedded in `ddr2_init_steps_pkg.sv` and `lpddr2_init_steps_pkg.sv`.

27.1 DDR2 Cold Init

Per JESD79-2F §4.1 and Micron TN-47-09.

Step	Type	Description
1	SET_CTRL_BITS	Power stable; CKE = 0 for $\geq 200 \mu\text{s}$
2	DELAY	NOP for $\geq 400 \text{ ns}$
3	SET_CTRL_BITS	CKE $\rightarrow 1$, ODT = 0, RESET_N = 1
4	ISSUE_CMD	PRECHARGE all banks (PREA with <code>addr[10] = 1</code>)
5	MRS_LOAD	EMRS(2) = 0 (extended mode register 2 defaults)
6	MRS_LOAD	EMRS(3) = 0
7	MRS_LOAD	EMRS(1) = enable DLL
8	MRS_LOAD	MRS = reset DLL + initial CL / BL
9	ISSUE_CMD	PRECHARGE all
10	ISSUE_CMD	AUTO REFRESH (REF)
11	ISSUE_CMD	AUTO REFRESH (REF)
12	MRS_LOAD	MRS = final CL / BL / WR config (no DLL reset)
13	MRS_LOAD	EMRS(1) = OCD default
14	MRS_LOAD	EMRS(1) = OCD exit
15	END	Assert <code>init_done</code>

Real-time delays embedded in `post_delay`: step 1 = $200 \mu\text{s}$; step 2 = 400 ns ; step 3 = 200 ns NOPs; intermediate MRS steps = $t_{MRD} (\geq 2 \text{ clocks})$.

27.2 LPDDR2 Cold Init

Per JESD209-2F §6.1 and Micron TN-46-22.

Step	Type	Description
1	SET_CTRL_BITS	tINIT1: CKE low ≥ 100 ns
2	SET_CTRL_BITS	tINIT2: CKE high ≥ 200 μ s
3	DELAY	tINIT3: NOP ≥ 11 μ s
4	MRS_LOAD	MR63 reset
5	DELAY	tINIT5: ≥ 10 μ s
6	MRS_LOAD	MR10 ZQ calibration
7	WAIT_FOR_BIT	Wait dfi_init_complete (ZQ done)
8	MRS_LOAD	MR1: BL, WRAP, BT
9	MRS_LOAD	MR2: RL, WL
10	MRS_LOAD	MR3: drive strength
11	MRS_LOAD	MR4 read for temperature class (programs internal tREFI scale)
12	MRS_LOAD	MR16: PASR bank mask (default 0 = all banks refresh)
13	MRS_LOAD	MR17: PASR segment mask (default 0)
14	END	Assert init_done

Real-time delays embedded in post_delay: step 1 = 100 ns; step 2 = 200 μ s; step 3 = 11 μ s; step 5 = 10 μ s; step 7 timeout = 1 ms; subsequent MRS = tMRD.

27.3 Self-Refresh Entry (Both Memtypes)

Used during power-state FSM transitions:

Step	Type	Description
1	WAIT_FOR_BIT	Wait for all banks idle and refresh complete
2	ISSUE_CMD	PRECHARGE all
3	SET_CTRL_BITS	CKE low + ODT off (initiates self-refresh)
4	END	FSM transitions to SELF_REFRESH state

27.4 Self-Refresh Exit (Both Memtypes)

Step	Type	Description
1	SET_CTRL_BITS	CKE high (initiates self-refresh exit)
2	DELAY	tXSNR (~200 ns)
3	END	FSM transitions to ACTIVE

27.5 LPDDR2 Deep Power Down Entry

Step	Type	Description
1	WAIT_FOR_BIT	Wait for all banks idle and refresh complete
2	ISSUE_CMD	PRECHARGE all
3	ISSUE_CMD	DPD command (special LPDDR2 encoding)
4	SET_CTRL_BITS	CKE low
5	END	FSM transitions to DPD state

27.6 LPDDR2 Deep Power Down Exit

DPD exit loses DRAM content; the full cold-init sequence (LPDDR2 init above) must run again. The DPD exit step table is effectively just:

Step	Type	Description
1	JUMP_TO_TABLE	Restart the LPDDR2 cold init step table from step 1

This is implemented by having the power-state FSM signal `init_engine` to restart, rather than a separate step table.

27.7 Sim Scaling

All `post_delay` fields are scaled by `SIM_INIT_SCALE` during simulation. For example, a 200 μ s delay (about 20,000 cycles at 100 MHz) becomes 20 cycles at `SIM_INIT_SCALE = 1000`. This keeps test runtimes practical without changing the step table logic. Silicon builds always use `SIM_INIT_SCALE = 1`.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

28 CSR Register Map

The APB CSR slave exposes a 4 KB address space (12-bit paddr). All registers are 32 bits wide.

28.1 Control and Status (0x000 – 0x00F)

28.1.1 CTRL (0x000, R/W)

Bit	Field	Reset	Description
0	init_start	0	Write 1 to start init
1	init_force_restart	0	Write 1 to force re-init even mid-sequence
4	pwr_req_low_power	0	Request power-down state
5	pwr_req_dpd	0	Request DPD (LPDDR2 only)
6	pwr_req_active	0	Request return to ACTIVE
7	pwr_req_self_refresh	0	Request self-refresh
31	soft_reset	0	Write 1 to assert internal soft reset

28.1.2 STATUS (0x004, R only)

Bit	Field	Description
0	init_done	1 when init complete
1	init_error	1 on init failure
7:4	power_state	Current power-state FSM state (encoded)
8	pasr_active	LPDDR2: PASR mask is non-zero
23:16	init_step_dbg	Current init step number (for bring-up)
31	version_match	1 when build matches expected version

28.1.3 STATUS_HISTORY (0x008, R only)

Last 8 power-state transitions, 4 bits each. New transitions push the oldest off. Useful for bring-up debugging of power-state oscillation.

28.2 Timing Parameters (0x010 – 0x01F)

Packed timing parameters. Each register holds three to four 8-bit timing values.

28.2.1 TIMINGS_RC_RCD_RP (0x010, R/W)

Bits	Field	Description
7:0	tRC	tRC in cycles
15:8	tRCD	tRCD in cycles
23:16	tRP	tRP in cycles
31:24	tRAS	tRAS in cycles

28.2.2 TIMINGS_RAS_RFC_REFI (0x014, R/W)

Bits	Field	Description
15:0	tRFC	tRFC (or tRFCab) in cycles
31:16	tREFI	tREFI in cycles

28.2.3 TIMINGS_RRD_FAW_WTR (0x018, R/W)

Bits	Field	Description
7:0	tRRD	tRRD
15:8	tFAW	tFAW
23:16	tWTR	tWTR
31:24	tCCD	tCCD

28.2.4 TIMINGS_CL_CWL_WR (0x01C, R/W)

Bits	Field	Description
7:0	CL	CAS latency
15:8	CWL	CAS write latency
23:16	tWR	Write recovery
31:24	tRFCpb	LPDDR2 per-bank tRFC

28.3 Mode Register Values (0x020 – 0x02F)

28.3.1 MR0, MR1, MR2, MR3 (0x020, 0x024, 0x028, 0x02C, R/W)

Low 16 bits contain the MR value loaded during init. Upper 16 bits reserved.

28.4 LPDDR2-Specific (0x030 – 0x03F)

28.4.1 PASR_BANK_MASK_RANK<R> (0x030 + R×4, R/W; R = 0 .. NUM_RANKS–1)

Bits	Field	Description
7:0	pasr_banks	Per-rank bank mask (MR16); bit N = 1 means bank N is masked off on this rank

One register per rank. For NUM_RANKS=1 only PASR_BANK_MASK_RANK0 (0x030) exists; higher ranks return ps1verr.

28.4.2 PASR_SEG_MASK_RANK<R> (0x080 + R×4 within the multi-rank reserved area is reused — see register layout note)

For implementation simplicity, segment masks share the rank-indexed pattern: PASR_SEG_MASK_RANK0 lives at 0x034, and additional ranks consume successive 32-bit slots in the same window. Implementations that ship with NUM_RANKS > 1 may instead place the additional masks in the per-rank observation window (0x0C0+, see below) — the CSR map vector exposed at 0xFF8 provides discovery.

28.4.3 TEMP_DERATE_RANK<R> (0x038 + R×4, R only)

Bits	Field	Description
1:0	temp_class	From this rank's MR4: 00 = nominal, 01 = 2x refresh, 10 = 4x refresh

One register per rank — LPDDR2 ranks can run at different temperatures (e.g., one DRAM near a hot SoC die, another further away), so refresh derating is computed per-rank.

28.5 Scheduler / Refresh / Page Tuning (0x040 – 0x04F)

28.5.1 SCHED_TUNING (0x040, R/W)

Bits	Field	Description
3:0	lookahead_active	Active lookahead window (0..LOOKAHEAD_DEPTH_MAX). 0 disables lookahead at runtime, forcing the page-policy fallback. Reset value matches build-time default.
4	force_inorder	0 = OoO FR-FCFS scheduling (default). 1 = force first-ready FIFO at runtime; row-hit reordering disabled. For debug, formal verification, or real-time-mode

Bits	Field	Description
5	happy_enable	runs. Takes effect at next quiet point. 0 = HAPPY predictor disabled at runtime (lookup returns “weakly hit-likely” regardless of training). 1 = predictor active. Reset = 1 if PAGE_POLICY ∈ HAPPY_HYBRID was synthesized; else 0. Useful for A/B’ing predictor contribution without rebuild.
15:8	age_max_runtime	Runtime override for AGE_MAX (clipped to ≤ build-time AGE_MAX). 0 = use build-time default.
23:16	txn_queue_high_water	Backpressure-assertion threshold (≤ build-time TXN_QUEUE_DEPTH).
27:24	lookahead_max_obs	(R only) Echo of build-time LOOKAHEAD_DEPTH_MAX. Lets software discover the synthesized ceiling.

28.5.2 PAGE_PRED_TUNING (0x044, R/W, only when HAPPY enabled)

Bits	Field	Description
15:0	warmup_cycles	Cycles before predictor is trusted
23:16	hysteresis	Saturating-counter hysteresis

28.5.3 REFRESH_TUNING (0x048, R/W)

Bits	Field	Description
1:0	refpb_policy_or	Runtime override: 00 = use build-time; 01 = RR; 10 = OLDEST_FIRST; 11 = DARP. Takes effect at next quiet point.
3:2	page_policy_or	Runtime override for PAGE_POLICY: 00 = build-time default; 01 = force OPEN; 10 = force CLOSE; 11 = force HAPPY_HYBRID (only valid when HAPPY was synthesized). Takes effect at next quiet point.
7:4	refresh_defer_active	Active refresh deferral count (1..REFRESH_DEFER_MAX). 1 = no batching. Larger values batch deferrals up to the synthesized MAX.
31:16	zqcs_freq_hz	Periodic ZQCS interval in Hz. 0 disables periodic ZQCS (init-only mode). Reset = 1 Hz.

28.5.4 ADDR_MAP_TUNING (0x04C, R/W)

Bits	Field	Description
1:0	scheme_or	Runtime mux among synthesized schemes: 00 = build-time default; 01 = ROW_MAJOR; 10 = BANK_INTERLEAVE; 11 = XOR_HASH. Only schemes selected for synthesis via ADDR_MAP_SCHEMES_SYNTH are switchable; writing an unsynthesized scheme returns <code>pslverr</code> .
7:4	synth_mask_obs	(R only) Bitmask of schemes synthesized: bit 0 = ROW_MAJOR, bit 1 = BANK_INTERLEAVE, bit 2 = XOR_HASH. Lets software discover the available choices.

28.5.5 INIT_TUNING (0x050, R/W)

Bits	Field	Description
3:0	zq_retries	ZQ calibration retry count before raising <code>init_error</code> . Reset = 3. Range 1–8.
15:8	init_timeout_ms	Per-step init timeout in ms (scaled by <code>SIM_INIT_SCALE</code> in sim). Reset = 10. Range 1–255.

All runtime overrides take effect at the next configuration quiet point (no DRAM commands in flight, no refresh sequence in progress). The CSR slave does not enforce quiet points; the SoC may force a brief drain by strobing `CTRL.config_apply`.

28.6 Per-(Rank, Bank) Observation (0x080 – 0x0FF)

For each (rank R, bank N) pair within `NUM_RANKS` × `NUM_BANKS`, two registers. The address layout interleaves bank-first within rank-block; the per-rank stride is `NUM_BANKS` × 4 (32 bytes for 8-bank parts).

28.6.1 OBS_ROW_HIT_RANK<R>_BANK<N> (0x080 + (R × NUM_BANKS + N) × 4, R only)

Rolling row-hit count, rank R bank N. Resets when read or on `soft_reset`. For `NUM_RANKS`=1 this collapses to the single-rank `OBS_ROW_HIT_BANK<N>` layout at 0x080–0x09C.

28.6.2 OBS_REF_LATENCY_RANK<R>_BANK<N> (0x0C0 + (R × NUM_BANKS + N) × 4, R only)

Average refresh blocking time for rank R bank N. Per-rank tracking matters because per-rank PASR masks, temperature derating, and DARP scheduling can produce different refresh-blocking profiles across ranks even on the same DIMM.

28.7 System Observation (0x100 – 0x1FF)

Offset	Register	Description
0x100	OBS_TXN_QUEUE_DEPTH_MAX	Max queue depth observed
0x104	OBS_TXN_QUEUE_DEPTH_AVG	Time-averaged depth
0x108	OBS_REFRESH_PENDING_MAX	Max refresh_pending value observed
0x10C	OBS_REFRESH_DEFER_HIST_0	Refresh deferral histogram bin 0
0x110	OBS_REFRESH_DEFER_HIST_1	Histogram bin 1
0x114	OBS_REFRESH_DEFER_HIST_2	Histogram bin 2
0x118	OBS_REFRESH_DEFER_HIST_3	Histogram bin 3
0x120	OBS_PAGE_PRED_ACCURACY	HAPPY mode: rolling prediction accuracy (%)
0x130	OBS_AXI_R_LATENCY_AVG	Avg AXI read latency in cycles
0x134	OBS_AXI_R_LATENCY_P99	99th-percentile read latency
0x138	OBS_AXI_W_LATENCY_AVG	Avg AXI write latency

28.8 Module Identification (0xFF0 – 0xFFC)

28.8.1 ID (0xFF0, R only)

Bits	Field	Description
7:0	version	Build version
15:8	memtype	0 = DDR2, 1 = LPDDR2
23:16	n_phases	Gear ratio (1, 2, or 4)
31:24	module_id	0xD2 (fixed)

28.8.2 BUILD (0xFF4, R only)

Build hash / version word; lowered to upper bits at synthesis.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

29 Verification Strategy

A layered verification approach with the existing DFI BFM as the primary reference.

29.1 Layered Approach

29.1.1 Layer 1: Module-Level Unit Tests

Each of the 17 modules gets cocotb unit tests:

Module	Test focus
axi4_slave	Channel handshakes, backpressure, burst splitting, ordering
addr_mapper	Address translation across ADDR_MAP_SCHEME modes
txn_queue	Insert / scan / removal; age saturation; row-hit caching
scheduler	FR-FCFS priority order; page-policy dispatch; refresh prioritization
page_predictor	HAPPY table read/write; accuracy after warm-up
bank_machine	FSM transitions; timing-counter enforcement

Module	Test focus
xbank_timers	tRRD / tFAW / tWTR / tCCD against JESD79-2 / JESD209-2 truth tables
refresh_mgr	tREFI counter; REFab / REFpb selection; DARP idle-bank logic
power_state	All FSM transitions; SR-entry/exit; DPD entry/exit (LPDDR2)
init_engine	Step-by-step execution against canonical init sequences
cmd_encoder	Round-trip table tests against JEDEC encoding
gear_out	1-phase to N-phase replication; idle-phase handling
wdata_path	CWL alignment; ID tagging; mask conversion
rdata_path	CL alignment; OOO completion; back-pressure
dfi_master	Tie-off behavior; aggregation correctness
csr_slave	APB3 protocol; CDC; error responses

29.1.2 Layer 2: Subsystem Tests

Multi-module integration tests:

- **AXI frontend** (axi4_slave + addr_mapper + txn_queue): full AXI transaction flow with diverse traffic
- **Scheduler subsystem** (scheduler + page_predictor + bank_machines + xbank_timers): scheduler decisions on representative queue contents
- **Refresh subsystem** (refresh_mgr + bank_machines): REFab / REFpb sequencing on uneven bank activity
- **Init + power FSM** (init_engine + power_state): full init followed by SR entry/exit cycle

29.1.3 Layer 3: End-to-End with DFI BFM Slave

The full controller drives the DFI BFM slave through our existing co-sim harness (tests/sim/dfi/test_litedram/... patterns in the DV repo).

- AXI4 stimulus generators issue representative traffic
- DFI BFM slave captures the controller's command stream
- Assertions check: command sequencing matches AXI, timing constraints are honored, data integrity round-trips through the slave's MemoryModel

29.1.4 Layer 4: External-Reference Cross-Validation

For DDR2: cross-validate against LiteDRAM’s DDR2 model.

- Generate LiteDRAM controller alongside ours
- Both drive the same DFI BFM slave with the same AXI stimulus
- Compare command streams; differences flag potential design issues

LPDDR2 has no LiteDRAM analog. Cross-validation for LPDDR2 is limited to the BFM master ↔ BFM slave path.

29.2 Characterization Sweep

Post-functional verification, the sweep runs each characterization parameter through its choices on the benchmark suite from §5.2:

29.2.1 Sweep Matrix

Parameter	Sweep values
LOOKAHEAD_DEPTH	0, 1, 2, 4
PAGE_POLICY	OPEN, CLOSE, HAPPY_HYBRID
REFPB_POLICY	ROUND_ROBIN, OLDEST_FIRST, DARP
REFRESH_DEFER_MAX	1, 2, 4, 8
TXN_QUEUE_DEPTH	8, 16, 32
N_PHASES	(build-determined; usually 4)

The full cross-product would be $4 \times 3 \times 3 \times 4 \times 3 = 432$ builds; in practice we sweep one parameter at a time holding others at recommended defaults, which reduces to ~17 builds per memtype. The exception is the LOOKAHEAD_DEPTH × PAGE_POLICY pair — these interact strongly (the fallback policy only matters when lookahead is inconclusive), so this pair is swept as a full $4 \times 3 = 12$ -point grid.

29.2.2 Sweep Outputs

Each (parameter, value) run produces a JSON report:

```
{
  "memtype": "LPDDR2",
  "page_policy": "HAPPY_HYBRID",
  "refpb_policy": "DARP",
  "workload": "mobile-mixed",
  "metrics": {
```

```

    "read_bw_pct": 73.2,
    "write_bw_pct": 68.1,
    "read_latency_avg_ns": 142,
    "read_latency_p99_ns": 198,
    "refresh_block_pct": 4.3,
    "row_hit_rate_avg": 0.81,
    "page_predictor_accuracy": 0.91
  }
}

```

These feed into a sweep visualization tool that selects defaults from data.

29.2.3 Out-of-Order Scheduler Trade-Offs

The centralized FR-FCFS scheduler proposed in §1 (Differentiating Features) is the most architecturally aggressive choice in this controller and warrants explicit characterization beyond just the sweep matrix above. Three concrete concerns:

- **Timing-critical path scales with TXN_QUEUE_DEPTH.** Scanning the full queue every cycle for the best candidate is a comparator pyramid: priority resolves over (ready-flag, row-hit-flag, age) for every entry simultaneously. At TXN_QUEUE_DEPTH = 16 this is well within an embedded-SoC clock budget; at 32 it stays comfortable; at 64 it becomes the critical path on the controller side. The characterization sweep should report scheduler combinational depth at synthesis time alongside the performance numbers — the right TXN_QUEUE_DEPTH is the largest value that doesn't push the scheduler off the clock budget.
- **Area cost is super-linear in queue depth.** Each queue entry needs its full metadata replicated (bank, row, col, ID, age, row-hit cache). At depth 16, ~2K gates; at depth 64, ~10K gates — comparable to the rest of the scheduler combined. The HAPPY predictor's gate cost is a separate line item.
- **Verification cost is significantly higher than FIFO.** OoO issue means assertions like “request N landed at cycle X” can't be used. The DV plan relies on protocol-level scoreboarding: the BFM master records arrival order per ID, the BFM slave records issue order, and the scoreboard verifies the AXI4 per-ID ordering invariant is preserved. Within-ID ordering violations are a higher-severity bug than performance regressions and should be caught at Layer 1 unit tests, not deferred to integration.
- **Fairness tail matters under adversarial workloads.** With AGE_MAX as the only anti-starvation cap, pathological patterns can drive 99th-percentile

latency well above mean. The random-narrow and multi-master workloads in the benchmark suite intentionally exercise this; the mobile-mixed workload simulates the common case. The recommended default for AGE_MAX should be picked based on tail-latency targets, not throughput targets.

These concerns are unique to the OoO scheduler design choice — a simpler per-bank FIFO controller (the typical open-source pattern) would not face them, at the cost of leaving row-hit reordering performance on the table.

29.3 Verification Hooks

Internal observation outputs are exposed via CSR (see §6.3) for in-system characterization. Additionally, the controller emits internal “event” signals that can be tied to waveform-dump triggers:

- `ev_init_done`
- `ev_refresh_issued`
- `ev_page_conflict_detected`
- `ev_queue_full`
- `ev_refresh_pending_critical`

Specific events can be wired to a test-bench-controlled waveform-capture trigger during bring-up.

29.4 Coverage Targets

Functional coverage (collected during regression):

- All FSM states reached in each module
- All page policies exercised
- All refresh policies exercised
- All scheduler priority levels exercised
- All AXI burst types exercised (if enabled)
- All ADDR_MAP_SCHEME modes exercised

Code coverage targets: 100% line / branch coverage for all modules.

Assertion coverage: all design assertions (registered protocol checks) hit during regression.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

30 Open Issues and TODOs

Items where design choices were made provisionally and should be reviewed before the HAS is promoted to formal status.

30.1 Open Issues

30.1.1 1. AXI4 Narrow-Burst Handling Confirmation

AXI4 allows narrow transfers (data width less than bus width). The current plan is to handle them via byte-mask through `wrdata_mask`. **Confirm** the `wstrb-to-wrdata-mask` wiring honors the AXI partial-write semantics correctly, particularly for unaligned bursts.

30.1.2.2. AXI4 Read-Modify-Write Strobe Handling

Partial writes in DDR2 / LPDDR2 use `wrdata_mask` to mask byte lanes; this is straightforward. **Confirm** that per-byte masking is wired all the way through `wdata_path` and that `axi4_slave` doesn't widen narrow writes implicitly.

30.1.3.3. APB CSR Address Space Allocation

12-bit `paddr` provides a 4 KB region. **Confirm** the SoC's APB address-map allocation has at least 4 KB available, and that there's no conflict with adjacent peripherals.

30.1.4.4. Per-Bank Refresh Book-Keeping Cost

Keeping `last_ref_age` per bank costs one counter per bank. For 8 banks this is 8 small counters. **Confirm** the synthesis area impact is acceptable; we expect ~200 LUTs for the per-bank refresh tracking on a typical FPGA.

30.1.5.5. HAPPY Predictor Hash Function

Ghasempour 2015 suggests bank-XOR-low-row-bits as the hash function. **Confirm** with our typical workloads. If the workloads have systematic bank-row correlations, a different hash may be needed.

30.1.6.6. Self-Refresh Exit Latency

JESD79-2 requires `tXSNR` (~200 ns) before any command after SR exit. **Confirm** the power-state FSM enforces this correctly and that the scheduler is gated for the full duration.

30.1.77. Init Engine Retry on Error

If ZQ fails, do we retry up to 3 times then halt, or just halt? Current plan: 3 retries (parameter INIT_ZQ_RETRIES) then raise init_error. **Confirm** with the system architect — some systems prefer fail-fast.

30.1.88. CSR Write Atomicity

Multi-register parameter changes (e.g., updating both MR0 and a related timing) need a “config quiet period” to take effect atomically. **Document** the protocol the SoC software should follow; potentially expose a config_apply strobe.

30.1.99. Burst Splitting at Row Boundaries

AXI4 allows up to 256 beats per burst; a DRAM row contains $2^{\text{COL_WIDTH}}$ columns. Burst splitting happens in axi4_slave. **Confirm** the split logic correctly tracks partial-burst completion for ID-based ordering.

30.1.10 10. Multi-Rank

Explicitly out of scope for v1. **Note for v2:** add a RANK_WIDTH parameter, rank-aware bank machines, and rank-coordinated refresh.

30.1.11 11. ECC

Out of scope for this controller. ECC is expected to be handled at the SoC level or in a sideband wrapper, not the controller itself.

30.1.12 12. DFI Training Sub-Interface

Not driven in v1; the assumption is that the PHY handles its own training or training is not required for this DDR2 / LPDDR2 target. **Confirm** with the PHY vendor before silicon tape-out.

30.1.13 13. AXI Quality of Service (QoS)

Currently the awqos / arqos fields are forwarded to the scheduler as priority hints, but the scheduler does not implement explicit QoS classes. **Decide** whether to add proper QoS support (multi-class scheduler) or leave QoS as a future enhancement.

30.1.14 14. Multi-Master AXI

Currently single AXI port. **Decide** whether to add a multi-port AXI crossbar internally (simplifies SoC integration but adds area) or leave as a single-port with the SoC responsible for AXI arbitration upstream.

30.1.15 15. Observation Counter Overflow

Observation counters (row-hit, queue depth max, etc.) are 32-bit. At high traffic rates they could wrap. **Decide** on overflow handling: saturate, clear-on-read, or rely on SoC reading frequently.

30.2 TODOs Before v0.2

- Add bit-level pinout tables for the AXI, DFI, and APB interfaces
- Add timing diagrams for the WR / RD command issue, including CWL / CL alignment
- Add FSM state-transition diagrams for bank_machine, power_state, and init_engine
- Add a draft of the Verilog package skeletons for ddr2_init_steps_pkg.sv and lpddr2_init_steps_pkg.sv
- Cross-reference each section to the corresponding pre-aspec.md bullet
- Add quantitative area / power estimates per module (synthesis pass needed)
- Add waveform examples for the canonical init sequences